

Department of Physics
Ludwig-Maximilians-Universität München
in cooperation with
Technische Universität München



Master's Thesis

Data Driven Learning of Material Science from Composition

Datengetriebenes Lernen
der Materialwissenschaft aus
der chemischen Zusammensetzung

Joachim Czapiński

Internal Supervisor: Prof. Dr. Emiliano Cortés

External Supervisor: Prof. Dr. Helge Stein

January 2026

Submitted in partial fulfillment of the requirements for the degree of M. Sc.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, **Prof. Dr. Helge Stein**, for his excellent guidance, valuable discussions, and continuous support throughout this project. His insights were essential to the success of this work.

I would also like to thank my internal supervisor, **Prof. Dr. Emiliano Cortés**, for his willingness to oversee this thesis and for offering his support during the process.

I am also grateful to my friends and colleagues for the distractions, the laughter, and for keeping my spirits high during the challenging moments of this research, as well as for a warm welcome into the lab family.

I would be remiss not to mention my Parents and my Brother, thank you for developing my curiosity from the early stages of my life, your emotional support throughout my scientific career and for making this journey possible.

Finally, a very special thanks goes to my girlfriend and editor, Zuzia. Thank you for your unwavering belief in me and your amazing support.

Use of AI I acknowledge the use of Google's Gemini 3 Pro[1], Grammarly[2], and Google Translate[3] models under careful and strict supervision in some sections of this work for stylistic and linguistic aid, as well as for the acceleration of mundane tasks, such as generating large LaTeX tables.

Abstract

Modern materials science needs to keep evolving and finding new ways of discovery, as all the low hanging fruit has already been picked. In order to find novel, interesting or better materials, researchers need to go through enormous chemical spaces just to find candidates exhibiting desired properties. Various methods have been revolutionizing this branch of science over the last decades, enabling high-throughput discovery, both *in-vitro* and *in-silico*. After automation and computational methods (DFT), comes Machine Learning revolution. This paper contributes to this emerging paradigm, but introducing a generative model tasked with generating visual images of participated complex metal oxides. The novelty of the presented approach comes from:

1. Conditioning solely on composition of the sample – no crystallographic or other structural data.
2. Working on solely experimental data.

Predicting material properties from chemical composition represents a task that, if accomplished, would be revolutionizing for material discovery.

This thesis presents a Conditional Stable Diffusion Model (SDM) capable of generating material micrograph conditioned solely on chemical composition. To bridge the domain gap between chemical stoichiometry and visual morphology, four distinct embedding strategies were evaluated: NLP based Word2Vec embeddings, domain-specific graph representations MegNet, baseline ElemNet embeddings, and novel CCIP encoding using contrastive loss.

Quantitative evaluation on ternary material images reveals that Word2Vec embeddings significantly outperform alternatives, achieving the highest structural fidelity (SSIM ≈ 0.79) and the most accurate color reproduction (CIEDE2000 $\Delta E^* \approx 7.24$). In contrast, the graph-based MegNet encodings (also the smallest) performed the worst, resulting in a 68% deterioration in color accuracy. These findings suggest that the rich, high-dimensional highly abstract relationships captured by unsupervised text embeddings were superior for visual generation compared to lower-dimensional, property-optimized features. The CCIP model was a close second behind Word2Vec; a possible bottleneck was identified, pointing to a possibly better encoding model, capable of beating the Word2Vec approach in the presented use application.

Leveraging this generative capability, a high-throughput "Materials Funnel" was established to screen 11,480 ternary composition spaces. By integrating the SDM with a semi-supervised Synthesizability-stoi-CGNF synthesizability gauging model, interesting materials were flagged.

In addition to the image generator a UV-Vis predictor was developed, a simple Feed Forward Neural Network. Which goal was to predict spectra based on composition. This model achieved an median R^2 score of 0.9168, and generalized well onto most of the unseen compositional space, leaving out some outliers for which the prediction collapsed – as well in the seen and unseen spaces.

Contents

1	Introduction	1
1.1	Background	1
1.2	Thesis Structure	2
2	Theoretical Foundations	3
2.1	The Landscape of Complex Metal Oxide Systems	3
2.1.1	The Ubiquity and Versatility of Oxides	4
2.1.2	Defining Complex Metal Oxides	4
2.1.3	The Combinatorial Challenge	4
2.2	Computational Materials Discovery: The First Paradigm Shift	5
2.2.1	The Quantum Mechanical Basis	5
2.2.2	Density Functional Theory (DFT)	5
2.2.3	High-Throughput Screening	6
2.2.4	The Scaling Bottleneck	6
2.3	Machine Learning: The Second Paradigm Shift	7
2.3.1	From Explicit Programming to Learning	7
3	Theoretical Foundations of Machine Learning	9
3.1	What is Machine Learning – A Broad Introduction	9
3.2	Learning Algorithms	13
3.2.1	The Task T	13
3.2.2	The Performance Measure P	13

3.2.3	The Experience E	14
3.3	Data and Representations	15
3.3.1	Structure of Data	15
3.3.2	Feature Spaces	15
3.3.3	Representations	16
3.3.4	Quality of Data	16
3.3.5	Dimensionality and Information Compression	16
3.4	Optimization and Training	16
3.4.1	Objective function	17
3.4.2	Gradient-Descent Optimization	17
3.4.3	Backpropagation	18
3.4.4	Model Generalization	19
3.4.5	Data Splits	19
3.5	Building Blocks of Neural Networks	20
3.5.1	Neurons	20
3.5.2	Activation functions	20
4	Generative Models	23
4.1	Probability Theory Behind Diffusion	23
4.1.1	Evidence Lower BOund (ELBO)	24
4.2	Variational Autoencoders (VAE)	26
4.2.1	Standard Variational Autoencoders (VAE)	26
4.2.2	Hierarchical Variational Autoencoders (HVAE)	27
4.3	Variational Diffusion Models (VDM)	28
4.3.1	Minimization of the Denoising Term	31
4.3.2	The Denoising Network Architecture: U-Net	33
4.4	Additional Components of Denoising Networks	35
4.4.1	Attention is All You Need	35
4.4.2	Residual Blocks	36

4.4.3	Encoding Strategies	37
4.4.4	Noise Scheduling	40
4.5	Stable Diffusion	41
4.5.1	Perceptual Compression	42
4.5.2	Conditioning with Text Encoders	43
4.5.3	Time Conditioning	45
5	Dataset	49
5.1	Sample Synthesis	49
5.2	Data Structure	49
5.3	Statistical Analysis of the Features	52
5.4	Absorption Spectra	54
5.4.1	Statistical Analysis of Optical Properties	54
5.4.2	Estimation of Optical Bandgaps	55
5.4.3	Principal Component Analysis of the spectral data	56
5.5	The Image Data	58
6	Models' Implementation and Training	61
6.1	Data Preprocessing and Splitting	61
6.2	Latent Diffusion Generative Model	63
6.2.1	Architecture Overview	63
6.2.2	Variational Autoencoder	64
6.2.3	Building Blocks of the Variational Autoencoder	67
6.2.4	VAE Training	69
6.2.5	Composition Encoding	70
6.2.6	Diffusion Architecture and Dynamics	76
6.2.7	Latent Diffusion Training Protocol	78
6.3	Spectral Prediction Network	78
6.3.1	SP Encoding Scheme	78
6.3.2	SP Architecture	79

6.3.3	SP Training Protocol	79
7	Results and Discussion	81
7.1	Variational Autoencoder (VAE) Evaluation	81
7.1.1	Reconstruction Fidelity	81
7.1.2	Latent Space Analysis	81
7.1.3	Latent Continuity	82
7.1.4	Quantitative Metrics and Generalization	83
7.2	Evaluation of Compositional Embeddings (CCIP)	84
7.2.1	Cross-Modal Retrieval Performance	85
7.2.2	Chemical System Identification	85
7.3	Generative Modeling via Latent Diffusion	86
7.3.1	Qualitative Analysis	86
7.3.2	Evaluation Metrics for Quantitative Analysis	86
7.3.3	Overall Performance Summary	91
7.3.4	Color Accuracy Analysis	92
7.3.5	Structural Quality Analysis	96
7.3.6	Key Findings and Implications	97
7.4	Material Funnel	99
7.4.1	Synthesizability Screening and Experimental Validation	99
7.5	Spectral Predictor	100
7.5.1	Hyperparameter Optimization	100
7.5.2	Model Performance	101
7.5.3	Discussion	104
8	Conclusion and Outlook	105
8.1	Conclusion	105
8.1.1	NLP Embeddings Superiority	105
8.1.2	The Embeddings' Shortcomings	106
8.1.3	Materials Funnel	106

8.1.4 Spectral Predictor	106
8.2 Outlook	106
A Additional Materials	115
A.1 Jensen's Inequality	115
A.2 Distribution of a Sum of Independent Gaussian Variables	116
B Algorithms	117
B.1 Bandgap Extraction Using Tauc Plot	117
B.2 AdamW optimizer	118
B.3 CIEDE2000 Color Difference Formula	120
B.3.1 Interpretation and Thresholds	121
C Additional Results	123
C.1 The Generative Funnel Fractions	123
C.2 Interesting Composition Spaces	125

Introduction

1.1 Background

The proliferation of Machine Learning (ML) has, without a doubt, had an impact on many aspects of everyday life, from the most mundane to the most complex. The Artificial Intelligence (AI) revolution is said to be 'unprecedented' in human history [4], touching almost every aspect of life. Scientific research is no exception; some examples include the following:

- **Catalyst & Material Design**
 - Automated catalyst discovery [5]
 - Fast simulation [11]
- **Condensed Matter & Materials Science**
 - Crystal structure prediction [6]
 - Potential energy surfaces [7]
 - Phase transition detection [8]
- **Particle & High-Energy Physics**
 - Event classification [9]
 - Track reconstruction [10]
- **Astrophysics & Cosmology**
 - Galaxy morphology classification [12]
 - Gravitational wave detection [13]
 - Cosmological parameter estimation [14]
- **Biology, Biotechnology**
 - Protein structure prediction (AlphaFold [15, 16])
 - Drug discovery [17]

These are just some examples, but across virtually all branches of the (not only) natural sciences, there is active research integrating machine learning techniques to advance understanding, analysis, and prediction. One of the branches where ML has been increasingly disruptive is material science.

The quest for novel materials with precisely engineered properties is the foundation for pushing the envelope in many fields of science, e.g. catalyst research, electronics, photovoltaics, and a multitude of others. Material discovery has traditionally been a tedious, 'Edisonian' process requiring intuition of experienced researchers, many sleepless hours spent in the lab, and a lot of trial and error. The first 'revolution' in the field took place, when computers became powerful enough to allow for the use of ab-initio methods like: Density Functional Theory (DFT), Post-Hartree-Fock (Post-HF), and Ab Initio Molecular Dynamics (AIMD). This

allowed for first in-silico aided research. And the first major paradigm shift in the approach to Material Science. However, these methods, as revolutionizing as they were, were computationally very intensive, making them unfeasible for researching large material-spaces without any prior assumptions. This is where the Machine Learning methods have already started to disrupt the field of materials discovery, bringing the second paradigm shift in. The ML methods can be applied to the majority of steps in the discovery process, from predicting material properties and screening candidates to guiding experimental synthesis and optimizing performance.

The main focus of this research is to develop and validate a robust machine learning framework capable of predicting formation of new, stable phases in a combination of three metal elements and oxygen. This thesis argues that by integrating physics-informed feature engineering with a carefully curated dataset, and a bespoke machine learning model; a high-throughput pre-screening tool can be obtained, highly speeding up the material discovery and guiding the subsequent DFT calculations, or synthesis.

1.2 Thesis Structure

This thesis is structured as follows:

- **Chapter 2, "Theoretical Foundations,"** provides a comprehensive overview of the core concepts underpinning this work. It begins by discussing the chemical and physical importance of ternary metal oxides. It then briefly reviews established *ab-initio* computational methods like Density Functional Theory (DFT).
- **Chapter 3, "Theoretical Foundations of Machine Learning,"** provides an in-depth introduction of machine learning methods needed to understand the methods utilised in the following chapters.
- **Chapter 4, "Generative Models"** is an in-depth introduction into generative models relevant for this study, giving a good understanding of this branch of machine learning.
- **Chapter 5, "Dataset,"** provides the reader with the information about the dataset used in the study and data manipulation methods employed.
- **Chapter 6, "Models' Implementation and Training,"** showcases a detailed architecture of the models, as well as the detailed training schemes utilized to obtain the best possible predictors and generators.
- **Chapter 7, "Results and Discussion,"** presents the performance of the developed model. It analyzes the model's generative accuracy and ability to generalize, using a suite of statistical metrics. This section delves into a feature importance analysis. The chapter culminates in the application of the model as a high-throughput screening tool as a top-down method to identify novel, high-probability ternary oxide candidates.
- **Chapter 8, "Conclusion and Future Outlook,"** summarizes the key achievements of this thesis, reiterating how the developed ML framework successfully functions as a pre-screening tool to accelerate materials discovery. It discusses the current limitations of the model and concludes by proposing promising directions for future research.

Theoretical Foundations

Chapter 2 establishes the central premise of this thesis: that the modern discovery in the materials space requires data-driven methods for effective exploration. This chapter introduces the key scientific and computational foundations that form the basis needed for understanding the methods and challenges discussed in this thesis are introduced. The main questions it addresses are:

1. What are complex metal oxide systems, and what is their significance?
2. How have materials been computationally discovered before the widespread adoption of machine learning?
3. What is machine learning? (briefly, expanded in Chapter 3)
4. How can machine learning be used in material discovery?

By reviewing these core concepts, this chapter aims to prepare the reader for what awaits them in the following chapters.

2.1 The Landscape of Complex Metal Oxide Systems

Oxides are the most abundant materials in the Earth's crust, arguably they are also the most diverse class of materials (At least in an oxygen based atmosphere). This class of materials has been crucial for humans for millennia in every aspect of life, from arts and culture in form of pigments and ceramics, to SiO_2 powering the informational revolution or superconductors, that allow for the functionality of levitating trains

The reason for such an abundance of metal oxides is their very high chemical stability. The oxidized state is the lowest free energy state for most elements found on earth. This is due to the tendency of chemical elements to achieve noble gas electron configuration, and abundance of oxygen in the Earth's atmosphere.

2.1.1 The Ubiquity and Versatility of Oxides

Chemically speaking oxides are compounds in which at least one oxygen atom is combined with another element(s), their general formula can be written as:

$$M_xO_y, \quad (2.1)$$

where M is the metal, O – oxygen, and x/y their ratio.

As previously mentioned, oxides are a very diverse group of compounds: in terms of conductivity they span from conductors (ITO) through semiconductors (ZnO, W_3O) to insulators (Al_2O_3), and high temperature superconductors (YBCO) [18]; their magnetic properties span from paramagnetic (KO_2) through ferromagnetic (Fe_3O_4) to diamagnetic (ZnO).

Furthermore, their optical properties range from transparent to opaque meaning their band gaps vary to a great extent; their catalytic activity is the backbone of chemical synthesis. This unmatched functional diversity stems from the complex interactions within the crystal structure, the electronic configuration of the metal cations, and the nature of the metal-oxygen bonds.

2.1.2 Defining Complex Metal Oxides

While binary oxides (containing a single metal element) exhibit remarkable utility, the possible compounds grow exponentially when introducing multiple cationic species. These materials are referred to as **Complex Metal Oxides (CMOs)**. These possibilities give rise to emergent phenomena within these spaces.

Generally, a complex oxide can be described by the formula:

$$A_xB_yO_z, \quad (2.2)$$

where A and B represent different metal cations. The A-site is typically occupied by a larger, electropositive cation (e.g., alkali or alkaline earth metals like Ba^{2+} , Sr^{2+}), while the B-site usually hosts a smaller transition metal cation (e.g., Ti^{4+} , Mn^{3+}) capable of variable oxidation states. However, there are exceptions to this rule.

The significance of CMOs lies in their tunability. By doping the oxides with different elements, one can subtly distort the crystal lattice or alter the electron structure, leading to drastic modification of the materials' macroscopic properties. This makes CMOs a great space for discovering new, exciting materials, with unexpected properties like high temperature super conductivity (e.g., $La_{2-x}Ba_xCuO_4$) and very big magneto resistance in manganites (e.g., $La_{1-x}Sr_xMnO_3$), but the possibilities are endless.

2.1.3 The Combinatorial Challenge

The very flexibility that makes complex metal oxides so scientifically attractive also presents a great challenge for materials discovery. The compositional space is vast or even enormous. If we consider a simple ternary perovskite ABO_3 and select candidates from roughly 50 possible metal elements for the A and B sites, the number of unique combinations is already in the thousands ($50 \times 50 = 2500$).

However, modern materials engineering rarely stops at stoichiometric ternary compounds. We frequently employ solid solutions (e.g., $A_{1-x}A'_x B_{1-y}B'_y O_3$) to fine-tune properties. Once we account for continuous compositional variables (x, y), quaternary and quinary systems, and the potential for polymorphs, the search space becomes effectively infinite.

This explosion of possibilities makes the traditional trial-and-error experimentation obsolete. It creates a critical need for methods that can efficiently navigate this high-dimensional landscape – motivating the shift toward computational screening and, more recently, data-driven machine learning approaches discussed in the remainder of this work.

2.2 Computational Materials Discovery: The First Paradigm Shift

Before the advent of data-driven methods, the primary approach for accelerating materials discovery were *ab initio* simulations – methods able to predict material properties solely from quantum mechanical laws, without empirical data. This approach fundamentally transformed materials science from a purely experimental discipline into a predictive one.

2.2.1 The Quantum Mechanical Basis

At the core of computational materials science lies the many-body Schrödinger equation. For a system of N electrons and M nuclei, the non-relativistic time-independent equation is given by:

$$\hat{H}\Psi(\mathbf{r}_1, \dots, \mathbf{r}_N, \mathbf{R}_1, \dots, \mathbf{R}_M) = E\Psi(\mathbf{r}_1, \dots, \mathbf{r}_N, \mathbf{R}_1, \dots, \mathbf{R}_M), \quad (2.3)$$

where $\hat{H}$ is the Hamiltonian operator representing the total energy (kinetic and potential), E is the energy eigenvalue, and Ψ is the wavefunction.

While exact, this equation is computationally infeasible for any system larger than a few atoms due to the complex electron-electron interactions. The computational cost scales exponentially with the number of electrons, making direct solution impossible for bulk crystals such as complex metal oxides.

2.2.2 Density Functional Theory (DFT)

The breakthrough solution to this many-body problem came with the development of **Density Functional Theory (DFT)** by Hohenberg, Kohn, and Sham in the 1960s [19, 20]. DFT simplifies the problem by proving that the ground state energy is a unique functional of the electron density, $n(\mathbf{r})$, rather than the high-dimensional wavefunction.

This reduction from $3N$ dimensions to just 3 spatial dimensions allows the system to be modeled as a set of non-interacting electrons moving in an effective potential. The resulting Kohn-Sham equations can be solved iteratively:

$$\left[-\frac{\hbar^2}{2m} \nabla^2 + V_{\text{eff}}[n(\mathbf{r})] \right] \psi_i(\mathbf{r}) = \epsilon_i \psi_i(\mathbf{r}), \quad (2.4)$$

where the effective potential V_{eff} includes the external potential from nuclei, which are approximated to be stationary (Born-Oppenheimer approximation), the classical Coulomb interaction between electrons, and the exchange-correlation potential (V_{XC}).

For complex metal oxides, DFT has been one of the most important tools, pushing the envelope of material discovery; the field would not be where it is without it. It allows researchers to calculate lattice parameters, band gaps, and magnetic moments with high accuracy. There is one, however, very important drawback to this method – it is incredibly computationally expensive.

2.2.3 High-Throughput Screening

With the development of more powerful DFT software (e.g., VASP[21], Quantum ESPRESSO[22]) and the exponential growth of computing power, the 2000s and 2010s saw the emergence of High-Throughput (HT) Computing, allowing for much faster and accurate quantum calculations.

Instead of simulating a single material to explain an experiment, researchers could simulate hundreds of hypothetical structures to find optimal candidates. This ‘screening’ approach generally follows a funnel structure:

1. **Generation:** A combinatorial engine generates thousands of candidate structures according to a given recipe (e.g., swapping B-site cations in a perovskite lattice).
2. **Geometry Optimization:** DFT relaxes the atomic positions to find the local energy minimum.
3. **Property Calculation:** Key metrics (formation energy, band gap, etc.) are computed.
4. **Filtering:** Candidates that are thermodynamically unstable or lack desired properties are discarded.
5. The desired materials can be worked on further, computationally or experimentally.

This paradigm shift led to the creation of massive open-access databases e.g. the Materials Project [23]. These repositories aggregate calculated properties for hundreds of thousands of compounds, providing the structured data, making it possible for researchers to share their results in a structured manner and eventually enabling the machine learning revolution discussed in the next section.

2.2.4 The Scaling Bottleneck

Despite its success, HT-DFT faces a hard limit. The computational complexity of standard DFT scales as $\mathcal{O}(N^3)$ with respect to the number of atoms N . While sufficient for unit cells containing <100 atoms, it becomes prohibitively expensive or even unfeasible for more complex systems.

Given that the combinatorial space of complex oxides is effectively infinite, simulating every possible combination with DFT is impossible. This realization created the demand for a surrogate model – a method that could approximate the accuracy of DFT but at a fraction of the computational cost. This is where machine learning enters the picture.

2.3 Machine Learning: The Second Paradigm Shift

While the first paradigm shift moved materials science from experimentation to *ab initio* simulation, the second shift – currently underway – tries to utilize data-driven approaches. If Density Functional Theory (DFT) represents a **deductive** approach (deriving properties from established physical laws), Machine Learning (ML) represents an **inductive** approach (inferring patterns and rules from observed data).

2.3.1 From Explicit Programming to Learning

In traditional computational materials science, the researcher explicitly programs the physical rules (e.g., the Kohn-Sham equations) that map an input structure x to a property y . The computer merely executes these instructions.

In contrast, Machine Learning algorithms do not rely on explicit physical equations to make predictions. Instead, they construct a mathematical model based on a dataset of examples. Machine Learning is so much faster mainly due to two reasons: 1. It doesn't need to simulate the whole system, just find the desired properties – scaling linearly. 2. It can utilize the GPU speedup and parallelization.

This training of machine learning models and its application to high-throughput search for novel materials is the goal of this thesis.

Theoretical Foundations of Machine Learning

The goal of the following chapter is to introduce the concepts needed to grasp the computational aspect of this thesis. First, the underlying concepts of machine learning will be introduced in order to accustom the reader with the terminology and provide broad understanding needed to understand the basics of any ML model. Subsequently in Chapter 4, concepts more specific to this work will be introduced i.e. autoencoders, variational autoencoders, diffusion models, generative machine learning.

3.1 What is Machine Learning – A Broad Introduction

Machine Learning (ML) is a branch of Artificial Intelligence (AI) which in turn is a subfield of computer science. The main aim of AI is to make a machine (computer) capable of performing tasks native to human intelligence, e.g. learning, reasoning, perception, decision-making, and problem-solving. Machine Learning is currently most prominent and most studied branch of AI. Keeping in mind that the boundaries between them are very blurred, one can distinguish the following subfield of AI:

- **Machine Learning** which goal is to learn patterns from data and make predictions without explicit rules.
- **Deep Learning** (subfield of ML) – it uses multi-layer neural networks to learn complex representations in data.
- **Natural Language Processing** – the idea of translating human language to computer language, so that machine translation, sentiment, and text summarization can be achieved.
- **Computer Vision** – the ability of machines to interpret and understand content of images and videos. Used in image classification or object detection.
- **Reinforcement Learning** – agents making decisions based on some given constraints, for example: machines capable of playing games (AlphaGo [24]), simulating complex biological systems, or for use in robotics.

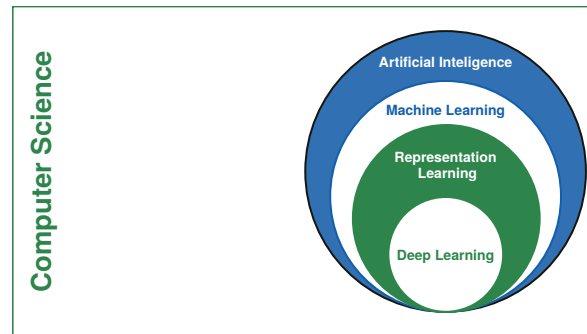


Figure 3.1: Hierarchical relationship between AI, ML, Representation Learning, and Deep Learning. Deep learning is a specific form of representation learning within machine learning, which in turn is a subset of artificial intelligence., image adapted from [25]

This is by no means an exhaustive list. These examples are brought up only to introduce to the reader how broad and constantly developing field AI is. Keeping in mind the fluidity and ambiguity of terms in this field, a broad map, in order to help the reader navigate the concepts introduced in this work will be sketched in the following paragraphs.

As mentioned before, Machine Learning is a subfield of Artificial intelligence (see fig. 3.1). The main concept of machine learning is the idea, that a computer, rather than being explicitly programmed, learns from the data. Based on the 'acquired knowledge' about the data, it can make predictions about unseen data. A machine learning algorithm, in a very reductionist view, identifies patterns and relationships in the data during training, and then, based on what it has learned makes predictions about the unseen data.

At its core, ML is an approach that teaches computers how to **generalize**. The learner, not dissimilar to humans, first learns based on given examples – the **training set** – and then applies the gathered knowledge to predict properties of unseen data **test set**. Classical ML algorithms include: logistic regression, decision trees, Support Vector Machines (SVM), naïve Bayes. These are the methods that lay foundation to the modern, more complex (yet less interpretable) modern machine learning methods. The fundamental issue of classic ML algorithms is that the performance of these methods is strongly dependent on the representation of the data they are provided with.

As an example of the shortcoming of such an approach, lets imagine a scenario where a logistic regression is used to classify patients in a cancer risk group, the algorithm is provided with data about a patient, that is curated by the researcher. Each information included in the representation is known as **feature**. Simple logistic regression learns how **each** of these features correlates with the 'cancer' class. However if the algorithm was trained on an MRI image of the potentially cancerous tissue, it would not be able to classify the cases successfully. This is because the individual pixels have a very low correlation with the predicted class – there are correlations between the features that need to be captured by the model. Even when a trained professional could easily identify cancer from an image like this, such an basic algorithm would be unable to do so.

Many machine learning tasks can be solved by smart choice of features and simple machine learning model. For vast number of tasks, however, the choice of representation plays a crucial role in the machine learning process. In Figure 3.2 a simple visual example of this phenomenon is shown. The simple choice of weather we input our data in cartesian or polar coordinates can make or break the predictive ability of the model.

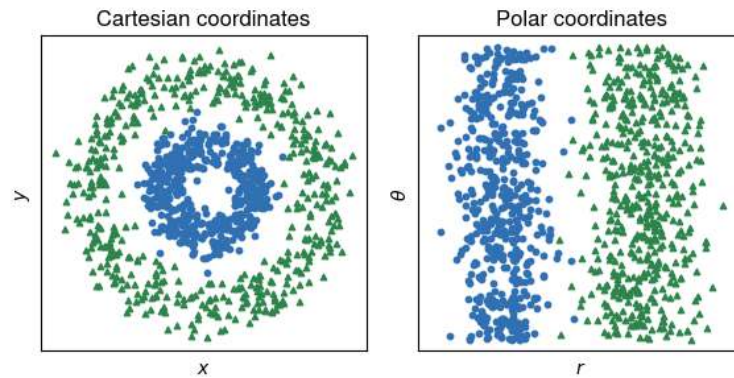


Figure 3.2: Example of different representations. Both plots represent the same data, one in cartesian coordinates, and the other – in polar. Two classes are represented by blue circles and green triangles. If we wanted to separate the two classes using one line, its impossible in the first case, and trivial in the second one. Figure adapted from [26]

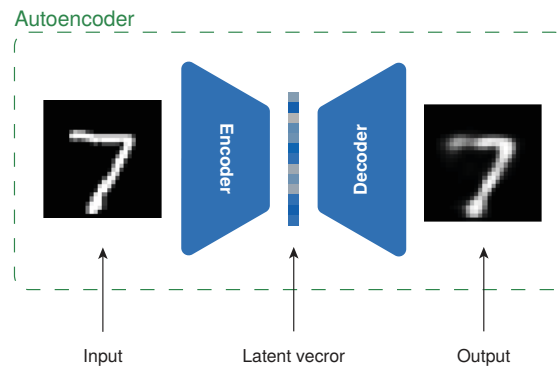


Figure 3.3: Simple visualization of an autoencoder. It is compromised of input – the original data, encoder – a ML script (neural network) that creates a latent vector, which is a compressed version of the original data, where the most important features are extracted, this latent vector can be used in further machine learning tasks. The next component is the decoder, which reconstructs the data to its original form from the latent vector

The solution is to use machine learning to uncover not only the mapping from the representation, but the representation itself as well. This is what is known as **representation learning** and is the next logical step after simple machine learning models. This approach allows for even less of human intervention into learning, and therefore, less probability of human error. Moreover, this approach allows for the use of machine learning in new class of problems, not possible to tackle with the simple ML methods. A very basic example of representation learning is extracting velocity of a particle from location and time data. other examples include convolutional neural networks, **autoencoders** (see fig. 3.3), which are important building blocks of modern machine learning models and will be explained in detail in the following chapters. For now, it is enough to know, that an autoencoder is a combination of **encoder** function that converts data into different representation, and a **decoder**, that converts the new representation back. The goal of autoencoders is to preserve as much information about the original data as possible, while achieving various goals within the new (called latent) representation.

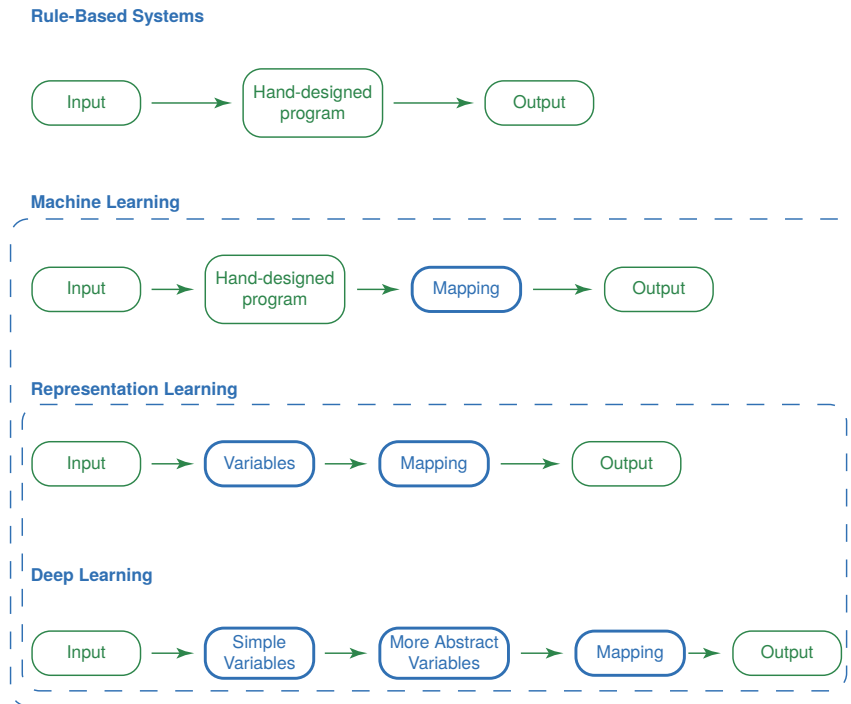


Figure 3.4: Comparison of learning paradigms. The figure illustrates the evolution from traditional rule-based systems to modern deep learning approaches. In rule-based systems, the program and rules are explicitly hand-crafted. Machine learning automates the mapping from input features (still hand-designed) to output. Representation learning further automates the discovery of useful variables or features from data. Deep learning extends this idea by hierarchically learning increasingly abstract representations directly from raw input, enabling end-to-end learning.

A big issue that engineers and researchers face, when trying to utilize representation learning in their applications, is the separation of the **factors of variation**. They are the independent latent variables that explain variation in data. A good example of such factors is the variability of human voices. Lets imagine that our task is to create a system, that will perform speech recognition. We would like, however, our system to be able to recognize the speech of both a person with a Scottish, or Jamaican accent, it should be transcribe, wether a girl or a man is speaking. If our task is to recognize cars in an image, we would like the program to work regardless of the color of the car, the angle at which it is in the picture, or the time of day.

Deep learning is the answer to this central problem in representation learning. It inherently builds a much more complex representations out of simpler ones, hierarchically. The quintessential and most basic example of deep learning is **MultiLayer Perceptron (MLP)**, or **Deep Neural Network (DNN)** which consists of many **neurons**, connected in a complex web. The neurons are arranged into **layers**, within one layer the data from previous layers is processed, to finally produce an output. DNNs are a special case of **Artificial Neural Networks (ANN)** with more than three hidden layers. On Figure 3.4 the different learning paradigms of the concepts presented up-to now in this introduction can be found.

3.2 Learning Algorithms

As mentioned before, a machine learning algorithm is an algorithm that is able to learn from data, and infer what it learned on unseen data. The aim of this section is not to provide the reader with a compendium about machine learning methods, but to give an intuition needed to understand the more complex models that are yet to come. Following Mitchell [27] we will introduce fundamental concepts needed for understanding machine learning methods:

- the Task T ,
- the Performance Measure P ,
- and the Experience E .

Mitchell provides a very concise description of machine learning: 'A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P of its performance at tasks in T , as measured by P , improves with experience E .' [27] Let us break it down.

3.2.1 The Task T

This is the problem that we are trying to tackle using the machine learning methods. These tasks are typically problems that are difficult or impossible to solve with hand designed programs. There is a multitude of tasks that can be solved using machine learning methods, I will introduce two most fundamental ones in detail here:

- **Regression:** Its a type of supervised learning task where the goal is to predict a continuous numerical value based on the input data. Formally, such an algorithm tries to learn a mapping function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, where n is the number of input features. Examples may include: estimating temperature based on weather data, estimating the location of an earthquake based on seismological data, or predicting material's bandgap based on its properties.
- **Classification:** In this task the goal is to specify to which one of m discrete categories the input to. Formally, such an algorithm tries to learn a mapping function $f : \mathbb{R}^n \rightarrow \{1, \dots, m\}$, where n is the number of input features. Examples may include: deciding whether an email is spam or not, or predicting whether a material is a conductor, semiconductor, or insulator.

Other examples of machine learning tasks are: classification with missing inputs, transcription, machine translation, anomaly detection, denoising, or generation.

3.2.2 The Performance Measure P

Evaluating how well a machine learning model performs is essential to judge its effectiveness, and generalization. The choice of a quantitative performance measure depends on the task at hand. For instance for regression we would use different metrics than for classification. Some examples of performance measures are introduced here:

- **For regression:** In regression tasks the goal is for the predicted output be as close to the true value as possible. The metrics include: Mean Absolute Error (MAE): $MAE = 1/n \sum_{i=1}^n |y_i - \hat{y}_i|$, where n is the number of samples, y_i and $\hat{y}_i$ are the true and the predicted value of the i -th sample; Mean Squared Error (MSE): $MSE = 1/n \sum_{i=1}^n (y_i - \hat{y}_i)^2$; Root Mean Squared Error (RMSE); Coefficient of Determination (R^2)
- **For classification:** In classification tasks the model predicts discrete categories. the goal in this class of problems is to correctly assign each sample to its correct class. The metrics include: Accuracy – the proportion of correctly classified samples; Precision and Recall; F1-Score; Confusion Matrix.

In reality choosing the right metric is a lions part of designing a well performing machine learning model. The choice is often not as trivial as one might expect.

3.2.3 The Experience E

Table 3.1: The iris dataset with marked features and label [28]. The dataset contains 150 datapoints of which half belongs to the class labeled as *I. setosa*, and half to *I. virginica*

No.	Features				Label
	Sepal length	Sepal width	Petal length	Petal width	Species
1	5.1	3.5	1.4	0.2	<i>I. setosa</i>
2	4.9	3.0	1.4	0.2	<i>I. setosa</i>
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
150	5.9	3.0	5.1	1.8	<i>I. virginica</i>

The most general distinction of machine learning models is whether they are **supervised**, **unsupervised**, or **reinforced**; this corresponds to the experience a machine learning model can have while learning.

- In **supervised learning**, the dataset, as well as features, also contains **targets (labels)** associated with the samples. One of the widely used datasets by statisticians and data-scientists, the Iris dataset [28], includes multiple properties of iris plants as features, and the name of species as label (see table 3.1). This dataset is a great example of how supervised learning experience could be used for classification. A supervised learning algorithm can use the Iris dataset to be trained to recognize whether a plant is *I. setosa*, or *I. virginica* based on the dimensions of their flowers.
- In **unsupervised learning**, the dataset contains only features, without any corresponding **labels** or **targets**. The goal is to uncover hidden patterns, relationships, and structures within the data itself. A classic example would be **clustering** of the iris dataset, this time **without providing the species names** (labels). An unsupervised algorithm such as k-means clustering, can analyze the features and solely based on these measurements group them into distinct clusters only by finding similarities in the feature space. Interestingly, these clusters often correspond closely to the actual species, and classifications based on supervised learning.
- In **reinforcement learning**, the dataset does not consist of fixed input-output pairs. Instead an **agent** learns by interaction with an environment, making decisions, and

receiving feedback in the form of rewards and penalties designed by the researcher. The agent's goal is to **maximize the total reward** by discovering an optimal strategy, or **policy**. An example of reinforcement learning is AlphaGo, a program developed at Google's Deep Mind Lab [24], where the agent was trained to play the game of go, a much more complex game – at least for a computer – than chess, where the rule-based or brute force approach, like in Deep Blue, would not work.

Supervised and unsupervised learning are not formally well defined terms, and the borders between them are blurred, with intermediate steps like semi-supervised learning or self-supervised learning.

3.3 Data and Representations

A well structured dataset is often as important if not more, as the algorithm itself. Therefore, in this section we will introduce the general concepts of curating, constructing, spiting, and all together working with datasets and representations.

Before an algorithm can learn, the raw data – whether images, spectra, measurements, or words – must be encoded in a way a computer can understand. This means it needs to be translated into numerical, well structured form that captures its relevant characteristics.

3.3.1 Structure of Data

Most machine learning methods operate on datasets consisting of n samples

$$\{\vec{x}_i, y_i\}_{i=1}^n, \quad (3.1)$$

where each sample i consists of a vector of **features** $\vec{x}_i$, and (if present) an associated **label** or **target** y_i . As mentioned in Section 3.2.3, in general sense, the presence of targets differentiates between the supervised and unsupervised learning.

In many applications a single observation may correspond to a complex, multidimensional measurement. For example an optical spectrum, image data, or atomic configuration. The data has to be encoded into numerical features; intensities, pixel values, or coordinates; that can be used by the algorithm. Optionally the data can be normalized, cleaned, or further processed.

3.3.2 Feature Spaces

The samples are represented numerically in a space called **feature space**. Each dimension of this space corresponds to one measurable property or derived characteristic. In the iris dataset (see table 3.1) the dimension of the feature space is $D = 4$, the features are: sepal length, sepal width, petal length, and petal width. The MNIST dataset [29], on the other hand, consists of 70 000 28×28 images of handwritten digits. Therefore, the dimension of the feature space of the MNIST dataset would be $D = 28 \times 28 = 784$. The process of feature engineering – selecting or constructing effective features – has historically been very important part of the machine learning process. Now with proliferation of representation- and deep-learning, while still significant, it is not as crucial as it used to be.

3.3.3 Representations

A representation is a specific way of expressing data that highlights useful patterns or relationships within it, similarly to the example of cartesian and polar coordinates in Figure 3.2. While the traditional machine learning methods required manually designed features, and manual representation generation, modern DL frameworks learn the more complex representations automatically and in a hierarchical fashion (see fig. 3.4). This representation learning process allows the model to discover latent variables that capture abstract factors of variation within the data.

A good example of this hierarchical approach is the processing of images by deep learning frameworks. In the lower layers of a neural network local edges or textures might be encoded and recognized, leading to more complex structures being represented in the deeper layers such as shapes or object categories.

3.3.4 Quality of Data

The representation's quality, and therefore the ability to generalize by the model depends strongly on the quality of underlying data. Noisy, imbalanced or biased datasets can lead to distorted representations, which in turn propagate the errors throughout the learning process.

Ensuring data diversity, accurate labeling, unbiased approach, or employing unsupervised methods to learn about the underlying structure of the data is therefore crucial for a well functioning machine learning model.

3.3.5 Dimensionality and Information Compression

High dimensionality in data can be thought of a virtue – we have more features to base our learning on. However, most of the times, in high-dimensional data some of the features can have very low correlation with the targets or be just noise for our model. In high dimensional spaces we have to deal with the *curse of dimensionality* – this are various phenomena that lead to problems in high dimensional spaces, such as increased sparsity of the data. More features also increase the computation times, they can 'overcrowd' the model, so that it doesn't pay the enough 'attention' to the meaningful features. Therefore, frequently, for high-dimensional feature spaces, dimensionality reduction techniques are useful. Such techniques include: Principal Component Analysis (PCA) – mainly for visualization, autoencoders, or variational autoencoders. These techniques can project the data into lower-dimensional latent spaces that preserve the essential structure behind the data while removing redundancy. These methods as well as useful for computations, also can aid in visualization of the intrinsic geometry in the datasets.

3.4 Optimization and Training

At the core of every machine learning task lies an optimization problem, namely, finding the optimal parameters, given the model, that best explain the data.

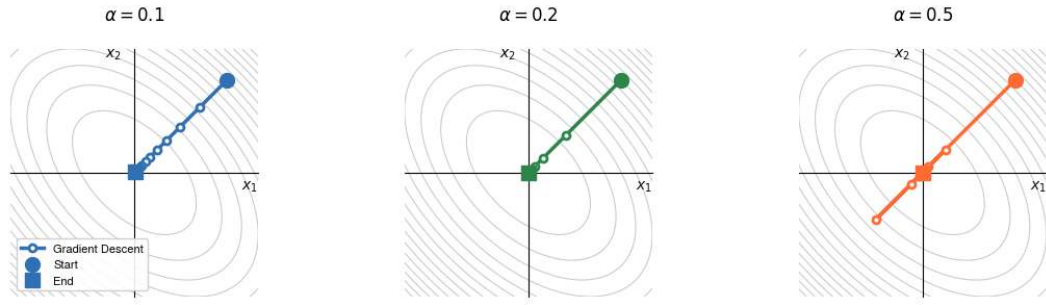


Figure 3.5: Visualization of gradient descent optimization for a function $f(x_1, x_2) = x_1^2 + x_2^2 + x_1x_2$. The three plots vary only by the learning rate α , smaller alpha $\rightarrow$ smaller steps

Optimization is the task of minimizing or maximizing a given function $f(x)$ by adjusting x . Traditionally most optimization problems are phrased in terms of minimizing $f(x)$

The function being optimized in machine learning is called the **criterion** or the **objective function**. When we are strictly talking about minimization it can also be called **cost function**, **loss function**, or **error function**. Whatever its name, this function is a quantitative measure of how far the model's predictions deviate from the desired outcomes i.e. labels.

Training a model involves iterative adjustment of its parameters in order to minimize the loss function. This section introduces the principles of optimization and familiarizes the reader with some of the techniques used for optimization in machine learning.

3.4.1 Objective function

Let a model be defined by parameters θ , and a function $f_\theta(x)$ be the model function, mapping the inputs x to outputs (predictions). The learning process optimizes the parameters in order to minimize a scalar cost function $\mathcal{L}(\theta)$ frequently expressed as the average over the training set (for scaling purposes):

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(x_i), y_i), \quad (3.2)$$

where $\ell(\cdot, \cdot)$ is the loss function measuring the discrepancy between the predicted and true labels. Examples of cost functions can be found in Section 3.2.2.

The optimized value is denoted by asterisk: *. For instance optimization problem for θ would be written as follows:

$$\theta^* = \arg \min \mathcal{L}(\theta) \quad (3.3)$$

3.4.2 Gradient-Descent Optimization

Gradient descent is an **first-order** optimization technique, meaning, it relies on the first derivative (gradient) of the objective function to find the minimum (or maximum). This method is widely adapted in many machine learning use cases due to its simplicity and efficiency for deep neural networks and models based on those.

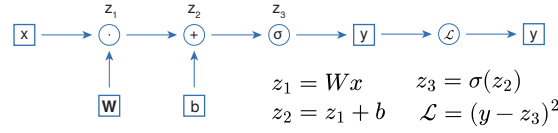


Figure 3.6: Single node network for backpropagation visualization purposes. x represents the network input, $\cdot$ multiplication, $+$ addition, σ the activation function. $\mathcal{L}$ is the loss function, in this case – squared error. z_1 and z_2 are intermediate steps, and z_3 – the network prediction

In its simplest form, gradient descent works by iteratively adjusting the parameters θ , which are randomly initialized, using the **update rule**

$$\vec{\theta} := \vec{\theta} - \alpha \nabla \mathcal{L}(\vec{\theta}), \quad (3.4)$$

where α is the **learning rate**, a scalar hyperparameter that controls the size of the step in one iteration, $\mathcal{L}(\theta)$. This process is repeated until the model converges. The visualization of a gradient based optimization can be found in Figure 3.5

Variants of gradient descent include: Stochastic Gradient Descent (SGD), Adaptive gradient (Adagrad), RMSProp, Momentum, Adam, and AdamW. These are called optimizers in ML nomenclature.

3.4.3 Backpropagation

More complex feed-forward neural networks use the concept of backpropagation in order to optimize their parameters. Backpropagation utilizes the **chain rule** to propagate the errors back through the network's layers. This process allows the model to allocate the responsibility for errors to specific parameters.

Let us take as an example, for clarity, a single-node neural network in Figure 3.6 in this network the feature space is 1-dimensional, and is represented by x , the neuron gives an output

$$z_3 = \sigma(z_2) = \sigma(z_1 + b) = \sigma(Wx + b), \quad (3.5)$$

where z_1 and z_2 are the intermediate steps, and W and b are the network's parameters. Therefore, in order to optimize the parameters, we need to find the partial derivatives of the loss function $\mathcal{L}$ in respect to W and b , this can be achieved using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial z_3} \frac{\partial z_3}{\partial W} = \frac{\partial \mathcal{L}}{\partial z_3} \frac{\partial z_3}{\partial z_2} \frac{\partial z_2}{\partial W} = \frac{\partial \mathcal{L}}{\partial z_3} \frac{\partial z_3}{\partial z_2} \frac{\partial z_2}{\partial z_1} \frac{\partial z_1}{\partial W} \quad (3.6)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial z_3} \frac{\partial z_3}{\partial b} = \frac{\partial \mathcal{L}}{\partial z_3} \frac{\partial z_3}{\partial z_2} \frac{\partial z_2}{\partial b} \quad (3.7)$$

The calculation of derivatives is done in the opposite direction to the direction of forward pass, hence the name, backpropagation. This method has been a real revolution in deep learning, allowing for simultaneous optimization of all the network's parameters, and utilizing the Graphical Processing Unit (GPU) well.

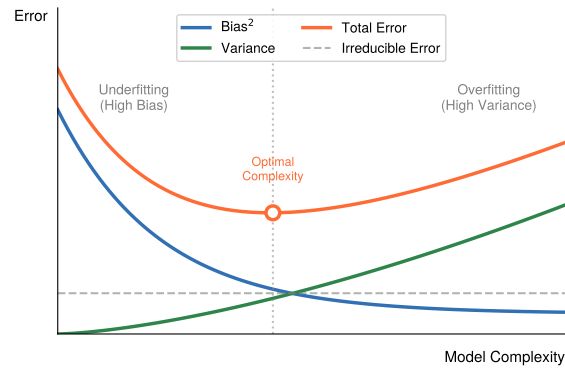


Figure 3.7: The bias variance tradeoff. Higher bias leads to underfitting, and higher variance, to overfitting. Well generalizing model minimizes the sum of bias squared and variance.

3.4.4 Model Generalization

A well optimized model should perform well on both seen and unseen data – it should generalize well. A fundamental aspect of model generalization is the balance between **bias** and **variance** (see fig. 3.7). These properties determine the expected prediction error on data unseen by the model. This relationship can be expressed as:

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error} . \quad (3.8)$$

- **Bias** corresponds to the systematic error and comes from the hypothesis space's oversimplification of reality (for example approximation of a parabola by a linear function). A model with high bias fails to capture the underlying complexity of the data, leading to **underfitting**
- **Variance** is the measure of the sensitivity of the model to the small fluctuations of the data-points in the dataset. High variance means that the model might be modeling the random noise in the data, leading to **overfitting**.
- **Irreducible Error** is the noise that exists in the data and is irreducible, as the name suggest.

A machine learning model should both capture the complexity of the true distribution behind the data, and be no more complicated than it needs to be. Finding this equilibrium is called the bias variance tradeoff. Striking the right balance leads to good generalization of the model. When a model overfits, a good start is employing regularization techniques for variance reduction, these include: $L1$ or $L2$ regularization, dropout, or early stopping.

3.4.5 Data Splits

In machine learning data splitting is a crucial step that ensures a model's performance being evaluated fairly, and that generalization is tested well on unseen data. Typically dataset is divided into three subsets:

- **Training set** – used by the model for training and optimizing the parameters. The model uses this split to learn the underlying probability distribution, and the relationships in the data.
- **Validation set** – Used for tuning of the hyperparameter of the model and intermediate tests of the model predictive abilities. It helps to judge the generalization mid-training. This data is seen by the model during training, but is not used for optimization of the parameters.
- **Test set** – used only after the whole training and validation process. It is used to estimate the true generative performance of the final model.

Commonly a data is split into test, validation, and test sets at a ratio of approx 8:1:1. The exact proportion depend on the specific case. For smaller datasets techniques such as k -fold cross-validation can be used to utilize the data more efficiently.

3.5 Building Blocks of Neural Networks

3.5.1 Neurons

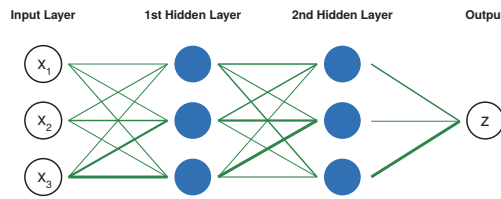


Figure 3.8: Schematic representation of a feed forward neural network with two hidden layers, three input features x_{1-3} , and one output z . The information flow is illustrated by weighted connections – lines of different thicknesses.

Neural networks are comprised of an input layer, hidden layers, and output (see fig. 3.8). Each hidden layer contains from few to hundreds of neurons or nodes. At each node two transformations are performed. First, an affine transformation, which performs a weighted sum of the neurons in previous layer, and introduces bias. This transformation can be written as

$$\vec{y} = \mathbf{W}\vec{x} + \vec{b}, \quad (3.9)$$

where $\vec{y}$ is the vector containing the neurons in the current layer, $\vec{x}$ the vector of neurons from the previous neurons multiplied by the weights matrix $\mathbf{W}$, and $\vec{b}$ is the bias vector. After the affine transformation an activation function $\sigma(\cdot)$ is applied to each neuron (typically same for each neuron):

$$\vec{y}' = \sigma(\vec{y}) = \sigma(\mathbf{W}\vec{x} + \vec{b}). \quad (3.10)$$

A visual explanation of the neurons can be found in Figure 3.9.

3.5.2 Activation functions

Networks have many layers with multiple nodes each and their ability to build complex, hierarchical representations, and therefore, their predictive power depends on this multiplicity of

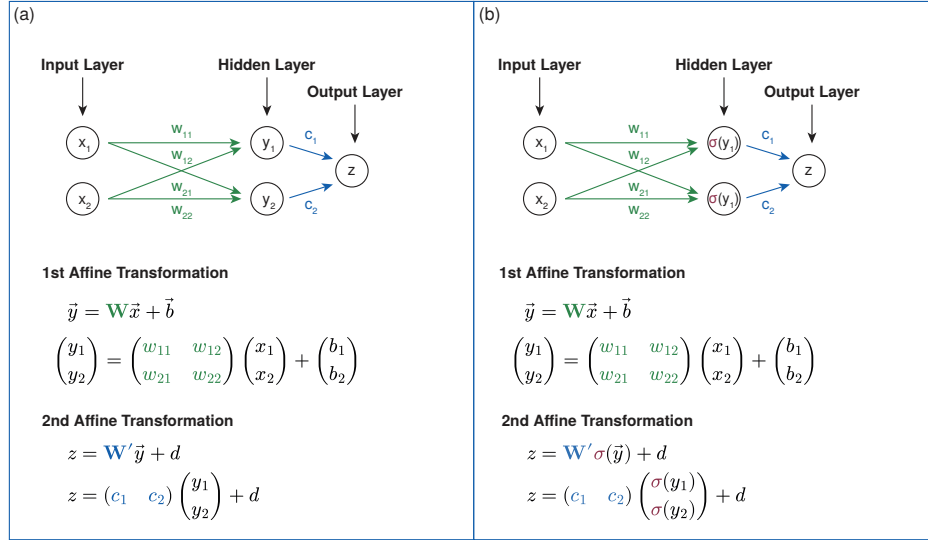


Figure 3.9: Illustration of a two-layer neural network. (a) A purely linear network performing two consecutive affine transformations: the input vector $\mathbf{x}$ is first mapped to a hidden representation $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$, and then linearly transformed again to the output $z = \mathbf{W}'\mathbf{y} + d$. Since no nonlinearity is applied, the entire system remains equivalent to a single affine transformation. (b) The same architecture with a nonlinear activation function $\sigma(\cdot)$ applied to the hidden layer outputs. This introduces nonlinearity, allowing the network to model complex, nonlinearly separable relationships between inputs and outputs.

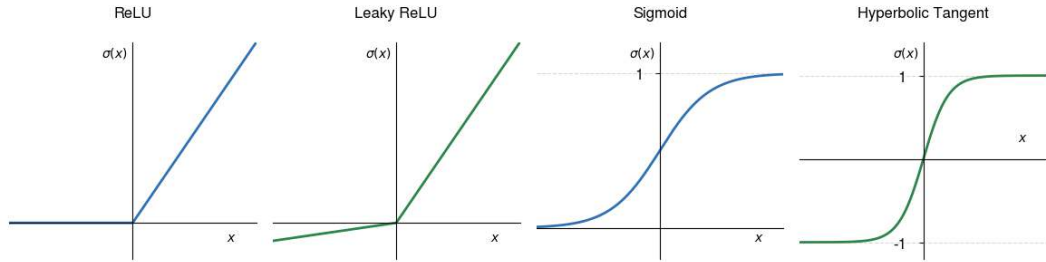


Figure 3.10: Plots of different activation functions. From left to right: Rectified Linear Unit (ReLU), Leaky ReLU, Sigmoid, Hyperbolic Tangent,

neurons. Many layers allow for much more elaborate connections between the input features, making the representation learning more effective. Due to the nature of matrix multiplication, an introduction of non-linearity is needed for the multilayer approach to work properly. Without some kind of non-linearity, multiple linear layers ($\mathbf{W}_{1-3}$) can be reduced to a single one ($\mathbf{W}$):

$$\mathbf{W}_3(\mathbf{W}_2(\mathbf{W}_1\vec{x})) = \mathbf{W}_3\mathbf{W}_2\mathbf{W}_1\vec{x} = \mathbf{W}\vec{x}. \quad (3.11)$$

Please keep in mind, that the bias has been omitted in Equation (3.11) for clarity, however, the equation still holds with the bias terms. As mentioned before, the nonlinearity is introduced after each affine transformation (see 3.9 and 3.10) by an activation function acting on a node.

There is an infinite number of potential activation functions, some of the most prevalent are introduced here (for visualizations see fig. 3.10):

1. Rectified Linear Unit (ReLU)

$$\sigma(x) = \max(0, x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases} \quad (3.12)$$

2. Leaky ReLU

$$\sigma(x) = \max(0, x) = \begin{cases} x & x \geq 0 \\ 10^{-3}x & x < 0 \end{cases} \quad (3.13)$$

3. Sigmoid (logistic) function

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (3.14)$$

4. Hyperbolic tangent

$$\sigma(x) = \tanh x \quad (3.15)$$

Generative Models

In this chapter generative machine learning models will be introduced and discussed. This chapter can be treated as both logical and historical introduction into the diffusion models, which will be discussed in the next chapter.

As opposed to discriminative models, which focus on predicting labels or outcomes from given inputs, generative models (sometimes called informative) aim to capture the underlying probability of the data over all variables in the system [30]. Once this distribution is learned, which is not trivial, new samples can be generated using that distribution.

In current literature the deep generative models are divided into four main groups:

- Autoregressive models (PixelCNN),
- Flow-based models (RealNVP),
- Latent variable models (GANs, VAEs, Diffusion models),
- Energy-based models.

We will focus on the latent variable models, this is the class of probabilistic models that assume that the data we observe is generated from some underlying latent variables. There are several directions of research within this branch of generative models. Generative Adversarial Networks (GANs) were the first revolution in modern generative machine learning, really impacting and initiating research in this field. Another class of models, more relevant in this work, are likelihood based models. These models include among others variational autoencoders and diffusion models. First, the underlying probability theory, needed to understand how the models function, will be introduced. After that the models will be explained in detail.

4.1 Probability Theory Behind Diffusion

In many cases data is just a projection of the real world onto a simplified model like a map is only a projection of a 3D sphere onto a 2D piece of paper, or a picture is only one moment in time, captured from only one perspective. The observed data has some underlying generative

process. This process is associated with an latent variable z , and then projected into our 'observed data' space. A great analogy of latent variables is the premise of the book *Flatland* by Edwin A. Abbott:

The title land, Flatland, is a 2D world, where the inhabitants perceive only two dimensions and time. They can see shapes (squares, polygons, or circles) but can not perceive height. When the *Flatlanders* are visited by a 3D sphere, for them it looks like a circle that grows and shrinks (or disappears) as the sphere moves through the space. They can only perceive the 2D projection of a higher-dimensional reality. The *Flatland* analogy illustrates the latent variables as unobservable representations that generate the observation.

In this analogy the 3D world is our latent space, the hidden variables responsible for the observation; the 2D projection is our observed data; and the *Flatlanders* are the algorithms, trying to figure out what actually is going on, i.e. what is the latent generative process. It's an illustrative allegory with, however, one caveat – in fact typically in latent-based generative models, we are trying to learn a lower-dimensional representations, not higher- ones.

4.1.1 Evidence Lower Bound (ELBO)

As mentioned before, in generative models we want the model, to learn the underlying likelihood $p(\mathbf{x})$ of the data $\mathbf{x}$

The latent variables and the observed data, can be thought of as modeled by a joint distribution $p(\mathbf{x}, \mathbf{z})$, where $\mathbf{x}$ is the data, and $\mathbf{z}$ is the latent variable. In likelihood based models the goal is for the model to maximize the likelihood function $p(\mathbf{x})$, so that the observed data $\mathbf{x}$ is the most probable. Direct maximization of $p(\mathbf{x})$ proves tricky as, in order to do so, according to the chain rule of probability

$$p(\mathbf{x}) = \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{z}|\mathbf{x})} \quad (4.1)$$

we would need to know the conditional probability of the latent $\mathbf{z}$ given $\mathbf{x}$. Even without this, we don't have access to the ground truth of this distribution. However we can derive the lower bound of evidence (ELBO), evidence being the log likelihood of the observed data $\log p(\mathbf{x})$.

Before deriving ELBO, we need a refresher on some aspects of probability theory.

Probability Theory Basics

1. A random variable X has a probability distribution $p(x)$ that tells one how likely different outcomes are. PD can be either discrete:

$$\sum_x p(x) = 1, \quad (4.2)$$

or continuous:

$$\int_x p(x) dx = 1 \quad (4.3)$$

2. For two dependent variables X and Z one can distinguish: joint $p(x, z)$, marginal $p(x) = \int p(x, z) dz$, and conditional $p(x|z) = p(x, z)/p(z)$ probabilities.

3. Bayes' Rule:

$$p(z|x) = \frac{p(x|z)p(z)}{p(x)}, \quad (4.4)$$

where $p(z|x)$ is the posterior, $p(x|z)$ – the likelihood, $p(x)$ – the marginal probability, and $p(z)$ – the prior probability.

4. Expectation is the expected value of a function $f(x)$, that is drawing from a distribution $p(x)$, and is defined as:

$$\mathbb{E}_{p(x)}[f(x)] = \int f(x)p(x) dx. \quad (4.5)$$

5. Kullback-Leiber (KL) Divergence, or relative entropy, is the distance (dissimilarity) between two distributions $q(x)$ and $p(x)$

$$D_{\text{KL}}(q(x) \parallel p(x)) = \mathbb{E}_{q(x)} \left[\log \frac{q(x)}{p(x)} \right] \geq 0, \quad (4.6)$$

it is equal to 0 only for $q(x) = p(x)$.

ELBO Derivation

Following [31] we start with the evidence (for clarity the variables will not be bolded in this case, but keep in mind that z and x are vectors, not scalars):

$$\log p(x) = \log p(x) \int q_\theta(z|x) dz \quad (\text{Multiplying by } 1 = \int q_\theta dz) \quad (4.7)$$

$$= \mathbb{E}_{q_\theta(z|x)} [\log p(x)] \quad (\mathbb{E}_{p(x)}[y] = y) \quad (4.8)$$

$$= \mathbb{E}_{q_\theta} \left[\log \frac{p(x, z)}{p(z|x)} \right] \quad (\text{Apply Equation (4.1)}) \quad (4.9)$$

$$= \mathbb{E}_{q_\theta} \left[\log \frac{p(x, z)q_\theta(z|x)}{p(z|x)q_\theta(z|x)} \right] \quad (4.10)$$

$$= \mathbb{E}_{q_\theta} \left[\log \frac{p(x, z)}{q_\theta(z|x)} \right] + \mathbb{E}_{q_\theta} \left[\log \frac{q_\theta(z|x)}{p(z|x)} \right] \quad (4.11)$$

$$= \mathbb{E}_{q_\theta} \left[\log \frac{p(x, z)}{q_\theta(z|x)} \right] + D_{\text{KL}}(q_\theta(z|x) \parallel p(z|x)) \quad (\text{Apply Equation (4.6)}) \quad (4.12)$$

$$\leq \mathbb{E}_{q_\theta} \left[\log \frac{p(x, z)}{q_\theta(z|x)} \right] \quad (\text{From Equation (4.6)}) \quad (4.13)$$

From Equation (4.12) we can observe, that the evidence is equal to the sum of ELBO and Kullback-Leiber Divergence of the modeled $q_\theta(z|x)$ and true $p(z|x)$ posterior. This derivation in fact proves that ELBO is in fact the lower bound of evidence, as $D_{\text{KL}} \geq 0$ (see eq. (4.6)). Moreover, one can see why maximizing ELBO is the objective. as ELBO and $-D_{\text{KL}}$ sum up to a constant, maximizing ELBO means minimizing the divergence between the predicted and true posterior, ideally by matching them. As, not knowing the true posterior $p(z|x)$, one can not minimize the KL Divergence term directly, the goal of maximizing ELBO is a proxy to minimizing the divergence. Additionally, optimized ELBO, can provide input to the true likelihood of the observed data, since it approximates the evidence $\log p(x)$.

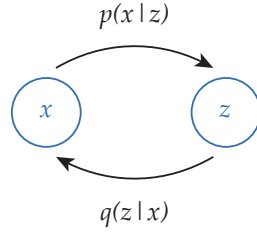


Figure 4.1: Graphical representation of Variational Autoencoder. Here, x is the observation and z the latent variable. Decoder $p(x|z)$ defines the distribution over observations for latent variables, and the encoder $q(z|x)$ defines the distribution over observations for latent variables.

4.2 Variational Autoencoders (VAE)

In this section Variational Autoencoders are introduced. On the base of this class of Generational Models, a strong understanding of the basic concepts needed for deciphering diffusion can be built. Firstly a standard VAE model is introduced, followed by a more complex, hierarchical one.

4.2.1 Standard Variational Autoencoders (VAE)

Variational AutoEncoder (VAE) is a type of autoencoder (see fig. 3.3). It, similarly to traditional autoencoders, is comprised of an encoder, latent bottleneck, and a decoder. It's called variational as the goal is to optimize for the best posterior distribution $q_\theta(z|x)$ from a family parametrized by θ . To see the bottle-necking into latent representation by a VAE, let's transform ELBO using Equation (4.1) and splitting of expectations:

$$\mathbb{E}_{q_\theta(z|x)} \left[\log \frac{p(x, z)}{q_\theta(z|x)} \right] = \mathbb{E}_{q_\theta(z|x)} [\log p_\phi(x|z)] + \mathbb{E}_{q_\theta(z|x)} \left[\log \frac{p(z)}{q_\theta(z|x)} \right] \quad (4.14)$$

$$= \mathbb{E}_{q_\theta(z|x)} [\log p_\phi(x|z)] - D_{\text{KL}}(q_\theta(z|x) \parallel p(z)), \quad (4.15)$$

[31] In the case of VAE $q_\theta(z|x)$ is the distribution that generates the latents z from input x , and can be understood as the **encoder**. Simultaneously, the model optimizes the distribution $p_\phi(x|z)$ which by analogue is the **decoder**. A visualization of this process can be seen in Figure 4.1.

In Equation (4.14) the first term is called the *reconstruction term*, and the second - *prior matching term*. The reconstruction term measures how well the model can reconstruct the input x from the latent variable z . It encourages the decoder to generate data from z , that 'looks' like the input. The prior matching term, sometimes called the regularization term, ensures that the learned latent distribution $q_\theta(z|x)$ doesn't deviate too much from the chosen prior $p(z)$. If it wasn't for the second term, the learner would collapse into a Dirac delta function.

In VAE multivariate Gaussian with diagonal covariance is commonly chosen as the probability density function of the the encoder, for the prior a standard multivariate Gaussian is a good first choice:

$$q_\theta(z|x) = \mathcal{N}(z; \mu_\theta(x), \sigma_\theta^2(x)\mathbf{I}) \quad (4.16)$$

$$p(z) = \mathcal{N}(z; \mathbf{0}, \mathbf{I}) \quad (4.17)$$

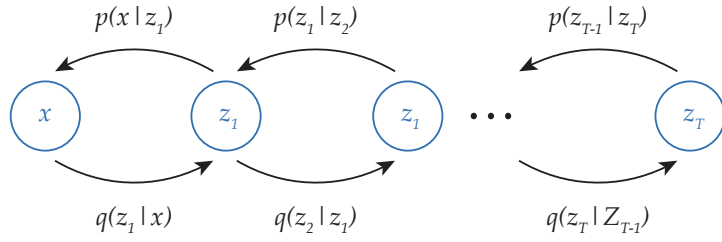


Figure 4.2: Graphical representation of a Markovian Hierarchical Variational Autoencoder with T hierarchical latents. The generative process is a Markov process, as each latent z_t is dependent only on the previous latent z_{t+1} .

While the regularization term (KL-Divergence in Equation (4.14)) can be computed analytically, reconstruction term, unless the mean $\mu_\phi(z)$ of $p_\phi(x|z)$ is a linear function of z , which in general in NN it is not. Therefore, we approximate it using a Monte Carlo estimate:

$$\arg \max_{\theta, \phi} [\mathbb{E}_{q_\theta(z|x)} [\log p_\phi(x|z)] - D_{\text{KL}}(q_\theta(z|x) \parallel p(z))] \approx \quad (4.18)$$

$$\arg \max_{\theta, \phi} \left[\sum_{i=1}^N \log p_\phi(x|z^{(i)}) - D_{\text{KL}}(q_\theta(z|x) \parallel p(z)) \right], \quad (4.19)$$

where latents $z^{(i)}$ are sampled from the variational posterior $q_\theta(z|x)$ for every x . In machine learning models in order to optimize parameters we need to be able to compute gradients in respect to the parameters (see section 3.4). This is not possible, as the sampling $z \sim q_\theta(z|x)$ is a stochastic process, which is non-differentiable, and therefore blocks gradients – backpropagation through a random draw is not possible.

This issue can be addressed with a **reparameterization trick**, where the random variable z is rewritten as a deterministic transformation of another variable ϵ that is independent from the parameters θ . In our example, the encoder defined in Equation (4.16), can be rewritten as

$$z = \mu_\theta(x) + \sigma_\theta(x) \odot \epsilon \quad \text{with} \quad \epsilon \sim \mathcal{N}(\epsilon; \mathbf{0}, \mathbf{I}) \quad (4.20)$$

This reparametrized version of z allows for gradients to be computed with respect to θ , for optimization of mean and covariance.

The generation of new data in VAE can be done by sampling directly from the latent space and decoding it.

4.2.2 Hierarchical Variational Autoencoders (HVAE)

This group of models is a generalization of variational autoencoders. It allows for stacking multiple layers of latent variables, where each layer is interpreted as being generated from the next, higher-level latent.

In general in HVAE each latent can condition on all previous latent layers. However, for understanding Diffusion Models a more specialized version of HVAE is relevant, namely a Markovian HVAE (MHVAE). It shall be presented in detail in this section.

Markovian HVAE owes its name to the way latent layers are dependent on each other. Each transition in this class of models is a Markov process. Markov process is a type of stochastic process that has no memory, the future state depends only on present, not on the past. In case of the HVAE, this means that each latent representation z_t is conditioned solely on immediately previous latent z_{t+1} . Visually this can be thought of a a matryoshka doll, where each consecutive layer is the next VAE, like in Figure 4.2. This is where a different name for this type of model comes from: Recursive VAE.

Let us formalize the layers in a Markovian HVAE. The joint distribution and the posterior are mathematically defined as:

$$p(\mathbf{x}, \mathbf{z}_{1:T}) = p(\mathbf{z}_T) p_{\phi}(\mathbf{x}|\mathbf{z}_1) \prod_{t=2}^T p_{\phi}(\mathbf{z}_{t-1}|\mathbf{z}_t), \quad (4.21)$$

$$q_{\theta}(\mathbf{z}_{1:T}|\mathbf{x}) = q_{\theta}(\mathbf{z}_1|\mathbf{x}) \prod_{t=2}^T q_{\theta}(\mathbf{z}_t|\mathbf{z}_{t-1}). \quad (4.22)$$

By analogous derivation as in Equations (4.7) to (4.13), we can obtain the ELBO for this system:

$$\log p(\mathbf{x}) = \mathbb{E}_{q_{\theta}(\mathbf{z}_{1:T}|\mathbf{x})} \left[\log \frac{p(\mathbf{x}, \mathbf{z}_{1:T})}{q_{\theta}(\mathbf{z}_{1:T}|\mathbf{x})} \right] \quad (4.23)$$

This is the objective of the Markovian HVAE, optimization is analogous as in the case of the standard VAE.

4.3 Variational Diffusion Models (VDM)

The shortcomings of VAEs and GANs have led to the development of new class of models – Diffusion Probabilistic Models. They were first proposed in 2015 by Sohl-Dickstein et al. [32]. The idea of learning the noise-to-data mapping was inspired by thermodynamical processes where diffusion is the flow of particles from high concentration to low concentration regions. In diffusion models the model learns the *noise-ification* process, and based on that it can reconstruct an image from noise. There exist various equivalent formalisms of diffusion models: Markov chains, denoising diffusion probabilistic models, stochastic differential equations and score networks. Here, for sake of continuity, they are introduced as Markov chains.

Variational Diffusion Model (VDM) is a type of a MHVAE (see section 4.2.2) with following holdbacks:

1. All the latent dimensions are equal to the dimension of the input data.

Thanks to this restriction, we can change the notation used in the aforementioned cases, and no longer denote the latent variables with $\mathbf{z}$, but together with the true data as $\mathbf{x}_t$, where $t = 0$ corresponds to the true data, and $t \in [1, T]$, $t \in \mathbb{N}^+$ denotes the latents

with hierarchy (also called **time-steps**) represented by t . This also allows us to simplify the posterior from Equation (4.22) to:

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}). \quad (4.24)$$

2. The latent encoder is not learned, but predefined as a linear Gaussian model i.e. each next latent encoder is a Gaussian centered around the output of the timestep before. The mean and the standard deviation of the latent-encoder-Gaussians can be either set as hyperparameters or learned as parameters. Common practice is to parametrize the Gaussian encoder with variance and mean

$$\Sigma_t(\mathbf{x}_t) = (1 - \alpha_t)\mathbf{I} = \begin{bmatrix} 1 - \alpha_t & 0 & 0 & \dots & 0 \\ 0 & 1 - \alpha_t & 0 & \dots & 0 \\ 0 & 0 & 1 - \alpha_t & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 - \alpha_t \end{bmatrix}, \quad (4.25)$$

$$\mu_t(\mathbf{x}_t) = \sqrt{\alpha_t}\mathbf{x}_{t-1}, \quad (4.26)$$

where the coefficients α_t are chosen in such a way, that the variance of the latents stays at the similar scale – the encoding process is Variance Preserving (VP), as opposed to Variance Exploding (VE) or sub-VP. The α_t coefficient can vary with t . The encoder transitions are denoted as:

$$q(\mathbf{x}|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\mathbf{x}_{t-1}, (1 - \alpha_t)\mathbf{I}). \quad (4.27)$$

3. The parameters of the encoders vary in such a way that at the final step $t = T$, the latent distribution $p(\mathbf{x}_T)$ is a standard Gaussian.

This restriction can be formalized by adapting Equation (4.21):

$$p(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_\phi(\mathbf{x}_{t-1}|\mathbf{x}_t), \quad (4.28)$$

where

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}) \quad (4.29)$$

This set of three restrictions on a Markovian HVAE describes a steady noise-ification of an input over time. The input data is progressively corrupted by the process of adding Gaussian noise until it becomes indistinguishable from Gaussian noise. The noise-ification step is called the **forward diffusion process**, while the denoising step is called the **reverse diffusion process**. You can see the process with the introduced restrictions in Figure 4.3.

In a diffusion model the encoder distributions $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ are no longer parametrized by θ as they are not dependent on any parameters. The mean and variance are predefined in Equation (4.27) at each timestep. We can derive the

$$\log p(\mathbf{x}) = \log \int p(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T} \quad (4.30)$$

$$= \log \int \frac{p(\mathbf{x}_{0:T}) q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} d\mathbf{x}_{1:T} \quad (4.31)$$

$$= \log \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\frac{p(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right], \quad (4.32)$$

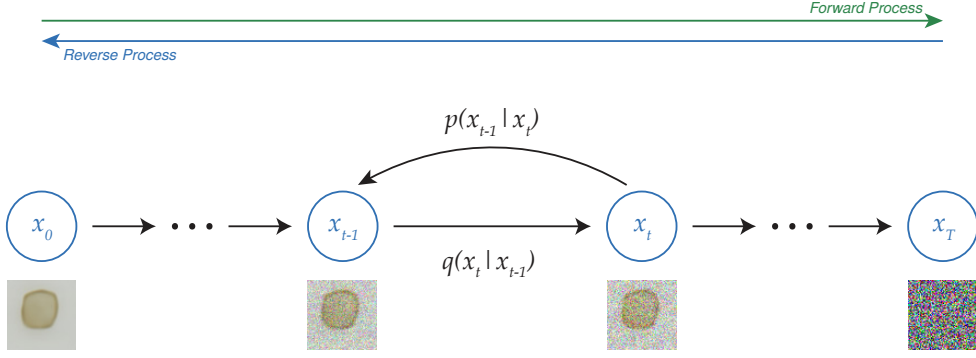


Figure 4.3: A visualization of diffusion process. The green and blue arrows indicate the forward and reverse diffusion processes directions. x_0 represents the true data, x_T represents the final pure Gaussian noise, $x_{1:T-1}$ are the intermediate latents which are increasingly noisified versions of x_0 . Each q is modeled as a Gaussian distribution with the output of the previous state as its mean

Using

- *Jensen's inequality* (see section A.1)) ($f(x) = \log x$ is a strictly concave function);
- as well as the fact that our model is a Markov chain, and adding a superfluous condition to the encoder: $q(\mathbf{x}_t | \mathbf{x}_{t-1}) \rightarrow q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0)$;
- and the Bayes rule:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0) = \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) q(\mathbf{x}_t | \mathbf{x}_0)}{q(\mathbf{x}_{t-1} | \mathbf{x}_0)} \quad (4.33)$$

we get:

$$\log p(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \left[\log \frac{p(\mathbf{x}_0 : T)}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right] \quad (4.34)$$

$$= \mathbb{E}_{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \left[\log \frac{p(\mathbf{x}_T) \prod_{t=1}^T p(\mathbf{x}_{t-1} | \mathbf{x}_t)}{\prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})} \right] \quad (4.35)$$

$$= \mathbb{E}_{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \left[\log \frac{p(\mathbf{x}_T) p_\theta(\mathbf{x}_0 | \mathbf{x}_1) \prod_{t=2}^T p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)}{q(\mathbf{x}_1 | \mathbf{x}_0) \prod_{t=2}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})} \right] \quad (4.36)$$

$$= \mathbb{E}_{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \left[\log \frac{p(\mathbf{x}_T) p_\theta(\mathbf{x}_0 | \mathbf{x}_1)}{q(\mathbf{x}_1 | \mathbf{x}_0)} + \log \prod_{t=2}^T \frac{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)}{\frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) q(\mathbf{x}_t | \mathbf{x}_0)}{q(\mathbf{x}_{t-1} | \mathbf{x}_0)}} \right] \quad (4.37)$$

$$= \mathbb{E}_{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \left[\log \frac{p(\mathbf{x}_T) p_\theta(\mathbf{x}_0 | \mathbf{x}_1)}{q(\mathbf{x}_1 | \mathbf{x}_0)} + \sum_{t=2}^T \log \frac{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) q(\mathbf{x}_{t-1} | \mathbf{x}_0)}{q(\mathbf{x}_t | \mathbf{x}_0) q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)} \right] \quad (4.38)$$

$$= \underbrace{\mathbb{E}_{q(\mathbf{x}_1 | \mathbf{x}_0)} [\log p_\theta(\mathbf{x}_0 | \mathbf{x}_1)]}_{\text{reconstruction term}} - \underbrace{D_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_0) \| p(\mathbf{x}_T))}_{\text{prior matching term}} - \underbrace{\sum_{t=2}^T \mathbb{E}_{q(\mathbf{x}_t | \mathbf{x}_0)} [D_{\text{KL}}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))]}_{\text{denoising matching term}} \quad (4.39)$$

The components of the ELBO can be understood as[31]:

- $\mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)}[\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)]$ as a *reconstruction term*, this is the term that predicts the log probability of the original data given the first-step latent. It is similar to VAE (see eq. (4.14)) and can be trained similarly using Monte Carlo methods.
- $D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p(\mathbf{x}_T))$ as a *prior matching term*. This term is not optimizable, as it is not dependent on any parameters. Also, it is equal to 0 under our assumptions.
- The last term as a *denoising term*. its goal is to ensure, that $\mathbf{x}_t$ is consistent both in the forward and reverse process, i.e. that $q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)$ and $p_\theta(\mathbf{x}_t|\mathbf{x}_{t+1})$ lead to the same result. As the KL Divergence is minimized when $p_\theta(\mathbf{x}_t|\mathbf{x}_{t+1})$ matches $q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)$.

The last term is the actual term that we want to minimize, in order to maximize ELBO. Whereas, this term is difficult to optimize for an arbitrary posterior, in VDM we can utilize the Gaussian transition assumption in order to enable optimization tracking. This is our objective function

$$\mathcal{L}(\theta) = \sum_{t=2}^T \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} [D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))] \quad (4.40)$$

4.3.1 Minimization of the Denoising Term

To minimize the denoising term we utilize the Bayes rule in Equation (4.33). As our model is a Markov chain we can use Equation (4.27) to obtain

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) = q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\mathbf{x}_{t-1}, (1 - \alpha_t)\mathbf{I}). \quad (4.41)$$

Under the reparameterization trick (see eq. (4.20)) samples $\mathbf{x}_t \sim q(\mathbf{x}_t|\mathbf{x}_{t-1})$ and $\mathbf{x}_{t-1}$ can be redefined as:

$$\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1 - \alpha_t}\epsilon \quad (4.42)$$

$$\mathbf{x}_{t-1} = \sqrt{\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon \quad \text{where } \epsilon \sim \mathcal{N}(\epsilon, \mathbf{0}, \mathbf{I}). \quad (4.43)$$

Then we can plug in the expression for $\mathbf{x}_{t-1}$ into Equation (4.42):

$$\mathbf{x}_t = \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-2} \right) + \sqrt{1 - \alpha_t}\epsilon_{t-1}, \quad (4.44)$$

We can use the property of normal distributions, which states that the sum of two iid random variables that follow normal distributions $Z = X + Y$ is also normally distributed with mean being the sum of the means of the addends $\mu_z = \mu_X + \mu_Y$, and variance – the sum of individual variances $\sigma_Z^2 = \sigma_X^2 + \sigma_Y^2$ (see section A.2). Using this property we can use the reparameterization trick and the summing rule on $(\alpha_t - \alpha_t\alpha_{t-1})^{1/2}\epsilon_2 \sim \mathcal{N}(\mathbf{0}, (\alpha_t - \alpha_t\alpha_{t-1})\mathbf{I})$ and $(1 - \alpha_t)^{1/2}\epsilon_1 \sim \mathcal{N}(\mathbf{0}, (1 - \alpha_t)\mathbf{I})$ obtaining $(1 - \alpha_t\alpha_{t-1})^{1/2}\epsilon_{t-2}^*$. We can repeat this procedure

recursively to obtain $\mathbf{x}_t \sim q(\mathbf{x}_t|\mathbf{x}_0)$:

$$\mathbf{x}_t = \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{\alpha_t - \alpha_t \alpha_{t-1}} \epsilon_{t-2} + \sqrt{1 - \alpha_t} \epsilon_{t-1} \quad (4.45)$$

$$= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \epsilon_{t-2}^* \quad \text{with } \epsilon_{t-2}^* \sim \mathcal{N}(\epsilon_{t-2}^*; \mathbf{0}, \mathbf{I}) \quad (4.46)$$

$$= \dots \quad (4.47)$$

$$= \sqrt{\prod_{i=1}^t \alpha_i} \mathbf{x}_0 + \sqrt{1 - \prod_{i=1}^t \alpha_i} \epsilon_0^* \quad (4.48)$$

$$= \sqrt{\tilde{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \tilde{\alpha}_t} \epsilon_0^* \quad \text{with } \epsilon_0^* \sim \mathcal{N}(\epsilon_0^*; \mathbf{0}, \mathbf{I}) \quad (4.49)$$

$$\sim \mathcal{N}(\mathbf{x}_t; \sqrt{\tilde{\alpha}_t} \mathbf{x}_0, (1 - \tilde{\alpha}_t) \mathbf{I}) \quad (4.50)$$

Now, that we know $q(\mathbf{x}_t|\mathbf{x}_0)$, $q(\mathbf{x}_{t-1}|\mathbf{x}_0)$, and $q(\mathbf{x}_t|\mathbf{x}_{t-1})$, we can obtain the expression for $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ using the Bayes formula in Equation (4.33)

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}) q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)} \quad (4.51)$$

$$\propto \exp \left[-\frac{(\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1})^2}{2(1 - \alpha_t)} - \frac{(\mathbf{x}_{t-1} - \sqrt{\tilde{\alpha}_{t-1}} \mathbf{x}_0)^2}{2(1 - \tilde{\alpha}_{t-1})} + \frac{(\mathbf{x}_t - \sqrt{\tilde{\alpha}_t} \mathbf{x}_0)^2}{2(1 - \tilde{\alpha}_t)} \right] \quad (4.52)$$

$\vdots$ (trivial)

$$\propto \mathcal{N}(\mathbf{x}_{t-1}; \underbrace{\frac{\sqrt{\alpha_t}(1 - \tilde{\alpha}_{t-1}) \mathbf{x}_t + \sqrt{\tilde{\alpha}_{t-1}}(1 - \alpha_t) \mathbf{x}_0}{1 - \tilde{\alpha}_t}}_{\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0)}, \underbrace{\frac{(1 - \alpha_t)(1 - \tilde{\alpha}_{t-1})}{1 - \tilde{\alpha}_t} \mathbf{I}}_{\boldsymbol{\Sigma}_q(t)}). \quad (4.53)$$

As we want to minimize the denoising term of Equation (4.39), we want the denoising step $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ to be as closely matched to the ground truth denoising step $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ as possible. Therefore, we can start by modeling it with a Gaussian. Moreover, as all α_t terms are known and fixed at each timestep (they are typically fixed as a hyperparameter using a scheduler), we can already use the variance of the ground truth denoising step from Equation (4.53) for the model:

$$\boldsymbol{\Sigma}_p(t) = \boldsymbol{\Sigma}_q(t) = \frac{(1 - \alpha_t)(1 - \tilde{\alpha}_{t-1})}{1 - \tilde{\alpha}_t} \mathbf{I}. \quad (4.54)$$

However, as $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ cannot condition on $\mathbf{x}_0$ (as this is the generated data), its mean $\boldsymbol{\mu}_\theta$, therefore, cannot be taken straight from the ground truth. This is the property of the Gaussian that we would like our model to learn, as minimization of divergence yields:

$$\arg \min_{\theta} D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) \quad (4.55)$$

$$= \arg \min_{\theta} \frac{1}{2\sigma_q^2(t)} [\|\boldsymbol{\mu}_\theta - \boldsymbol{\mu}_q\|_2^2] \quad (4.56)$$

where for brevity $\boldsymbol{\mu}_q = \boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0)$ is the true mean defined in Equation (4.53) and $\boldsymbol{\mu}_\theta = \boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ is the predicted mean, we can set it closely to the true mean by defining it as:

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{\sqrt{\alpha_t}(1 - \tilde{\alpha}_{t-1}) \mathbf{x}_t + \sqrt{\tilde{\alpha}_{t-1}}(1 - \alpha_t) \hat{\mathbf{x}}_\theta(\mathbf{x}_t, t)}{1 - \tilde{\alpha}_t}, \quad (4.57)$$

where $\hat{x}_\theta$ is the output of the neural network for an input a timestep t given the noisy image x_t . For the model mean defined in this way the optimization problem simplifies to

$$\arg \min_{\theta} D_{KL}(q(x_{t-1}|x_t, x_0) || p_{\theta}(x_{t-1}|x_t)) \quad (4.58)$$

$$= \arg \min_{\theta} \frac{1}{2\sigma_q^2(t)} \frac{\bar{\alpha}_{t-1}(1 - \alpha_t)^2}{(1 - \bar{\alpha}_t)^2} [\|\hat{x}_\theta(x_t, t) - x_0\|_2^2] \quad (4.59)$$

Therefore, the loss function comes down to the distance between: the predicted input data for a given number of time-steps, and the latent at that timestep; and the original data. As stated in Equation (4.40) our cos function is the sum over all the timesteps, which for a large number of t (typically on the order of 10^3) can be very computationally expensive. Also it was found that the optimization works better with omission of the weighing terms [33]. Therefore, we can simplify our optimization goal to

$$\arg \min_{\theta} \mathcal{L}(\theta) \approx \arg \min_{\theta} \mathbb{E}_{t \sim U\{2, T\}} [\mathbb{E}_{q(x_t|x_0)} D_{KL}(q(x_{t-1}|x_t), x_0) || p_{\theta}(x_{t-1}|x_t)] \quad (4.60)$$

$$= \arg \min_{\theta} \mathbb{E}_{t \sim U\{1, T\}} [\mathbb{E}_{q(x_t|x_0)} \|\hat{x}_\theta(x_t, t) - x_0\|_2^2] \quad (4.61)$$

A slightly different derivation leads to a definition of VDMs as noise predictors, which have been found to lead to better results [33]. The loss function in this case is defined as

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim U\{1, T\}} [\mathbb{E}_{q(x_t|x_0)} \|\hat{\epsilon}(x_t, t) - \epsilon_t\|_2^2] \quad (4.62)$$

$$= \mathbb{E}_{t \sim U\{1, T\}} [\mathbb{E}_{q(x_t|x_0)} \|\hat{\epsilon}(\sqrt{\tilde{\alpha}_t}x_0 + \sqrt{1 - \tilde{\alpha}_t}\epsilon_t, t)\|_2^2], \quad (4.63)$$

where $\hat{\epsilon}(x_t, t)$ is the model that learns to predict the noise $\epsilon_t \sim \mathcal{N}(\epsilon_t; \mathbf{0}, \mathbf{I})$

Algorithm 1 Training VDM

```

1: repeat
2:    $x_0 \sim q(x_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ 
5:   Take gradient descent step on
       $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\tilde{\alpha}_t}x_0 + \sqrt{1 - \tilde{\alpha}_t}\epsilon, t)\|_2^2$ 
6: until converged

```

Algorithm 2 Sampling from VDM

```

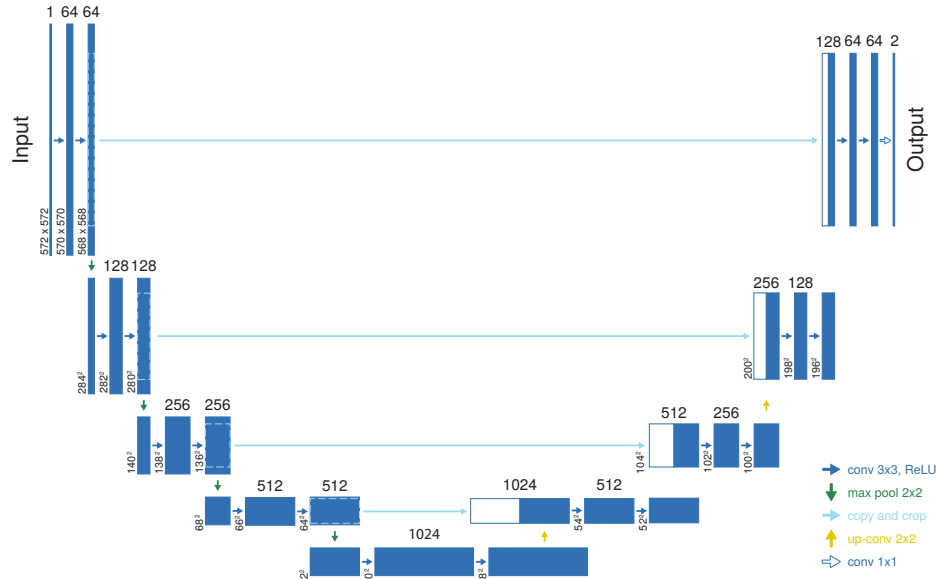
1:  $x_T \sim \mathcal{N}(0, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:   if  $t > 1$  then
4:      $z \sim \mathcal{N}(0, \mathbf{I})$ 
5:   else
6:      $z = 0$ 
7:   end if
8:    $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$ 
9: end for
10: return  $x_0$ 

```

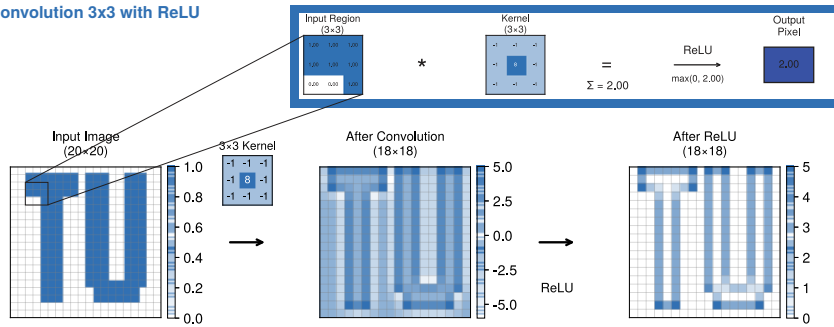
A standard training and inference algorithms using diffusion can be found in Algorithms 1 and 2

4.3.2 The Denoising Network Architecture: U-Net

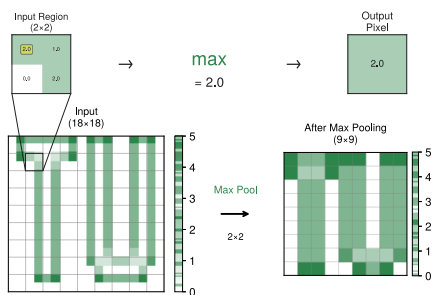
The model ϵ_{θ} that predicts the noise at each timestep is a crucial component of the diffusion model. In practice, this network is almost universally implemented using a U-Net architecture. U-Net is an encoder-decoder network originally designed for biomedical image segmentation [34]. A typical U-Net architecture is shown in the Figure 4.4



A. Convolution 3x3 with ReLU



B. Max Pool 2x2



C. Upconvolution 2x2

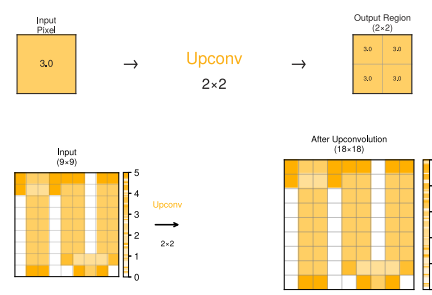


Figure 4.4: The top part shows the full U-Net architecture (adapted from [34]). Each navy box represents a multi-channel feature map, labeled with the number of channels (top) and x–y dimensions (left). White boxes indicate copied feature maps, and arrows denote operations. The main operations are illustrated in (A–C): **A** – convolution with ReLU activation using a 3×3 kernel; **B** – 2×2 max pooling, selecting the maximum value per region; **C** – up-convolution, doubling the spatial dimensions.

Its design is uniquely suited for denoising tasks for two main reasons: Encoder-Decoder Structure – It has a down-sampling encoder path to capture the contextual, semantic information of the image. In this part the input gets repeatedly convoluted (see fig. 4.4 A), and max-pooled (see fig. 4.4 B) until a desired dimension of the data is reached into the bottleneck – the bottom part of the U . After the bottleneck comes the up-sampling decoder path to reconstruct the image the sample is up-sampled using combination of convolutions and up-convolutions (see fig. 4.4 C). The key innovation of the U-Net are the *skip connections* (the copy and crop operation on fig. 4.4) that link layers from the encoder directly to corresponding layers in the decoder. These connections allow the network to pass low-level feature information (like textures and fine details) from the early layers directly to the later layers. This is critical for diffusion, as the model must “understand” the content of the noisy image, while also being able to reconstruct the fine pixel-level details, which the skip connections help preserve.

In the context of denoising diffusion probabilistic models (DDPMs) and their variants, the U-Net serves as the natural foundation, responsible for prediction of the noise component at a given diffusion step t . Each denoising step requires the network to be able to approximate the loss function or the noise conditioned on both the noisy sample and the timestep embedding and other conditions.

In SOTA diffusion architectures the basic U-Net is extended in several ways, these extensions will be explained in detail in the following sections. First, the temporal conditioning is incorporated by a time embedding – typically a vector whose elements are calculated using some sinusoidal functions and are injected throughout the network into the **residual blocks**. By encoding the timesteps in this way, the U-Net is able to learn a **shared** denoising function that can generalize across all timesteps (i.e. all levels of noise) rather than producing multitude of models – one per timestep. Second, **cross-attention** layers are introduced in order to condition the output on additional modalities like: text, class labels, or other images.

To sum up, the U-Net architecture revolutionizes diffusion models by enabling multi-scale feature extraction, preserving high-resolution details through skip-connections, stabilizing training of a very deep neural network. It also allows flexible conditioning via timestep embeddings and attention. These innovations allow U-Nets to make the generative process and training a lot cheaper.

4.4 Additional Components of Denoising Networks

In order to fully grasp the architecture of a stable diffusion, we need to understand the attention mechanism as well the residual neural network (ResNet) architecture, as they play a crucial part of the conditioning of diffusion model.

4.4.1 Attention is All You Need

This is the title of a landmark 2017 research paper [35] in machine learning that has revolutionized the field, leading to development of transformers, and therefore Generative Pre-trained Transformer (GPT) models and bringing a breakthrough in our understanding of memory in machine learning. It functionally rendered models like Long Short-Term Memory (LSTM) and other variation of Recurrent Neural Networks (RNNs) obsolete. In this section the attention mechanism will be explained in the context of diffusion models.

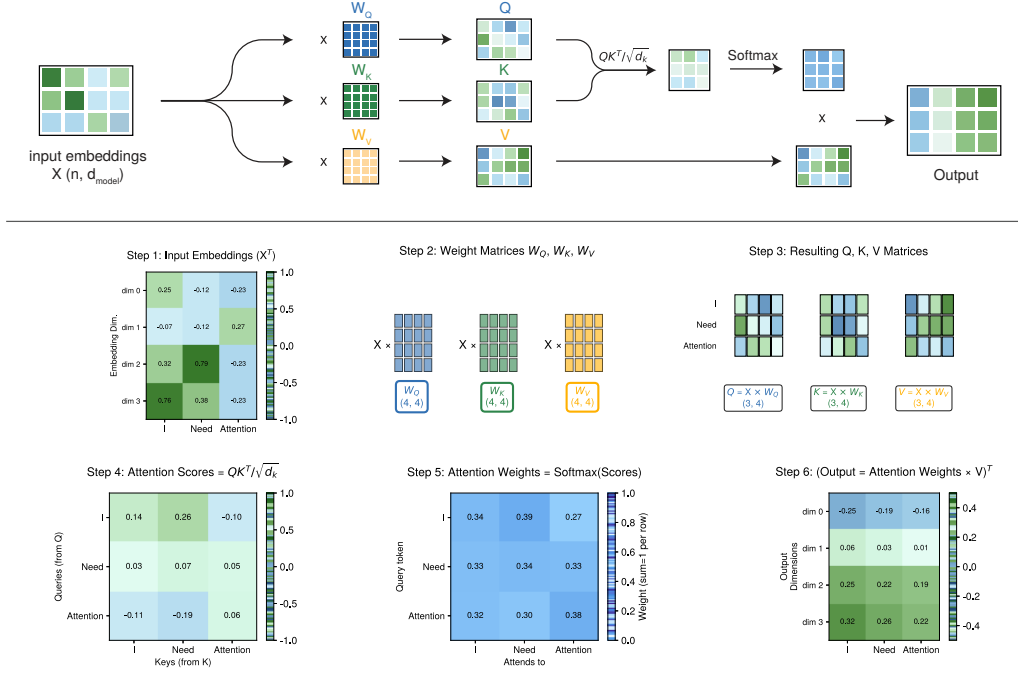


Figure 4.5: This diagram illustrates the mathematical process of self attention. The top section provides a schematic overview showing how input embeddings X are transformed into Queries Q , Keys K , and Values V via weight matrices W_Q , W_K , W_V . The bottom section breaks this down into a specific six-step numerical example with the use of the (tokenized) input phrase “I Need Attention”. It visualizes the matrix operations for calculating Attention scores, applying Softmax to determine attention weights and computing the final output vectors.

The attention mechanism is a component neural network that allows models to vary the importance of different elements of the input when generating the output. In simple terms attention attributes variable weights to input components, instead of processing all the information with equal weights. Mathematically attention is defined as

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \in \mathbb{R}^{m \times d_v}, \quad (4.64)$$

where $Q \in \mathbb{R}^{m \times d_k}$, $K \in \mathbb{R}^{n \times d_k}$, $V \in \mathbb{R}^{n \times d_v}$ are respectively: the Query matrix, containing m queries; the Key matrix; and the Value matrix. The last two jointly contain an unordered set of n key-value pairs. $d_{\{q,k,v\}}$ typically are the same, and can but don’t have to correspond to the dimension of the feature vector for which attention is calculated. For self attention $n = m$, for cross attention, which is crucial for conditioning generation (such as text-to-image generation), n generally differs from m . In such a case, the Query matrix Q is derived from the primary sequence, while the Key K , and Value V matrices are projected from an external conditioning context, such as class labels or text embedding.

4.4.2 Residual Blocks

While the attention mechanism in a U-Net captures long range dependencies, the Residual blocks are the other important aspect of the modern U-Net architecture, they are the components capturing the local dependence in the U-Net preserving the original data, while

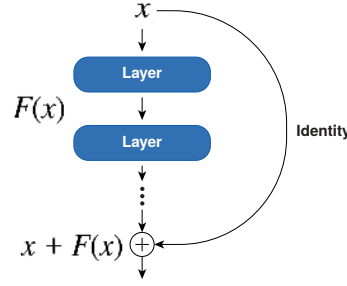


Figure 4.6: Visualization of a Residual Block. The layer goal is to learn the modifier of x

enabling conditioning. However, an even more important function of the residual blocks lies in mitigation of a vanishing gradient problem. As mentioned before (see section 3.4), when training a neural network, a gradient for each weight has to be calculated using the chain rule from the back of the neural network all the way to the beginning in order to update the weight. These gradients become exponentially small (or big) due to the chain rule, as it is effectively multiplication of thousands or millions of factors can easily lead to vanishing (exploding gradients), making the training unstable.

$$\lim_{n \rightarrow \infty} \prod_i^n a_i = \begin{cases} 0 & \text{for } a_i < 1 \\ \infty & \text{for } a_i > 1 \end{cases}, \quad (4.65)$$

this is just a visualization of an vanishing and exploding gradients, in general, this limit can also have different solutions.

Introduced by He et al. [36], the ResBlock modifies the goal of a NN layer, instead of learning completely new representation $H(x)$ at every layer, the layer is tasked with learning a residual function.

Consider a subnetwork with a certain number of stacked layers (see fig. 4.6). If x is the input and $H(x)$ is the function performed by the subnetwork, a standard neural network, tries to learn $H(x)$ directly without any constrain. The idea of Residual Learning is to reparametrize the subnetwork in order for the parameter layers to represent a *residual function*

$$F(x) = H(x) - x, \quad (4.66)$$

with that we can represent the output y of the subnetwork as

$$y = F(x) + x \quad (4.67)$$

the operation of adding unchanged input x to the output is called *skip connection*, and we have already seen it in the U-Net architecture, the skip connections are light-blue arrows in the upper panel of the Figure 4.4, where the unchanged (only cropped) outputs of intermediate stages of the U-Net down-sampler are cropped and concatenated with the output of respective intermediate up-sampler.

4.4.3 Encoding Strategies

It was mentioned before in Section 4.3.2, that conditioning on various labels can be achieved in generative models, either by prompting (textual conditioning), class labeling, image conditioning, and so on. Some of these labels are easier to understand by a computer, as they are

Table 4.1: Overview of standard encoding strategies used in machine learning.

Category	Encoding Method	Description
Categorical	One-Hot Encoding	Represents each category as a separate binary feature.
	Ordinal Encoding	Maps categories to integer ranks; assumes implicit order.
	Target Encoding	Replaces category with target statistic (e.g., mean).
	Leave-One-Out Encoding	Target encoding variant mitigating target leakage.
	Frequency Encoding	Encodes categories by their empirical frequency.
	Hashing Trick	Hashes categories into a fixed number of dimensions.
Text	Tokenization	Converts raw text into tokens (words/sub-words).
	Bag-of-Words	Represents documents by token occurrence counts.
	TF-IDF	Weights terms by importance in the corpus.
	Word Embeddings	Dense pretrained vectors (word2vec, GloVe, fastText).
	Transformer Embeddings	Contextual embeddings (BERT, RoBERTa).
Sequential	Positional Encoding	Encodes relative or absolute positions in sequences.
	Sliding Window	Constructs lagged or window-based features.
	FFT/DCT Encoding	Represents signal in the frequency domain.
Numeric	Min-Max Scaling	Rescales features to a fixed interval (typically [0,1]).
	Standardization	Normalizes features using mean and standard deviation.
	Quantile Encoding	Maps values to ranks or quantile bins.
Other	Learned Category Embeddings	Dense embeddings trained jointly with a model.
	Graph Encoding	Node representations (e.g., node2vec, GCN features).
	Image Encoding	CNN features or patch embeddings (ViT).

inherently numerical: we can represent classes by natural numbers, images are represented by RGB values, and so on. However, what if we have data in a form that is not easily understandable by a computer? In this case we need to find a strategy to translate the data from the human-readable format to a one understandable by a computer. In Table 4.1 you can find some encoding strategies that are standard in machine learning applications. Below some strategies most relevant for this work are explained in detail.

One-Hot Encoding

The most straightforward way of encoding technique. The idea is to represent each category as a binary vector, where each element of the vector corresponds to a different category. Inherently in this approach the shape of the embedding (feature) vector has to be the same (or larger) than the number of classes we want to represent. An example of such encoding would be encoding elements in the periodic table. This would mean that the feature vector would have to have the dimension of 118 if we wanted to include all the elements known to man. For example hydrogen and oxygen could be represented in a form of vectors, where all but one elements are 0. For hydrogen the non-zero element would be the first one as it is in the first column and the first row of the periodic table, for oxygen the non-zero element would be the eighth element, as it is the eighth element when going by columns.

$$\text{H} \rightarrow v_H = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \text{O} \rightarrow v_O = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad (4.68)$$

If this was our encoding strategy, one could encode compounds by just multiplying the appropriate chemical element encoding by the appropriate stoichiometric ratio. For instance an encoding for H_2O could be constructed in a following way

$$v_{\text{H}_2\text{O}} = v_H + 2v_O = [2, 0, 0, 0, 0, 0, 0, 1, 0, \dots, 0]^T, \quad (4.69)$$

or normalized

$$\hat{v}_{\text{H}_2\text{O}} = [0.(6), 0, 0, 0, 0, 0, 0, 0.(3), 0, \dots, 0]^T \quad (4.70)$$

One-hot encoding is a simple and intuitive method, avoiding any false assumptions about ordering and preserving all information about distinct classes. Its main advantages are clarity, stability, and strong compatibility with linear models. However, it can quickly lead to very high-dimensional, sparse vectors when the number of categories grows, increasing memory usage and slowing down training. It also fails to capture similarity between categories. This makes this method not ideal for our use-case, however it's an important one to know.

Descriptor Encoding

Descriptor encoding represents each sample using a vector of properties defining the samples relevant properties, for example: atomic mass, electronegativity, valence electron count, or

structural parameters. Its main strength is that it incorporates expert knowledge directly into the feature space, allowing models to learn from physically or chemically relevant properties rather than arbitrary identifiers. This often leads to better performance with less data and improves interpretability, since each dimension corresponds to a known property. However, this approach has its downsides, as descriptor encoding depends heavily on the quality and completeness of the descriptors curated by experts. This may introduce bias if important features are omitted, and may limit model flexibility compared to learned representations such as embeddings or graph-based features.

Learned Embeddings

Learned embeddings encode categorical or symbolic entities as dense, low-dimensional vectors. The values within the vectors are learnable and are optimized during training. Instead of relying on hand-designed descriptors, the model discovers relationships and similarities directly from data, in either a unsupervised or supervised fashion allowing related items to end up close together in the embedding space. This makes embeddings powerful, expressive, and scalable, especially for neural networks working with large vocabularies or complex structures. The main drawback of such approach is reduced interpretability – unlike descriptor-based features, embedding dimensions have no understandable meaning, they are purely statistical – and they require enough data for the model to learn meaningful structure. They can also inadvertently capture dataset biases, since all information comes from observed patterns rather than domain knowledge.

4.4.4 Noise Scheduling

This part of the model is responsible of generating the noise in the the image. As mentioned in Section 4.3.1 the noising process in diffusion models is defined by

$$\mathbf{x}_t = \sqrt{\tilde{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \tilde{\alpha}_t} \boldsymbol{\epsilon}, \quad \text{with } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (4.71)$$

$\mathbf{x}_0$ is an input example (e.g. an image), $\boldsymbol{\epsilon}$ is a sample from an isotropic Gaussian distribution, $\tilde{\alpha}_t$ is the combination of all the alpha parameters for previous steps: $\tilde{\alpha}_t = \prod_{i=1}^t \alpha_i$. t is the timestep taken from a uniform distribution: $t \in \mathcal{U}(1, \dots, T)$.

One can think of diffusion models in two ways: as learning the underlying image from a noisy image, or as learning the noise and then subtracting it from the noisy image. If we consider diffusion as learning the noise, it is more intuitive to define the noise scheduling through a parameter β_t and define the parameter α_t through it:

$$\alpha_t = 1 - \beta_t, \quad (4.72)$$

or equally

$$\beta_t = 1 - \alpha_t. \quad (4.73)$$

these are predefined sequence of parameters, that can be thought of as variance of the noise. They govern the magnitude and rate of the Gaussian noise addition in the forward process, but also define the denoising steps in the reverse process in Latent Diffusion Models. β_t typically is a monotonically increasing function with t increasing from $\beta_{\min}$ to $\beta_{\max}$ (therefore from Equation (4.72) α_t decreases with t). If β_t is the variance of the noise at step t , α_t can be thought of as the signal retention factor at that timestep. As mentioned before in Section 4.3.1

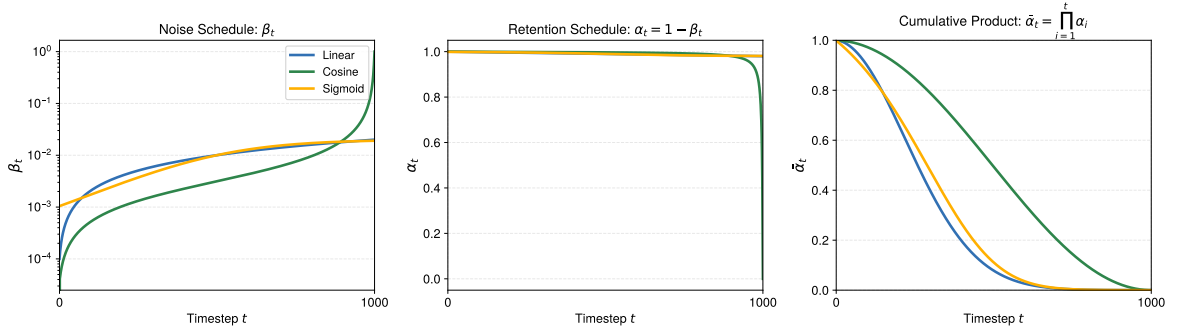


Figure 4.7: Noise schedules for diffusion models: **(Left)** Noise schedule β_t , **(Middle)** Retention schedule $\alpha_t = 1 - \beta_t$, **(Right)** Cumulative product $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$. Three schedules are compared: linear, cosine, and sigmoid, showing different trade-offs in signal degradation over $T = 1000$ timesteps.

the accumulated product of the signal retention factor $\tilde{\alpha}_t$ is particularly powerful as it allows for the noisy image x_t to be calculated from the original x_0 in one step.

A well defined schedule is crucial for stability of the training, ensuring that the model effectively learns to denoise across the full spectrum of signal-to-noise ratios, ultimately dictating the quality of the generated images. Some examples proposed in the literature include:

- Simple linear schedule (original diffusion paper)[33]
- Cosine schedule [37]
- Sigmoid schedule [38]

Important elements to keep in mind is that with a well defined scheduler, the culminative parameter $\tilde{\alpha}_t$ should be monotonically decreasing from $t = 0$ to $t = T$, additionally it should be $\tilde{\alpha}_0 = 1$ at, and $\tilde{\alpha}_T \propto 10^{-5}$ this ensures that no information about the image is retained in the final steps of applying the noise to the image.

4.5 Stable Diffusion

The diffusion model introduced in Section 4.3 operates directly in pixel-space. This approach, while effective, is very expensive computationally. The optimization of the likelihood function in high resolution images can require enormous gradient updates through deep networks. In order to mitigate this issue, Stable Diffusion utilizes the Latent Diffusion Model (LDM) architecture (see fig. 4.8).

This class of models, first introduced by the CompVis group at the Ludwig-Maximilians-Universität München, operates on the premise that the generative learning phase can be separated from the perceptual compression. **Perceptual compression** refers to the reduction of an image to a lower-dimensional space, called latent representation which preserves most important information to human *perception*: shapes, structure, and general semantics, while discarding unnoticeable fine-level details. The researchers at the LMU noticed that these steps can be separated, and the U-Net can learn to perform the semantic composition – generative modeling, while a second model takes care of the perceptual compression.

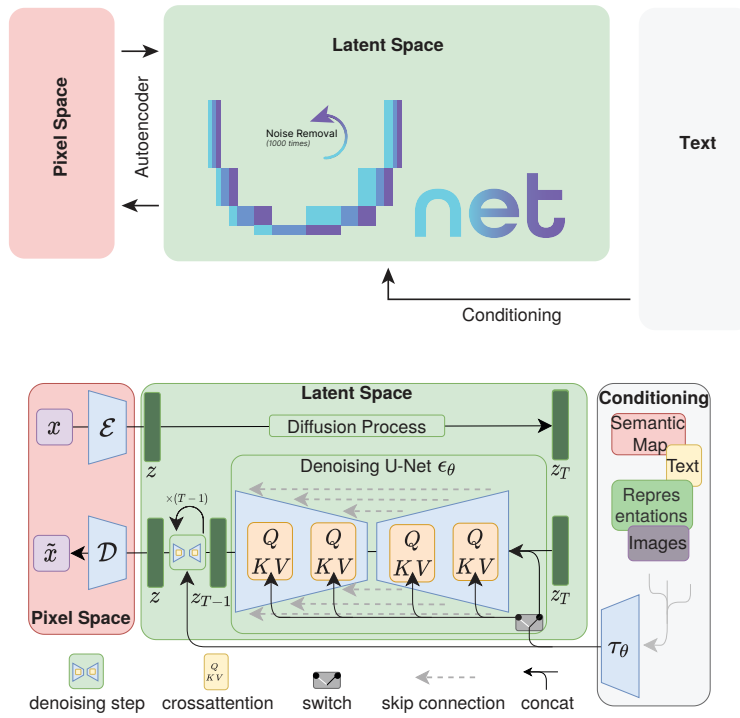


Figure 4.8: LDM Model introduced in [39]. Upper diagram shows the higher level overview of the model. On the bottom part there are all the components of the model shown. Noise x first gets transformed from pixel space to the latent space with an encoder $\mathcal{E}$. Then it can be conditioned via concatenation and run through the U-Net for T times, to later be transformed to the pixel space with a decoder $\mathcal{D}$ to obtain the final image $\tilde{x}$.

4.5.1 Perceptual Compression

In Stable Diffusion, similarly to previously introduced models, the generation process begins with noise. However, in this case, before the noise is feed to the Markovian chain noise predictive process, it gets compressed using a variational autoencoder, similar to the one introduced previously in Section 4.2.1, the autoencoder is pretrained and frozen in the Figure 4.8 it is represented as two trapezoids labeled $\mathcal{E}$ (encoder) and $\mathcal{D}$ (decoder). The shorter sides of the trapezoid point to the latent representation, and the longer sides – to the pixel representation.

The flow of information – just the picture for now, without conditioning – in the model is as follows.

1. **The Encoder $\mathcal{E}$:** Transforms the noise (in the shape of the output image) x from pixel space into a lower-dimensional latent space $z = \mathcal{E}(x)$
2. **The Diffusion Process:** The forward noise injection (training) and the reverse denoising process happen in its entirety in the latent space. The U-Net predicts the noise on the compressed latents, greatly reducing the computational complexity compared to the full-scale input.

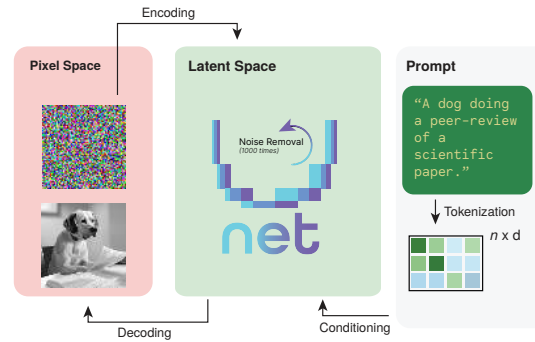


Figure 4.9: Simplified LDM where the concept of conditioning on text is visualized. First, the noise is encoded into the latent space and concatenated with the conditioning representation. throughout the denoisation by the U-Net, conditioning is reapplied through cross-attention mechanism (see fig. 4.8).

3. **The Decoder $\mathcal{D}$:** Once the reverse diffusion process has taken place (typically 500 – 1 000 iterations of the noise removal, creating a denoised latent vector z , the decoder reconstructs the image back into the pixel space $\tilde{x} = \mathcal{D}(z)$.

This addition of the VAE allows the U-net to focus entirely on the semantic and structural generation of the denoised image. This approach not only improves the quality of the generated outputs, but also greatly improves the speed of the generation. Compression of the VAE can be even

4.5.2 Conditioning with Text Encoders

As mentioned before (see sections 4.4.1 and 4.4.3) Latent Diffusion Models make it possible to condition the output of the generation on variation of data using encoders. Here we will explain conditioning using the example of conditioning on text data, as this is the conditioning method, that most readers will be familiar with, as prompting has gotten mainstream in the recent years, and this is the way laymen interact with generative models by far most frequently.

Firstly, our data has to get encoded into a machine readable format, we will show it on an example of one of the first modern encoding strategies – Word2Vec [40] – that has truly revolutionized the field of Natural Language Processing (NLP). This approach used the unsupervised learning techniques to learn the embeddings of words based on their co-occurrence.

Important Disclaimer: This example uses Word2Vec only as an example to illustrate the concept of conditioning. In reality SOTA diffusion models use more advanced, contextual encoders (e. g. CLIP [41]). Word2Vec, due to its static nature, has some drawbacks in encoding complex sentences and dependencies. This is, however, a good model to build the understanding of translating data into a numerical representation.

Step 1: Preprocessing and Tokenization

The initial stage of preparing the text data is breaking it down into constituent words (or parts of words), these are called **tokens**, punctuation and stop words like "a", "the", "on"

can be removed, depending on implementation. For example a sentence "A dog doing a peer-review of a scientific paper.", would be tokenized into ["dog", "doing", "peer", "review", "scientific", "paper"]. Depending on the model the tokenization could also be applied using tokens that are sub words (like in [42, 43, 44]).

Step 2: Encoding

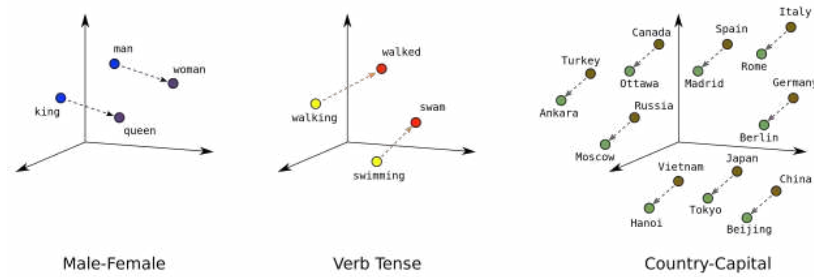


Figure 4.10: The relationships between different words in the Word2Vec model using 3-element Principle Component Analysis (PCA). On this visualization the preservation of the semantic meaning and relationships between words can be seen. $v_{\text{woman}} + (v_{\text{king}} - v_{\text{man}})$ brings us close to the v_{queen} . Similar relation ship can be seen between present continuous and past simple forms of verbs: $v_{\text{swimming}} - v_{\text{swam}} \sim v_{\text{walking}} - v_{\text{walked}}$. Another interesting case for preservation of meaning in the embedding is the relationship between countries and their respective capitals. The direction and length of the vector is similar across examples

After the text is tokenized, its constituent parts get encoded into pre-learned word embeddings. In Word2Vec The embeddings are learned in an unsupervised process. It is a shallow, two-layer neural network trained on a large corpus of text to learn word representations that capture semantic and syntactic relationships based on the assumption of the Distributional Hypothesis – words appearing in similar contexts have similar meanings. It operates using one of two architectures.

- the Continuous Bag-of-Words (CBOW) model or
- the Skip-Gram model

The CBOW model takes a set of surrounding context words within a defined window and attempts to predict the target word in the middle. In the case of the Skip-Gram model it takes a single target word as input and attempts to predict its surrounding words. In both cases, the initial word vectors are randomly assigned, and the network is trained using standard ML methods introduced in Section 3.1: gradient descent and backpropagation to minimize the loss. It iteratively adjusts the vectors so that words frequently appearing together are positioned closer in the resulting high-dimensional vector space. This semantic relationship can be observed in Figure 4.10 At the end of encoding we should have a vector for each of the tokens:

$$\begin{aligned}
 \text{"dog"} &\rightarrow \vec{v}_{\text{dog}} \\
 \text{"doing"} &\rightarrow \vec{v}_{\text{doing}} \\
 \text{"peer"} &\rightarrow \vec{v}_{\text{peer}} \\
 &\vdots
 \end{aligned}$$

Since the embeddings in this example are static, the encoder doesn't preserve the information about the meaning of the sentence as a whole. In more advanced models this limitation is mitigated by the implementations of advanced methods, such as positional encoding or cross attention.

Step 3: Aggregation into a Single Conditioning Vector

As Word2Vec returns one vector for each token, and not an comprehensive *sentence vector* (unlike more advanced methods like ELMo [45], BERT [46], or CLIP [41]). A simple aggregation method is needed to create the final conditioning vector $\vec{c}$. Simplest methods are

- **Averaging:** for example using the arithmetic mean of all the vectors obtained in the encoding step:

$$\vec{c} = \frac{1}{N} \sum_{i=1}^N \vec{v}_i, \quad (4.74)$$

where v_i is the encoding of the i -th token, and N – the number of tokens.

- **Summation:** not ideal for varying number of tokens.

$$\vec{c} = \sum_{i=1}^N \vec{v}_i. \quad (4.75)$$

in our example, the conditioning sentence would be calculated as:

$$\vec{c}_{\text{sentence}} = \frac{1}{6} (\vec{v}_{\text{dog}} + \vec{v}_{\text{doing}} + \vec{v}_{\text{peer}} + \vec{v}_{\text{review}} + \vec{v}_{\text{scientific}} + \vec{v}_{\text{paper}}) \quad (4.76)$$

Reader should keep in mind that this approach is not ideal for complex sentences as the mean operation is inherently lossy, and the semantic meaning of the sentence can be misconstrued by the model.

Step 4: Guiding the Image Generation Model

In the final step the produced numerical conditioning vector is injected into the diffusion model to influence the diffusion process. This is done multiple times in the denoising process by the U-Net. As you can see in Figure 4.8 the attention (see eq. (4.64)) is applied between the context and latent image, where the keys and values matrices are taken from the context, and the queries from the latent image. This attention is applied on each step of the U-Net, both in the up- and down-sampling steps in order to properly apply the conditioning

4.5.3 Time Conditioning

The prompt conditioning is an important concept of the stable diffusion model, allowing to influence the output of the model. However, equally, or even more crucial component is the conditioning on time. The model has to know which step it is currently decoding, both in generation and in training. The timestep is tracked through two interconnected mechanisms: **external iteration** and **internal conditioning**

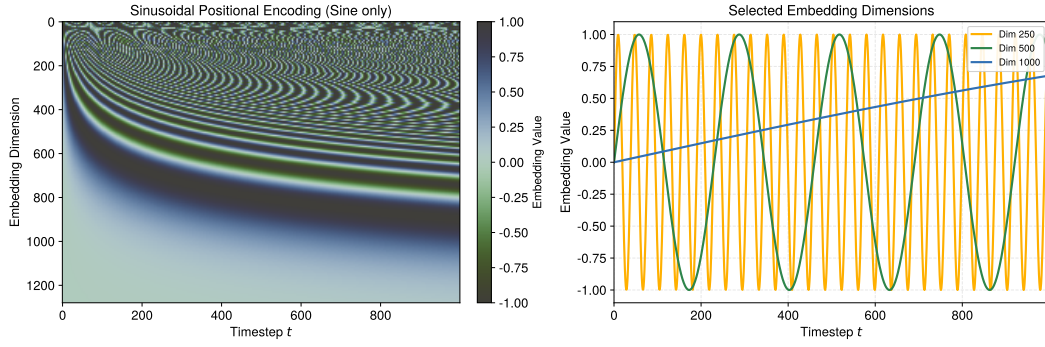


Figure 4.11: Sinusoidal positional encoding using sine function across 1000 timesteps and 128 embedding dimensions. **Left:** Heatmap visualization showing the encoding values for all dimensions, where each row represents an embedding dimension and each column represents a timestep. Lower-indexed dimensions (bottom) oscillate at lower frequencies, while higher-indexed dimensions (top) oscillate at higher frequencies, following the formula $PE(t, i) = \sin\left(\frac{t}{10000^{i/(d-1)}}\right)$ where t is the timestep, i is the dimension index, and d is the embedding dimension. **Right:** Line plots of selected dimensions (250, 500, and 1000) illustrating the frequency progression – lower dimensions capture long-range temporal patterns while higher dimensions encode fine-grained positional information. This multi-scale representation enables the model to distinguish between different timesteps in the diffusion process. Please note, here for clarity only sine function has been used for embedding calculation, in practical application interleaved sine and cosine functions are used to obtain the embedding vectors.

External Tracking

The most fundamental way the timestep is tracked by the model is the sampling algorithm itself. This program runs outside of the U-Net and keeps track of the denoising steps during training.

- **Initialization:** The sampling process begins at the maximum timestep $t = T$, at which $x_T \sim \mathcal{N}(0, \mathbf{I})$ is a pure Gaussian noise
- **Iteration:** The sampler uses a loop to iterate backward from $t = T$ to $t = 0$ through the timesteps.
- **Lookup Table:** at each of the iterations the sampler uses the current integer value to look up the pre-calculated noise schedule parameters $(\beta_t, \alpha_t, \tilde{\alpha}_t)$ required for the denoising formula (see eqs. (4.41) to (4.63)). For a refresher on the noise scheduler please refer to Section 4.4.4.

The sampler is responsible keeping track and managing the state of the process, ensuring that the latent vector x_t is correctly passed to the U-Net to predict x_{t-1}

Internal Tracking: Timestep Conditioning

While the external loop manages the training process externally keeping track of the timestep, the U-Net does not know which timestep it is currently denoising. If there is no information about the time, it would apply the same operation on the input data regardless of the

timestep, which would not produce appropriate results, as $\tilde{\alpha}_t$ is not linear. Therefore Ho et al. proposed the injection of time directly into the latent during denoising in the U-Net [33]. This is called timestep conditioning, as we train the model to behave in a particular way based on the current timestep. The way time is encoded in diffusion models bears resemblance to the positional encoding in NLP applications.

First, the discrete integer timestep $t \in \{1, \dots, T\}$ is transformed into a continuous, high-dimensional vector using a formula that generates a unique vector for each timestep, we want to use a vector, rather than an scalar, as high-dimensional sinusoidal (in our case) embeddings provide a smooth, continuous, and richer representation of time, enabling the U-Net to easily learn the complex, non-linear relationship between the current noise level and the required denoising operation. There exist many functions capable of generating such a "fingerprint"[47]. However, throughout this paper we will be using the encoding used in the original transformer paper [35] for encoding the position of words in a sequence.

This approach is called the **Sinusoidal Positional Embedding**. This approach ensures smoothness and continuousness of our embedding, as well as symmetrically bounds the values between 1 and -1 , stabilizing the training. The embedding is done by applying specific sine and cosine functions with frequency dependent on the timestep and the index in the embedding vector:

$$PE(t, 2i) = \sin\left(\frac{t}{10000^{2i/(d-1)}}\right) \quad (4.77)$$

$$PE(t, 2i + 1) = \cos\left(\frac{t}{10000^{2i/(d-1)}}\right), \quad (4.78)$$

where t is the timestep, i is the index taken from $[0, d/2[$ with d being the dimension of the embedding. The example visualization of the embeddings is shown in Figure 4.11. For clarity the shown embeddings are constrained only to sine functions (without cosine) in the final encoding.

In diffusion models the resulting embedding vector PE_t is then fed through a rescaling MLP, which output is used to generate scaling γ or shifting β parameters or both. These parameters are then injected into U-Net's residual blocks

$$h_{out} = \gamma(PE_t) \times \text{Norm}(h) + \beta(PE_t) \quad (4.79)$$

By modulation the future maps h based on the embedding vector, the U-Net effectively adapts its weights and internal state for every unique timestep. This allows the U-Net to learn a different noise prediction function for each timestep. This is a crucial improvement allowing us to train one universal U-Net rather than T number of U-Nets one for each timestep. **This timestep injection is done in each residual block of the U-Net.**

Dataset

The dataset that was used throughout this work is a dataset of optical images and UV-Vis spectra corresponding to metal oxide compositions and is taken from work by Stein et al. [48, 49]. The dataset contains 180 902 samples out them 1 908 are "blank" samples – meaning for these impressions there is nothing deposited on the substrate. The other 178 994 are metal oxide samples.

5.1 Sample Synthesis

The samples were synthesized via a high-throughput method using ink-jet printing of mixed elemental precursors (between [1, 5]), followed by thermal annealing in air or other O₂-containing atmosphere in varying temperatures between 300 and 800 °C. The workflow leading to the creation of the dataset is shown in the Figure 5.1. There choice of synthesis parameters is neither random, not systematic, as the dataset combines samples created for various experiments conducted by the Joint Center for Artificial Photosynthesis, California Institute of Technology, Pasadena, California in search for pursuing various research projects related to identification of metal oxide photoanodes with a visible bandgap. Which introduces a noteworthy bias in the data not mitigated in this work.

5.2 Data Structure

The dataset contains 180 902 samples, each represented by a combination of structured composition features, spectral measurements, and image data. Each sample is uniquely indexed by both its own ID, and the library plate ID it is physically on.

The composition information is provided by a vector of size 42, labeled `atfrac`, where each entry corresponds to the atomic fraction of an element (the fraction is calculated based on the precursors, and does not take into account the final content of oxygen). The corresponding element labels are given in `atfra_keys`. These elements are:

- Iron (Fe)
- Titanium (Ti)
- Cerium (Ce)
- Tin (Sn)
- Zinc (Zn)
- Cobalt (Co)

- Nickel (Ni)
- Bismuth (Bi)
- Vanadium (V)
- Copper (Cu)
- Manganese (Mn)
- Tungsten (W)
- Molybdenum (Mo)
- Lanthanum (La)
- Chromium (Cr)
- Yttrium (Y)
- Erbium (Er)
- Strontium (Sr)
- Gadolinium (Gd)
- Samarium (Sm)
- Antimony (Sb)
- Niobium (Nb)
- Gallium (Ga)
- Indium (In)
- Aluminum (Al)
- Lead (Pb)
- Magnesium (Mg)
- Calcium (Ca)
- Praseodymium (Pr)
- Europium (Eu)
- Zirconium (Zr)
- Lutetium (Lu)
- Barium (Ba)
- Ytterbium (Yb)
- Phosphorus (P)
- Scandium (Sc)
- Silver (Ag)
- Terbium (Tb)
- Neodymium (Nd)
- Palladium (Pd)
- Hafnium (Hf)
- Rubidium (Rb)

The absorption spectra of the samples are stored as vectors of a size 220 and are labeled spectra, they correspond to the Electro Magnetic Radiation energy in the range $E \in [1.32, 3.10]$ Ev, with the resolution of $\Delta E = 8.127 \cdot 10^{-3}$ Ev and are in electron-volts. The exact energy values corresponding to spectra are under energy_eV as a vector of size 220.

The loadings, which are the averaged amount of a given element per unit of space in a given sample. In this case the units are $\text{nmol} \cdot \text{cm}^{-2}$, and the values are stored under the name loadings, with the labels under loading_keys. The loading vector size is bigger than that of compositions, as it also stores the total amount of material per unit space and is the sum of the component parts. This data is not used in this work.

The images are stored in form of $64 \times 64 \times 3$ matrices, where the two first dimensions are the width and height of the image, and the third one – the channels corresponding to the Red, Green, and Blue (RGB) values. They are stored in the dataset under the name images.

The samples are distinguished by a unique sample_id and a plate_id on which the sample was printed. An auxiliary dataset [50] can be used to obtain the specific information on

Table 5.1: Structure of the dataset and shapes of all stored arrays, atfrac is transposed compared to other data to increase loading speeds in python.

Name	Shape	Dtype	Description
atfrac	(42, 180902)	float32	Atomic fractions (42 elements per sample)
atfrac_keys	(42)	string	Names of the 42 atomic fraction channels
energy_eV	(220)	float32	Energy grid for spectral data (in eV)
images	(180902, 64, 64, 3)	float32	RGB sample images (64×64 pixels)
loading_keys	(43)	string	Names of the 43 loading channels
loadings	(43, 180902)	float32	Precursor/oxide loading values
plate_id	(180902)	int64	Plate identifiers for samples
sample_id	(180902)	int64	Unique sample identifiers
spectra	(180902, 220)	float32	UV–Vis spectra for each sample

the experimental synthesis details, such as the annealing temperature on most of the plates present in the main dataset.

The summarized description of the dataset is shown in Table 5.1, where you can see all the keys in the dataset, together with the shape of the corresponding data, the datatype of the data, and a short description. A more intuitive visualization of the components of the dataset can be seen in Figure 5.1. There the categories of the data are showcased together with an example of data, and a visualization of that example. In the slide you can see how a typical image or UV-Vis spectrum of a sample looks like.

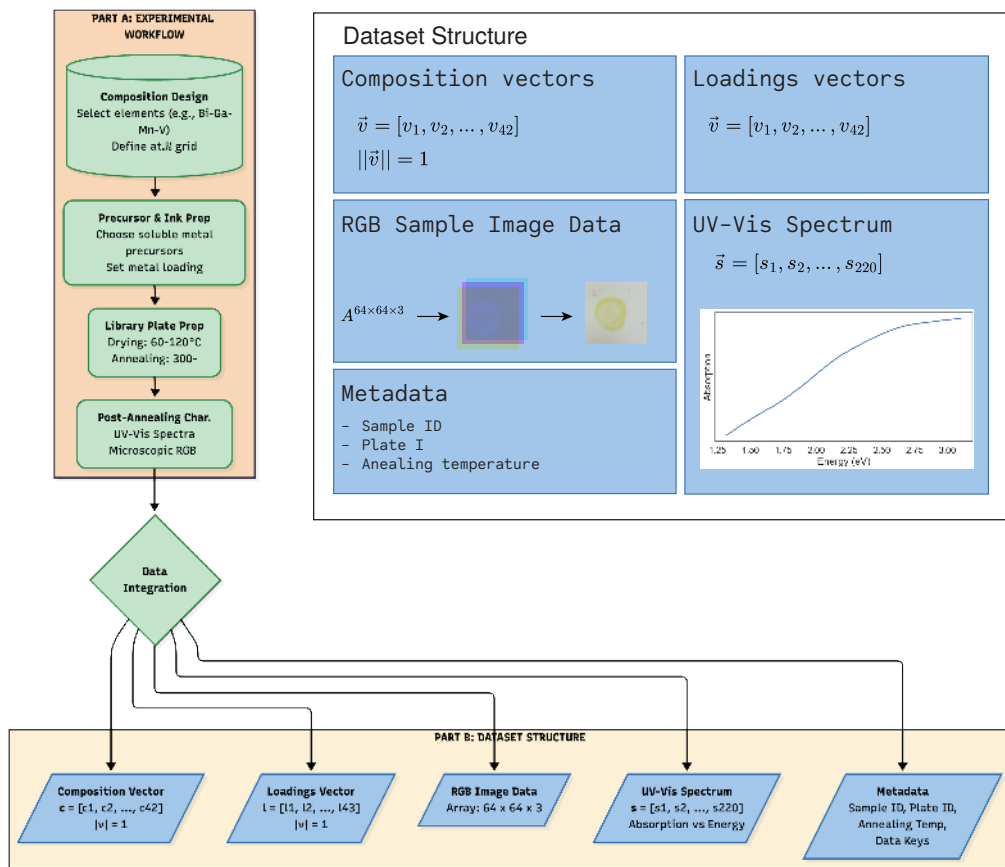


Figure 5.1: High throughput synthesis workflow and dataset structure. The process begins with composition design. This is then followed by preparation of appropriate inks and precursors. The library plate is prepared by depositing the appropriate amounts of inks to create the desired material compositions. After printing the plate is first dried and then annealed to convert the metals to oxides. Samples get characterized by UV-Vis spectra and RGB images of the samples. **Insert:** Dataset structure. The resulting dataset comprises of: composition vectors, where each element of the vector corresponds to the fraction of an element in the final composition; image data – an RGB sample image of the size 64x64; spectral UV-Vis absorption data from 1.32 to 3.10 eV, with the resolution of 0.008127 eV; and metadata that includes sample ID, plate ID, and annealing temperature.

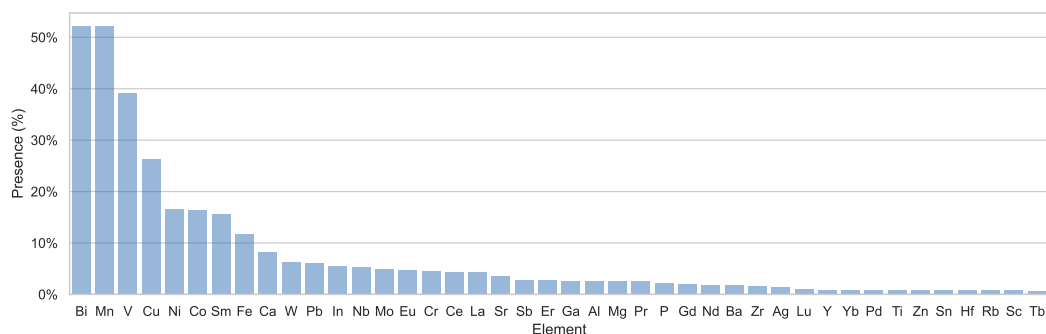


Figure 5.2: Frequency distribution of element presence. This chart details the prevalence of the chemical elements present in the dataset. The horizontal axis lists all the elements included in the datasets. On the vertical axis, the percentage of compositions in the dataset, where the given element is present in any amount is shown.

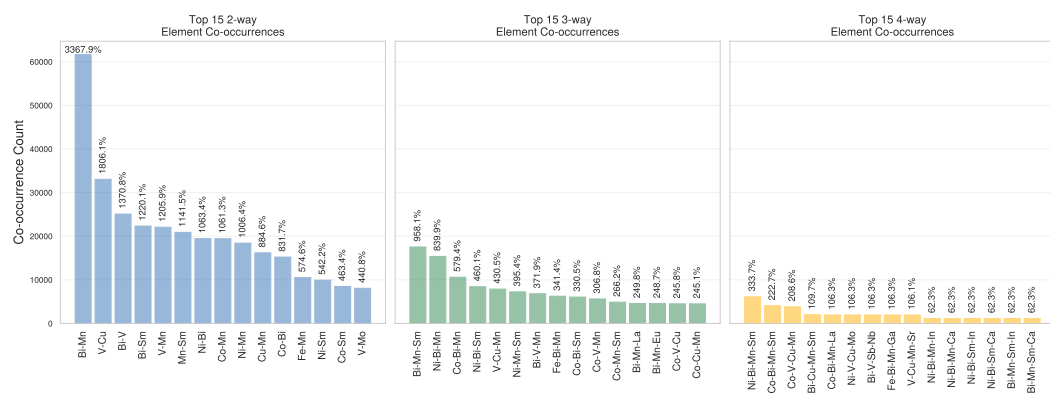


Figure 5.3: The frequency of different binary, ternary, and quaternary systems in the dataset

5.3 Statistical Analysis of the Features

Here, the features of the dataset – the compositions will be explained using statistical methods to familiarize the reader with the data further.

The compositions present in the dataset are not uniformly distributed, meaning that the elements appear in the data with varying frequency, i.e. there is more compositions containing elements like Bismuth, Manganese, or Vanadium; compared to Hafnium or Rubidium.

This bias was not mitigated in the current work, however, it is important to note this imbalance. The presence, which is the percentage of compositions where a given element is present in the whole dataset, can be seen in Figure 5.2. From this we can see, that the most prevalent elements in the datasets are: bismuth being present in 52% of compositions, Mn (52%), V (39%), Cu (26%). While the least common are: Y, Yb, Pd, Ti, Zn, Sn, Hf, Rb, Sc, Tb with all having presence in the range of $]0.5, 1.0[\%$.

Co-occurrence Analysis (see figs. 5.3 and 5.4) of the compositions gives us more inside into the biases in the data. We can see how common different co-occurrences are in the set, meaning which elements are likely to be found together. There is a high number of gray points, which means that a lot of combinations are not present (633 out of 882) in such a large ex-

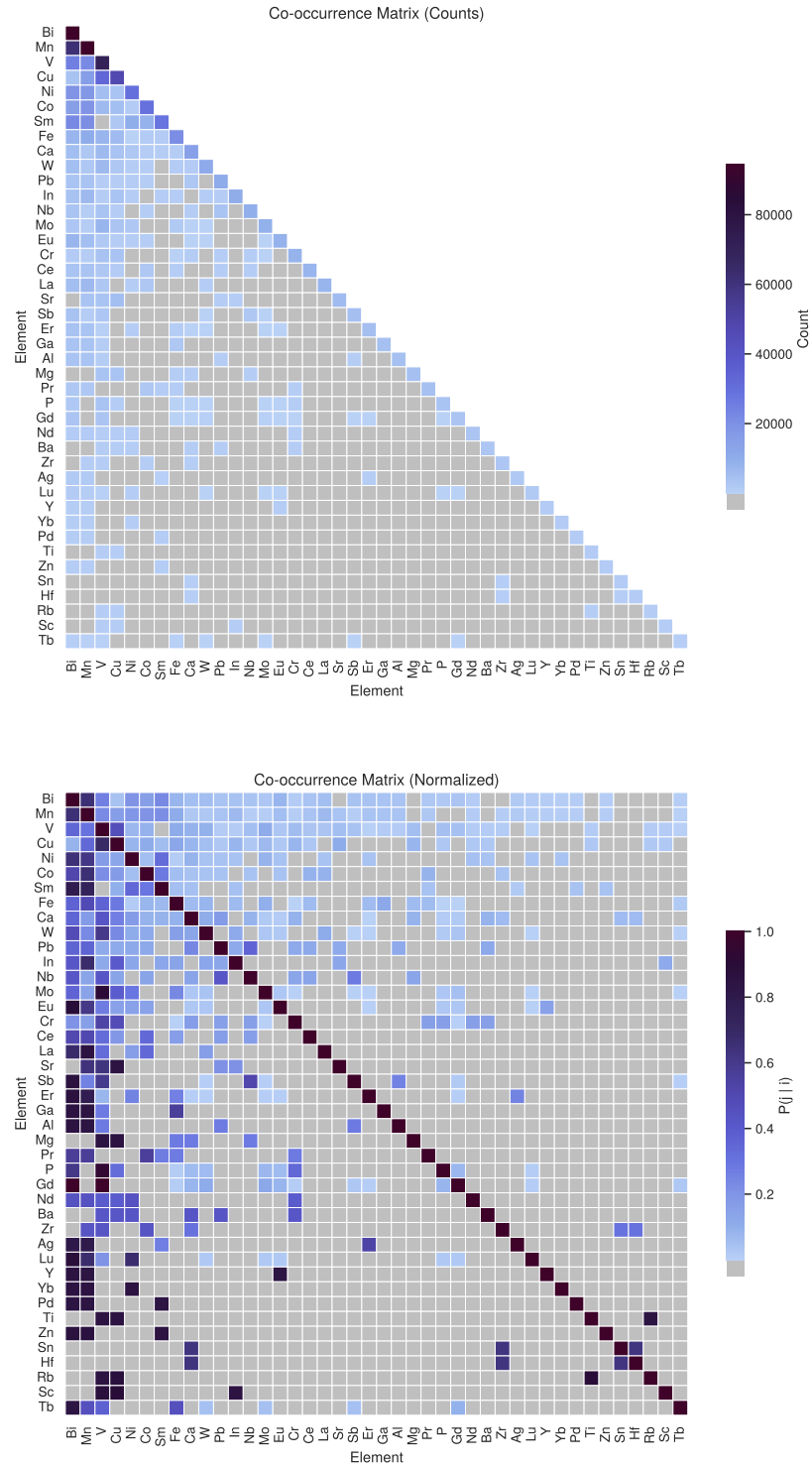


Figure 5.4: Elemental Co-occurrence Analysis. Top: The raw count matrix highlights the absolute frequency of paired elements. The dark top cluster involving Bi, Mn, V, and Cu confirms these are the most heavily sampled elements in the dataset. Bottom: The normalized matrix ($P(j|i)$) reveals the conditional probability of element presence. The prominent dark vertical bands at Bi, V and Mn indicate that these elements act as ubiquitous “host” materials or base components, appearing for nearly every other specific element in the dataset in high probability.

Table 5.2: Statistics of n -way element co-occurrences in the dataset of 180,902 material samples across 42 elements. The table shows how the combinatorial space explodes with increasing order while actual occurrences become increasingly sparse.

Order	Possible	Non-zero	Zero	Sparsity
2-way (Binary)	861	228	633	73.5%
3-way (Trinary)	11,480	300	11,180	97.4%
4-way (Quaternary)	111,930	116	111,814	99.9%

perimental dataset like this. This underlines the curse of dimensionality, our model tries to mitigate – based on observed combinations, predict properties of unseen combinations. As with the increase of number of elements in a combination, according to the formula for a binomial coefficient

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}, \quad (5.1)$$

the number of possible combinations grows exponentially (for $n \gg k$), where n is the number of elements to choose from (in our case 42), and k is the size of a combination. With this rapid growth, the sparsity of our dataset also increases with the combination size (see table 5.2). As mentioned before for a binary combination, we cover only 228 out of possible 882 combinations, that is 32.5%. However, for ternaries and quaternaries this ratio is even worse, with the respective coverages being 2.6% and 0.1% (!!!).

5.4 Absorption Spectra

5.4.1 Statistical Analysis of Optical Properties

In this work the introduced models predict both the RGB images of samples, as well as the UV-Vis spectra. In this section, a statistical analysis of one of these targets – spectra – is performed. The goal of this analysis is to ensure that the machine learning model is not merely learning a trivial mean representation but is capturing the diverse optical behaviors present in the material library.

Figure 5.5 (Left) displays the mean UV-Vis absorption spectrum across the entire dataset ($N = 180\,902$). The mean absorption (solid blue line) exhibits a characteristic increase with photon energy, rising from approximately 0.08 a.u. at 1.3 eV to 0.30 a.u. at 3.1 eV. Crucially, the standard deviation ($\pm\sigma$) is substantial, spanning nearly ± 0.1 a.u. around the mean, the variance is highest for small energies and increases proportionally with E . This wide variance confirms that the dataset contains a high degree of spectral diversity, ranging from highly transparent to dark samples rather than clustering tightly around a single material behavior indicating high diversity of materials. To quantify the overall opacity of the samples, the integrated absorption was calculated for each sample by summing the spectral intensity over the measured energy range (see fig. 5.5 (Right)). The frequency distribution of the samples' integrated absorption is either uni-modal with a right skew, or bimodal, with one mode much stronger than the other one; with a mean of 0.33 a.u. and a median of 0.31 a.u. Two distinct features in this distribution are noteworthy for the learning process:

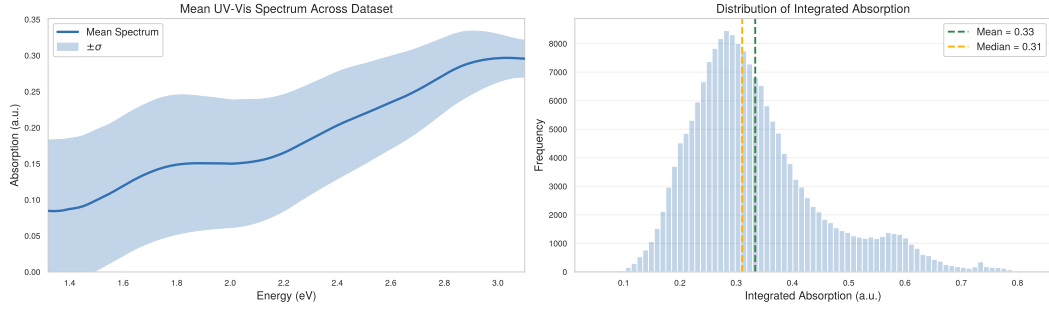


Figure 5.5: Statistical characterization of the optical absorption targets. **(Left)** The mean UV-Vis absorption spectrum across the dataset. The solid line represents the mean spectral shape across all samples ($N = 180\,902$), the shaded region indicates the standard deviation ($\pm\sigma$). **(Right)** Histogram of the integrated absorption intensity. The distribution is right-skewed with a mean of 0.33 (green dashed line) and median of 0.31 (yellow dashed line). A secondary shoulder around 0.6 a.u. indicates a distinct sub-population of materials with higher absorption.

- **Dominance of Moderate Absorbers:** The majority of the probability mass lies in the interval $[0.2, 0.4]$ a.u. This indicates that the dataset is dominated by materials with moderate transparency.
- **Secondary High-Absorption Population:** A distinct “shoulder” or secondary mode is observable in the range of $0.50 - 0.60$ a.u. This suggests the existence of a specific sub-population of strongly absorbing materials. Upon closer inspection, the dataset was found to contain plates, where the substrate plate differed compared to others. The plates with the following indices `plate_id = {1304, 1328, 3028}` were found to be silicon, non-transparent plates, and the corresponding samples, 2 346 of them were removed from the dataset. The removal of the outliers, doesn’t however mitigate the bump in the higher integral values. Therefore, the higher bump shall be attributed to materials with higher absorption.

5.4.2 Estimation of Optical Bandgaps

To quantitatively characterize the electronic structure of the material library, the optical bandgap (E_g) was estimated for each sample using the Tauc method [51]. While the samples exhibit varying degrees of structural disorder, the optical absorption in transition metal oxides is primarily governed by local metal-oxygen coordination rather than long-range lattice periodicity. Consequently, the absorption edge was modeled assuming a direct allowed transition. While this assumption may not be true for all the samples (they might be amorphous, or the bandgap may be indirect), it has been shown to provide robust estimates for combinatorial oxide library [52]. The Tauc plot was generated by transforming the absorption α using the formula:

$$y = (\alpha h\nu), \quad (5.2)$$

where y is the energy dependent value of the Tauc plot, and $h\nu$ is the photon energy. Traditionally the bandgap can be obtained by finding the linear segment of the plot, and extrapolating it to $h\nu = 0$ eV.

In this work it was done algorithmically. The method was inspired by the work of Suram et.al. [52], but a simplified model was used, there was no baseline extraction due to high thin-layer interference in the spectra of substrates. The tauc plots were smoothed using the

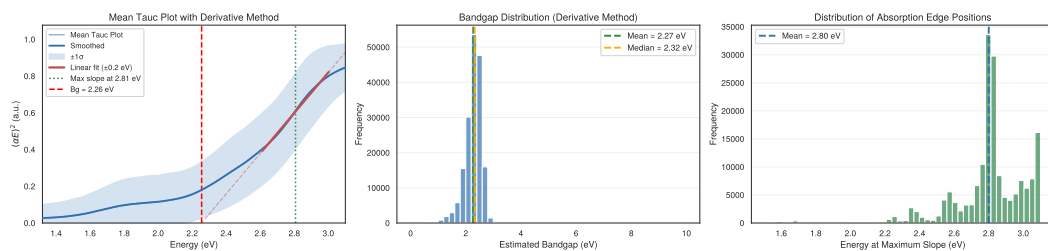


Figure 5.6: Statistical analysis of optical bandgaps (E_g) estimated assuming direct allowed transitions and the Tauc method. **(Left)** The mean Tauc plot (for the whole dataset), showing a distinct linear region characteristic of direct transitions. The linear fit (red line) extrapolates to a mean bandgap of 2.26 eV. **(Middle)** Histogram of estimated E_g . The distribution is centered at 2.27 eV. **(Right)** Distribution of the energy at maximum slope. The peaks in the 2.8 – 3.0 eV range indicate a significant population of relatively wide-bandgap materials, consistent with transparent conductive oxide hosts used in the library.

Savitzky-Golay filter [53]. Then, the linear fit region was chosen based on the first derivative of the smoothed Tauc curve (the highest derivative was chosen). This is the region of the highest slope and can be estimated as the fundamental absorption edge, then a linear fit was applied to obtain the. The detailed method can be found in the appendix in Section B.1. The bandgap statistics obtained in this way can be found in Figure 5.6.

The distribution of estimated bandgaps (see fig. 5.6. Middle) reveals a mean value of 2.27 eV and a median of 2.32 eV. This energy range corresponds to the visible light spectrum, however, they are on the higher end for photoactive materials. The distribution of absorption edge positions (see fig. 5.6. Right) shows a very high population around 2.8 eV.

This analysis, while interesting from the point of visualization of the dataset, no conclusion regarding the electronic structure should be drawn from it, as the Tauc method while being reliable for pure materials is not very robust against dopants or other impurities. As mentioned in the Section 5.1 the synthesis method may not free from impurities, as there are multiple materials annealed at the same time, not within an ultra-pure environment.

5.4.3 Principal Component Analysis of the spectral data

To further understand the dataset, a Principal Component Analysis (PCA) was performed on the UV-Vis absorption data. This dimensionality reduction technique allowed for projection of the high dimensional (220) data into a simplified 2D latent space, while retaining most of the relevant information.

The projection of the dataset onto the first two principal components (PC1 and PC2) reveals a continuous, non-clustered distribution, indicating a smooth variation in local chemical environments across the dataset. As you can see in Figure 5.7A, the first two components are highly effective at simplifying the spectra, accounting for 93.3% of the total variance, with PC1 contributing 76.4% and PC2 – 16.9%. The highest density of samples is concentrated in the lower central region of the plot

In order to correlate the abstract PC coordinates with actual UV-Vis spectral features, the PCA plot was divided into 25 sub-regions and local average spectra were sampled in each sub-region (see fig. 5.7B). From the plots on the grid we can draw conclusions what trends

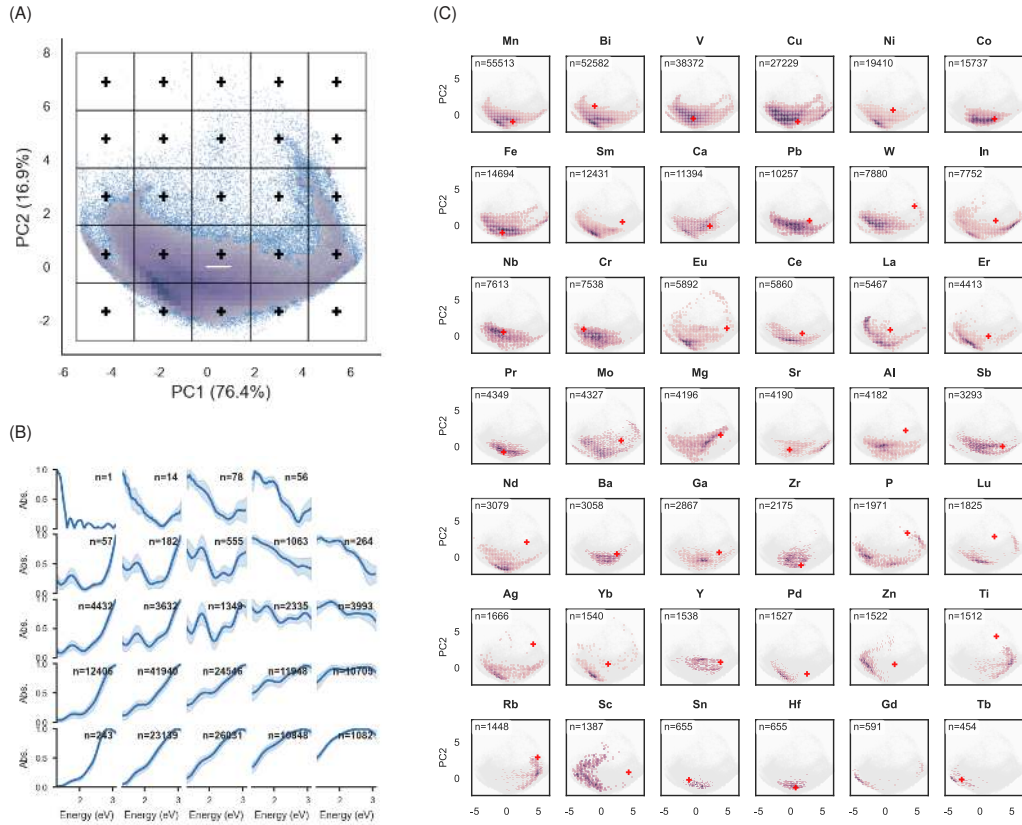


Figure 5.7: Principal Component Analysis (PCA) of spectral data and elemental distributions. (A) Density plot of the dataset projected onto the first two principal components. PC1 accounts for 76.4% of the variance, and PC2 accounts for 16.9%. The overlay grid and + markers indicate specific coordinates within the PC space used to sample local spectral characteristics. (B) Averages spectral profiles corresponding to the grid locations defined in (A). The plots display the normalized absorption intensity versus Energy. The blue lines represent the average spectrum at a corresponding location, while the shaded regions indicate one standard deviation from the average. The value n in each subplot represents the number of samples located within that region. (C) Element-wise projection of data onto the PC1-PC2 space. Each subplot shows the distribution density for compositions containing a specific element. The plots are ordered by sample count (n), ranging from high-frequency elements to lower-frequency elements. The red cross in each subplot marks the pure oxide's spectrum – only the relevant element and oxygen.

the PCs capture. While PC1 captures the *flatness* of the spectrum – the larger the component, the flatter the spectrum. PC2 on the other hand is responsible for the low-energy bump – for high values the low energy absorption is higher, than for low PCs. This way of plotting allows us to see the structure of the dataset in detail. Notably, regions with high sample counts (n) exhibit very low variance implying high structural redundancy in these regions. In contrast, regions with low sample counts display higher variance, reflecting a broad diversity of unique (perhaps outlier) local structures. This aligns with the analysis of the PCA density plot, where the higher populated grid cells exhibit high localization of sample-points, while for the less dense cells (upper part of the PCA plot) the samples are distributed more evenly across the cell.

Decomposing the global distribution by element in Figure 5.7C reveals how specific atomic

species influence the spectral fingerprint of the sample. The distributions are ordered by element frequency, which aligns with the Elemental Co-occurrence Analysis in Section 5.3. High frequency elements exhibit broad distributions covering large portions of the PC space, which is due to them being part of very diverse compositions, as the backbones of these compositions. The more localized elements suggest that either that their oxides exhibit very strong absorption *overshadowing* (literary) the weaker absorbing oxides in their systems, or that, in the dataset, they are combined only with a few different cations, leading to similar spectra. However interesting, these conclusions should be taken with a grain of salt, as often the interactions between different elements can be very complex, due to alloying, different crystallographic structures across one system and so on.

Comparison with the pure oxide standards (the spectra of only one cation and oxygen) – marked by red crosses – provides an inside into the alloying effects. For some materials like Iron (Fe), Chromium (Cr), the spectral density is centered near the red cross implying that the local environment often resembles that of the binary oxide. However, for elements like Calcium (Ca) and Lanthanum (La), the distributions are significantly offset from their respective pure oxide markers. This deviation suggests that in complex multi-component oxides, these elements are forced into coordination geometries or electronic states that differ distinctively from their simple oxide forms.

5.5 The Image Data

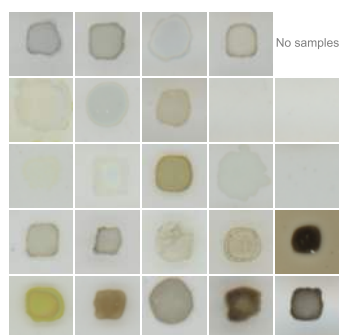


Figure 5.8: Different sample images. Each image is randomly chosen from a subset of samples, that correspond to grid on the PCA plot for spectra in the previous section.

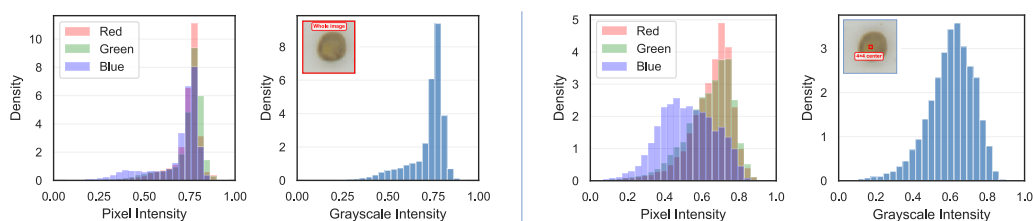


Figure 5.9: Statistical analysis of sample image pixel densities. The two histograms on the (Left) show pixel statistics for the full 64×64 images, showing a dominant peak corresponding to the bright background. (Right) histograms show the statistics for a cropped version of the images – 4×4 region signified on the insert, this isolates the material from the background.

Last one of the three most important data categories in the dataset, together with compositions and spectra, are the sample images. Some examples of this data class can be seen in Figure 5.8, these images correspond to the samples grouped according to the spectral PCA plot in Figure 5.7A. The images have format $64 \times 64 \times 3$, where the first two dimensions are the width and height in pixels, while the third one is the number of channels: Red, Green and Blue.

A statistical analysis of the pixel intensities in Figure 5.9 reveals that the sample images are dominated by the background substrate, which creates a high density peak at approximately 0.8 intensity. When only the central region of the samples is considered (see fig. 5.9(Right)), the red and green distributions are shifted towards higher intensities, while the blue distribution – towards lower. The lower intensity of the blue channel indicates that the material strongly absorbs blue light, while transmitting red and green. This physically correlates to the brownish-yellow colour of the metal oxide samples.

In the context of optical absorption, the drop in blue intensity corresponds to a higher absorption coefficient at lower wavelengths (higher energy), which is consistent with the analysis of the UV-Vis spectra in Section 5.4.1.

Models' Implementation and Training

This chapter introduces the specific model architectures used in this work and builds upon the theory introduced in Chapters 3 and 4 as well as training, validation schemes and hyperparameter tuning.

Firstly the train, validation and test splitting will be introduced, followed by the spectra prediction model architecture. After that, the main focus of the work – the generative diffusion model for image prediction will be discussed together with the encoding schemes used across the work.

6.1 Data Preprocessing and Splitting

One of the main goals pursued in this work is the prediction of unseen compositions. This means that we want our model to predict properties of oxide sample compositions never before seen by the model.

Therefore, the test split of the dataset needs to contain compositions unseen before by the model, neither in testing, nor validation step. In practice this means that the data is divided first into ternary systems and then some systems are assigned to the test split – these systems are strictly guarded against making their way into train and validation splits. All the samples corresponding to these ternaries, together with quaternaries, that contain the three elements are assigned to the `test_1` set. The rest of the systems are assigned into a `shared_data` container, which is further split into `test_2`, `train` and `validation` sets, the

Table 6.1: Ternary systems that exist only in the test set.

#	System	#	System	#	System
1	Bi-Cu-Eu	3	Bi-Ni-Sm	5	Cu-Fe-In
2	Bi-Mn-Ni	4	Ca-Cr-Pb	6	Cu-Mg-V

To ensure data quality and robustness, a multi-stage outlier filtering approach was applied to the spectral measurements. For each unique composition, multiple spectra were aggregated using a statistical filtering method based on Z-score analysis with a threshold of 2.0.

Table 6.2: All ternary systems in the training set.

#	System	#	System	#	System	#	System
1	Ag-Bi-Er	54	Bi-Ga-Mn	107	Ca-Sn-Zr	160	Cu-Pb-Sr
2	Ag-Bi-Mn	55	Bi-Ga-V	108	Ce-Co-Mn	161	Cu-Pb-V
3	Ag-Bi-Sm	56	Bi-Gd-V	109	Ce-Co-Nb	162	Cu-Rb-Ti
4	Ag-Er-Mn	57	Bi-In-Mn	110	Ce-Co-Pb	163	Cu-Rb-V
5	Ag-Mn-Sm	58	Bi-In-Ni	111	Ce-Cu-V	164	Cu-Sc-V
6	Al-Bi-Mn	59	Bi-In-Pb	112	Ce-Fe-Mn	165	Cu-Sr-V
7	Al-Bi-Pb	60	Bi-In-Sm	113	Ce-Mn-V	166	Cu-Ti-V
8	Al-Bi-Sb	61	Bi-In-W	114	Ce-Nb-Pb	167	Cu-V-W
9	Al-Bi-V	62	Bi-La-Mn	115	Co-Cu-Mn	168	Er-Fe-Mn
10	Al-Mn-Pb	63	Bi-La-Ni	116	Co-Cu-V	169	Er-Mn-Ni
11	Al-Mn-Sb	64	Bi-La-V	117	Co-Eu-Mn	170	Eu-Mn-Ni
12	Al-Mn-V	65	Bi-Lu-Mn	118	Co-Fe-Mn	171	Eu-Mn-Y
13	Ba-Ca-Ni	66	Bi-Lu-Ni	119	Co-Fe-Ni	172	Fe-Ga-Mn
14	Ba-Ca-Pb	67	Bi-Lu-V	120	Co-Fe-Pr	173	Fe-In-Mn
15	Ba-Cr-Cu	68	Bi-Mn-Nb	121	Co-Fe-Sm	174	Fe-Mg-V
16	Ba-Cr-V	69	Bi-Mn-Nd	122	Co-La-Mn	175	Fe-Mn-Mo
17	Ba-Cu-V	70	Bi-Mn-Pb	123	Co-Mn-Ni	176	Fe-Mn-Ni
18	Ba-Ni-Pb	71	Bi-Mn-Pd	124	Co-Mn-Pr	177	Fe-Mn-Sm
19	Bi-Ca-Co	72	Bi-Mn-Pr	125	Co-Mn-Sm	178	Fe-Mn-Tb
20	Bi-Ca-Mn	73	Bi-Mn-Sb	126	Co-Mn-V	179	Fe-Mn-V
21	Bi-Ca-Ni	74	Bi-Mn-Sm	127	Co-Mn-Zr	180	Fe-Mo-V
22	Bi-Ca-Sm	75	Bi-Mn-Tb	128	Co-Nb-Pb	181	Fe-Pr-Sm
23	Bi-Ca-V	76	Bi-Mn-V	129	Co-Ni-Sm	182	Fe-V-W
24	Bi-Ca-W	77	Bi-Mn-W	130	Co-Pr-Sm	183	Ga-Mn-V
25	Bi-Ce-Co	78	Bi-Mn-Y	131	Co-V-Zr	184	Hf-Sn-Zr
26	Bi-Ce-Fe	79	Bi-Mn-Yb	132	Cr-Cu-Nd	185	In-Mn-Pb
27	Bi-Ce-Mn	80	Bi-Mn-Zn	133	Cr-Cu-P	186	In-Mn-Sr
28	Bi-Ce-V	81	Bi-Mo-V	134	Cr-Cu-V	187	In-Mn-W
29	Bi-Co-Eu	82	Bi-Nb-Pb	135	Cr-Mn-Pr	188	In-Sc-V
30	Bi-Co-La	83	Bi-Nb-Sb	136	Cr-Nb-Pb	189	La-Mn-Ni
31	Bi-Co-Mn	84	Bi-Nb-V	137	Cr-Nd-V	190	La-Mn-V
32	Bi-Co-Ni	85	Bi-Nd-Ni	138	Cr-P-V	191	La-Mn-W
33	Bi-Co-Pr	86	Bi-Ni-V	139	Cu-Eu-Mn	192	La-V-W
34	Bi-Co-Sm	87	Bi-Ni-Yb	140	Cu-Fe-Mg	193	Lu-Mn-Ni
35	Bi-Co-W	88	Bi-P-V	141	Cu-Fe-Mn	194	Mg-Nb-V
36	Bi-Cr-Mn	89	Bi-Pd-Sm	142	Cu-Fe-Mo	195	Mn-Mo-V
37	Bi-Cr-Pr	90	Bi-Sb-V	143	Cu-Fe-V	196	Mn-Nb-Pb
38	Bi-Cr-V	91	Bi-Sm-Zn	144	Cu-Fe-W	197	Mn-Nd-Ni
39	Bi-Cu-Mn	92	Bi-Tb-V	145	Cu-In-Mn	198	Mn-Ni-Sm
40	Bi-Cu-Sm	93	Bi-V-W	146	Cu-In-Sc	199	Mn-Ni-V
41	Bi-Er-Fe	94	Ca-Co-W	147	Cu-In-Sr	200	Mn-Ni-Yb
42	Bi-Er-Mn	95	Ca-Cr-Nb	148	Cu-In-V	201	Mn-Pd-Sm
43	Bi-Er-Ni	96	Ca-Cu-Fe	149	Cu-Mg-Nb	202	Mn-Sm-Zn
44	Bi-Er-V	97	Ca-Cu-Mg	150	Cu-Mn-Sm	203	Mn-Sr-V
45	Bi-Eu-Mn	98	Ca-Cu-Mn	151	Cu-Mn-Sr	204	Mn-V-W
46	Bi-Eu-Ni	99	Ca-Cu-V	152	Cu-Mn-V	205	Mn-V-Zr
47	Bi-Eu-V	100	Ca-Fe-V	153	Cu-Mo-Ni	206	Mo-Ni-V
48	Bi-Eu-Y	101	Ca-Hf-Sn	154	Cu-Mo-V	207	Nb-Sb-V
49	Bi-Fe-Ga	102	Ca-Hf-Zr	155	Cu-Nb-V	208	Ni-V-W
50	Bi-Fe-Mn	103	Ca-Mg-V	156	Cu-Nd-V	209	Pb-Sr-V
51	Bi-Fe-Sm	104	Ca-Mn-V	157	Cu-Ni-V	210	Rb-Ti-V
52	Bi-Fe-Tb	105	Ca-Nb-Pb	158	Cu-Ni-W		
53	Bi-Fe-V	106	Ca-Ni-Pb	159	Cu-P-V		

1. Spectra exhibiting anomalous ascending trends in their final quarter were identified using linear regression and excluded from further analysis.
2. For compositions with only two measurements, the spectrum with the lower total intensity was selected to mitigate potential experimental artifacts.
3. For groups with three or more measurements, Z-scores were computed element-wise across all spectral features and only measurements where all features fell within the Z-score threshold were retained.

This approach effectively removed outlier spectra while preserving the representative spectral signatures for each composition. Additionally all empty samples were removed. This led to elimination of 2 347 samples.

Then the aforementioned splitting scheme was performed:

1. 3% of ternary systems were chosen as the *unseen* set. These systems are listed in Table 6.1, while all the rest of the systems can be seen in Table 6.2.
2. All the samples corresponding to the *unseen* systems, meaning all the ternary and quaternary compositions containing the six chosen combinations, were put into the `test_1` split. The number of samples in this split is 23 934, which is equal to 13% of the dataset.
3. The remaining samples were further split into `train:validation:test_2` at a ratio of 8 : 1 : 1 for the spectral model and 9 : 0 : 1 for the generative diffusion model.
4. Subsets `test_1` and `test_2` were combined into single test set – `test`.

6.2 Latent Diffusion Generative Model

The objective of this work is to develop a predictive model capable of generating material properties – specifically images and spectra from solely elemental compositions. In order to achieve the first part – image generation, a Latent Diffusion Model (LDM) architecture was implemented. This approach was chosen as it may address the issue of prediction of new phase generation in complex oxide systems. This chapter details the specific architecture, the conditioning mechanisms, and the training protocol of the model.

6.2.1 Architecture Overview

The model architecture follows the Latent Diffusion Model paradigm introduced in Section 6.2. This model (see fig. 6.1) combines a variational autoencoder, used for changing from the image space into a latent space, subsequently changing the dimension of the images: $64 \times 64 \times 3 \rightarrow 8 \times 8 \times 4$; and the composition encoder for changing the compositions from human readable into a machine readable format. These two data formats are fed into the denoising mechanism of the model. The denoising is done using the U-net architecture, where the conditioning vectors and the time embedding vectors are re-fed throughout the network to condition the noise removal. The noise removal is conducted for a 1 000 times using the U-net to obtain the final image of a sample for a given composition. In the following sections all the elements of the model will be discussed in detail.

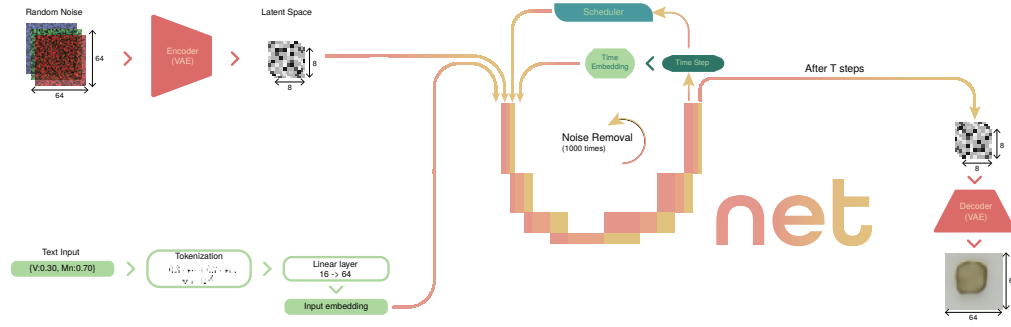


Figure 6.1: High view architecture of the LDM model. This diagram illustrates the composition-to-image generation pipeline within a Latent Diffusion Model. The process begins by converting Text Input into an Input embedding via Tokenization and a Linear layer. Simultaneously, Random Noise is mapped into a compressed Latent Space by an Encoder (VAE). The core of the model is an iterative Noise Removal loop (the "U-net") that processes these latents over T steps. A Scheduler manages this cycle by injecting a specific Time Step and Time Embedding into the network at each iteration. Once the noise is fully removed, a Decoder (VAE) transforms the final latent representation back into a pixel-based image.

6.2.2 Variational Autoencoder

The role of the Variational Autoencoder (VAE) is to reduce the size of the space the U-Net has to denoise by performing the perceptual compression of the noise by allowing the noise removal steps to focus on the semantic and structural generation of the final image, disregarding irrelevant or redundant part of the image.

In this work the VAE was designed so that the semantic meaning of the image is preserved, while reducing the number of elements in the image tensor 48 fold, from $64 \times 64 \times 3$ to $8 \times 8 \times 4$. To achieve this, the encoder and decoder were designed using a combination of convolution blocks, residual connections, self attention, group normalization, and up-sampling – detailed architecture can be seen in Figure 6.2.

The sequence of the blocks in the encoder is as follows:

- Conv2d: $3 \rightarrow 128$ channels, kernel 3, padding 1
- VAE_ResidualBlock: 128 channels
- VAE_ResidualBlock: 128 channels
- Conv2d (Downsample): 128 channels, stride 2, kernel 3
- VAE_ResidualBlock: $128 \rightarrow 256$ channels
- VAE_ResidualBlock: 256 channels
- Conv2d (Downsample): 256 channels, stride 2, kernel 3
- VAE_ResidualBlock: $256 \rightarrow 512$ channels
- VAE_ResidualBlock: 512 channels
- Conv2d (Downsample): 512 channels, stride 2, kernel 3

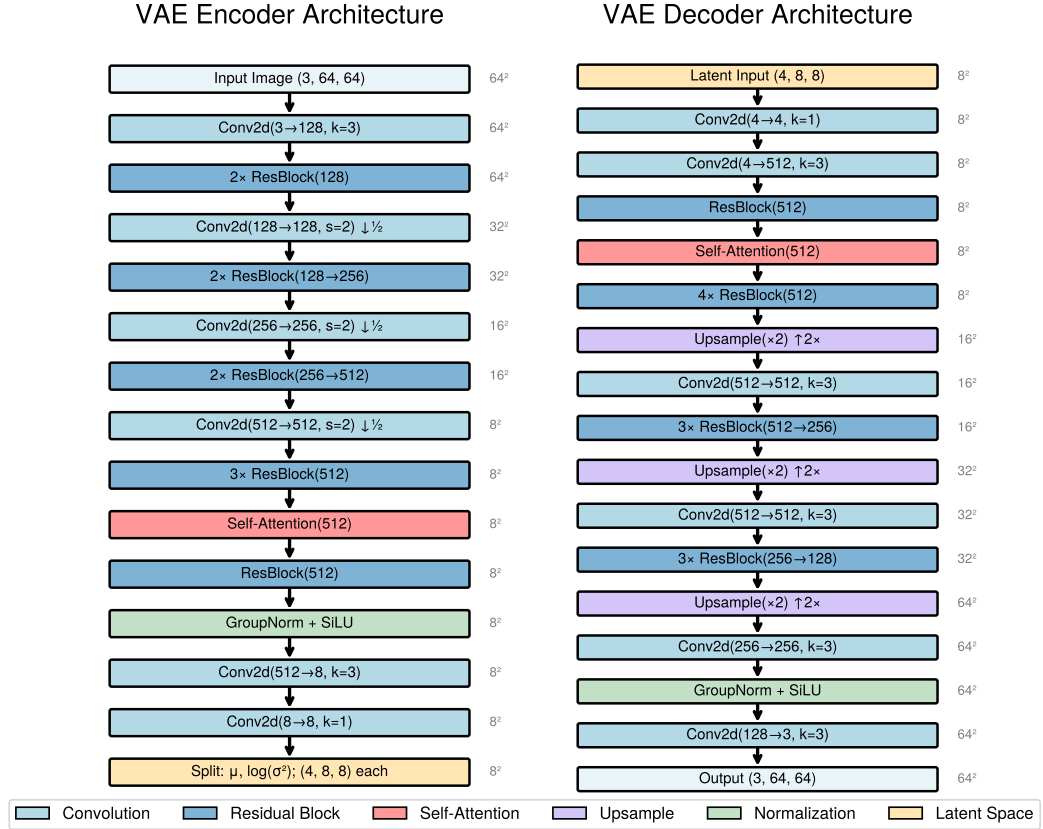


Figure 6.2: The architecture of the variational autoencoder, the model can be split into an encoder, which translates the input image into a latent space. It returns a tensor of the image in the latent space, as well as a tensor of mean values μ , and a tensor of the logarithm of variance $\log \sigma^2$. The decoder translates the image in the latent space into the image space.

- VAE_ResidualBlock: 512 channels
- VAE_ResidualBlock: 512 channels
- VAE_ResidualBlock: 512 channels
- VAE_AttentionBlock: 512 channels
- VAE_ResidualBlock: 512 channels
- GroupNorm: 32 groups, 512 channels
- SiLU Activation
- Conv2d: 512 $\rightarrow$ 8 channels, kernel 3, padding 1
- Conv2d: 8 $\rightarrow$ 8 channels, kernel 1, padding 0.

This output is then chunked into two separate tensors, one being the mean value μ tensor and the other one the logarithm of variance $\log \sigma^2$. These two tensors are used to calculate the final representation of the image in the latent space according to the *reparameterization trick* formula:

$$z = \alpha(\mu + \sigma \cdot \epsilon) \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}), \quad (6.1)$$

where z is the representation in the latent space, ϵ is the noise, and $\alpha = 0.18215$ is a constant helping with the stability of the training and is taken from Rombach's model [39].

The decoder recreates the image from the latent space using a series of operations similar to the ones used in the encoder. It starts from the latent space representation:

- Conv2d: $4 \rightarrow 4$ channels, kernel 1, padding 0
- Conv2d: $4 \rightarrow 512$ channels, kernel 3, padding 1
- VAE_ResidualBlock: 512 channels
- VAE_AttentionBlock: 512 channels
- VAE_ResidualBlock: 512 channels
- VAE_ResidualBlock: 512 channels
- VAE_ResidualBlock: 512 channels
- VAE_ResidualBlock: 512 channels
- Upsample: scale factor 2
- Conv2d: $512 \rightarrow 512$ channels, kernel 3, padding 1
- VAE_ResidualBlock: 512 channels
- VAE_ResidualBlock: 512 channels
- VAE_ResidualBlock: 512 channels
- Upsample: scale factor 2
- Conv2d: $512 \rightarrow 512$ channels, kernel 3, padding 1
- VAE_ResidualBlock: $512 \rightarrow 256$ channels
- VAE_ResidualBlock: 256 channels
- VAE_ResidualBlock: 256 channels
- Upsample: scale factor 2
- Conv2d: $256 \rightarrow 256$ channels, kernel 3, padding 1
- VAE_ResidualBlock: $256 \rightarrow 128$ channels
- VAE_ResidualBlock: 128 channels
- VAE_ResidualBlock: 128 channels
- GroupNorm: 32 groups, 128 channels
- SiLU Activation
- Conv2d: $128 \rightarrow 3$ channels, kernel 3, padding 1.

The output of the decoder is then the reconstruction of the original image.

Now, the particular blocks of the VAE will be introduced, these are: 2D Convolution; Residual Block; Self Attention; Upsampling; Group Normalization.

6.2.3 Building Blocks of the Variational Autoencoder

The Variational Autoencoder (VAE) architecture is composed of several fundamental neural network operations. This section details the block used for this model.

2D Convolution

The 2D Convolution operates by sliding a kernel (filter) over the input spatial dimensions to detect local semantic patterns such as edges, textures, and shapes. The model utilizes 3×3 kernels for feature extraction and 1×1 kernels for channel projection. To maintain spatial resolution during feature extraction, a padding of 1 is applied; for downsampling, a stride of 2 is used to halve the spatial dimensions. For a visual explanation please refer to Figure 4.4.

Group Normalization

Given an input feature map $x \in \mathbb{R}^{N \times C \times H \times W}$, GN divides the C channels into G groups (where $G = 32$). For a specific group g and a sample n , the mean $\mu_{n,g}$ and standard deviation $\sigma_{n,g}$ are computed as:

$$\mu_{n,g} = \frac{1}{m} \sum_{c \in S_g} \sum_{h=1}^H \sum_{w=1}^W x_{n,c,h,w} \quad (6.2)$$

$$\sigma_{n,g} = \sqrt{\frac{1}{m} \sum_{c \in S_g} \sum_{h=1}^H \sum_{w=1}^W (x_{n,c,h,w} - \mu_{n,g})^2 + \epsilon} \quad (6.3)$$

where S_g is the set of indices for channels in the g -th group, $m = (C/G) \cdot H \cdot W$, and ϵ is a stability constant. The normalized output is then scaled and shifted by learnable parameters γ and β :

$$y_{n,c,h,w} = \gamma_c \left(\frac{x_{n,c,h,w} - \mu_{n,g}}{\sigma_{n,g}} \right) + \beta_c. \quad (6.4)$$

Residual Block

To facilitate the training of deep networks and prevent vanishing gradients, the model employs Residual Blocks. These blocks learn the residual mapping rather than the underlying mapping itself. The output y of a residual block is defined as:

$$y = \mathcal{F}(x, \{W_i\}) + x, \quad (6.5)$$

(see section 4.4.2) where x is the input and $\mathcal{F}$ represents the residual function consisting of the sequence: GroupNorm $\rightarrow$ SiLU $\rightarrow$ Conv2d $\rightarrow$ GroupNorm $\rightarrow$ SiLU $\rightarrow$ Conv2d. If the dimensions of x and $\mathcal{F}(x)$ do not match, a linear projection (1×1 convolution) is applied to x to align the dimensions. The pseudo code describing the residual blocks used in this work can be found in Algorithm 3.

Algorithm 3 VAE Residual Block Forward Pass

```

1: Input: Tensor  $x$  of shape  $(B, C_{in}, H, W)$ 
2: Parameters:  $C_{in}$  (input channels),  $C_{out}$  (output channels), Groups = 32
3: Output: Tensor  $y$  of shape  $(B, C_{out}, H, W)$ 

4:  $residue \leftarrow x$  ▷ Cache input for skip connection
   First Convolution Block:
5:  $x \leftarrow \text{GroupNorm}(x, \text{groups} = 32)$ 
6:  $x \leftarrow \text{SiLU}(x)$ 
7:  $x \leftarrow \text{Conv2D}(x, \text{kernel} = 3, \text{padding} = 1)$  ▷ Channels:  $C_{in} \rightarrow C_{out}$ 
   Second Convolution Block:
8:  $x \leftarrow \text{GroupNorm}(x, \text{groups} = 32)$ 
9:  $x \leftarrow \text{SiLU}(x)$ 
10:  $x \leftarrow \text{Conv2D}(x, \text{kernel} = 3, \text{padding} = 1)$  ▷ Channels:  $C_{out} \rightarrow C_{out}$ 
   Residual Connection:
11: if  $C_{in} == C_{out}$  then
12:    $skip \leftarrow residue$  ▷ Identity mapping
13: else
14:    $skip \leftarrow \text{Conv2D}(residue, \text{kernel} = 1)$  ▷ Project channels  $C_{in} \rightarrow C_{out}$ 
15: end if
   Combine:
16:  $y \leftarrow x + skip$ 
17: return  $y$ 

```

Self-Attention Mechanism

A detailed introduction into the attention mechanism can be found in Section 4.4.1.

This is how attention is used in the VAE in this work. While convolutions process local neighborhoods, the Self-Attention mechanism allows the model to capture long-range dependencies across the entire image. The input feature map is reshaped into a sequence $x \in \mathbb{R}^{N \times L \times C}$, where $L = H \times W$. The input is projected into Queries (Q), Keys (K), and Values (V):

$$Q = xW_Q, \quad K = xW_K, \quad V = xW_V. \quad (6.6)$$

The attention scores are computed using the scaled dot-product formulation:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_{\text{head}}}} \right) V. \quad (6.7)$$

This operation allows every pixel to attend to every other pixel, ensuring a global coherence in the latent representation.

Upsampling

In the decoder, spatial resolution is recovered using Upsampling blocks. The model utilizes Nearest Neighbor interpolation with a scale factor of 2, followed immediately by a convolu-

tion layer. This two-step approach is preferred over Transposed Convolutions as it effectively mitigates checkerboard artifacts in the generated images.

6.2.4 VAE Training

The Variational Autoencoder was trained to learn the compressed, continuous latent representation of the input (image) data. The training process involved simultaneously optimizing the Encoder to map inputs into a **probabilistic** latent space and the Decoder to reconstruct the original inputs from sampled vectors from the latent space.

Dataset and Preprocessing

In addition to the preprocessing steps mentioned in Section 6.1, the data was also preprocessed using a standard normalization strategy to remap pixel values from $[0, 1]$ to the range $[-1, 1]$. A simple formula was used to achieve that:

$$x_{norm} = \frac{x - 0.5}{0.5}. \quad (6.8)$$

The dataset train and test splits were obtained using the scheme introduced in Section 6.1.

Objective Function

The training objective is to maximize the Evidence Lower Bound (ELBO). It is implemented as minimization of a composite loss function consisting of two terms: the Reconstruction Loss and the Kullback-Leibler (KL) Divergence (see section 4.2.1).

Reconstruction Loss: To ensure the generated images faithfully resemble the inputs, the model minimizes the divergence between the original image x and the reconstructed image $\hat{x}$. In this work this is achieved using the Mean Squared Error (MSE) between these two:

$$L_{rec} = \frac{1}{N} \sum_{i=1}^N ||x_i - \hat{x}_i||^2, \quad (6.9)$$

where N represents the number of pixels.

Regularization (KL-Divergence): To ensure the latent space is smooth and follows a standard normal distribution $\mathcal{N}(0, I)$, the model needs to minimize the KL-Divergence between the learned distribution and the Gaussian. In this implementation the closed-form formulation is:

$$L_{KL} = -0.5 \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2). \quad (6.10)$$

Total Loss: The final loss function is a weighted sum of these two components:

$$L_{total} = L_{rec} + \lambda_{KL} \cdot \langle L_{KL} \rangle. \quad (6.11)$$

Where λ_{KL} is a hyperparameter (set to $10E - 6$) that balances the trade-off between the reconstruction fidelity and latent space regularization. The chosen λ_{KL} ensures that the reconstruction quality is prioritized while maintaining a well structured gaussian latent manifold.

Optimization and Implementation Details

The model parameters θ (Encoder) and ϕ (Decoder) were optimized together using the **AdamW** optimizer (See section B.2), a variant of Adam optimizer that decouples weight decay from the gradient updates – improving generalization.

The hyperparameters used for training are:

- **Learning Rate:** Fixed at $10E - 4$.
- **Batch Size:** 1024.
- **Mixed Precision Training:** In order to reduce the memory usage, Automatic Mixed Precision was employed. Gradients were scaled using `GradScaler` to prevent underflow artifacts during backpropagation when using half-precision.
- **Hardware Acceleration:** The training pipeline utilized `DataParallel` to distribute load across multiple GPUs during training.
- **Early Stopping:** An early stopping scheme was utilized with `patience = 10`, and `min_delta = 0.0001`.

6.2.5 Composition Encoding

For the composition encoding various schemes were tested, from simple fractional encoding using different tokenizers, to more sophisticated approaches inspired by the Contrastive Language-Image Pre-training (CLIP) model introduced by the Open-AI team for their diffusion model [41]. We argue that the encoding part of the model’s architecture is the crucial part, allowing the model to generalize for the unseen compositions when predicting any material properties. An encoder that can preserve properties of the material to feed into the generational model is of the utmost importance for a well working generator able to predict properties better or on par with an expert.

This is why a lot of attention in this work was given to finding an appropriate encoder. Here we introduce the tested encoders.

One-Hot Embedding

The simplest encoder utilized was the One-Hot encoder, where the elements are encoded using a vector of a domain-size, each entry in the vector represents a single element, compositions are encoded using a simple scaled addition of vectors corresponding to the elements. Each elemental vector is a sparse vector with only single entry. For instance $v_H = [1, 0, 0, 0, 0, \dots, 0]$ and $v_{He} = [0, 1, 0, 0, 0, \dots, 0]$ and so on. A complex composition of elements is achieved by adding together the separate vectors multiplied by the fraction at which

the given cation is present in the oxide. lets say we wanted to encode an oxide composed of 50% iron, 25% copper and 25% bismuth, the final composition vector would be:

$$v_{composition} = 0.5 \cdot v_{Fe} + 0.25 \cdot (v_{Cu} + v_{Bi}) = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 0.5 \\ 0 \\ 0 \\ 0.25 \\ 0 \\ \vdots \\ 0 \\ 0.25 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad (6.12)$$

with the non-zero entries at the entries corresponding to the atomic numbers of the elements present in the composition (in this case: 26, 29, 83). This vector can be then used to condition the stable diffusion model.

MEGNet Embedding

The embedding framework discussed here is derived from the MatErials Graph Network (MEGNet), a generalized graph-based deep learning architecture for molecules and crystals [54]. The MEGNet model’s effectiveness stems from the way embeddings are construed as learnable parameters.

The elemental embedding, which maps integer atomic numbers (Z) to a continuous vector space, typically of dimension 16. Instead of relying on hand-crafted descriptors, these embeddings are learned directly from large datasets, such as a dataset of formation energies of $\sim 60,000$ crystals. This data-driven approach allows the model to autonomously encode chemical intuition; analysis done by the researchers shows that the learned vectors reproduce the periodic table’s trends, with elements in the same group (e.g., alkali metals, halogens) exhibiting high similarity. The embeddings even capture subtle chemical behaviors, such as the clustering of Europium (Eu) and Ytterbium (Yb) with alkaline earth metals rather than lanthanides, accurately reflecting their valence properties.

The composition vectors are obtained similarly to the strategy introduced for OH-Embedding (see eq. (6.12)).

Elem Embedding

This approach is not dissimilar to the MEGNet, in that the embeddings are learned, rather than hand-designed.

To represent chemical compositions without relying on pre-selected physical descriptors, ElemNet [55] utilizes a raw elemental encoding strategy. In this framework, the input is

defined as a sparse (like OH encoding) vector of stoichiometric fractions, which serves as a selector for the network's internal representation. Functionally, the weights of the first fully connected layer act as a learnable lookup table, and these are used in our models; the matrix multiplication of the sparse input vector with these weights effectively retrieves and linearly combines dense embedding vectors for each constituent element. This approach allows the model to construct a distributed representation of atomic species, optimizing the elemental embeddings specifically for the prediction task. The network, within the representations, autonomously captures latent chemical similarities and trends – such as group placement and valency – solely from the training data.

We have used the model only for element encoding, and combined different embeddings at appropriate fractions in a similar fashion as in the previously introduced encoding strategies.

Word2Vec Embedding

This is a completely different approach compared to the ones introduced before in that it does not utilize the chemical information about the elements in the training of the encoder, but the embeddings are trained in an unsupervised fashion based on large text corpora.

While previously mentioned embeddings are inferred from chemical or physical dependencies, an alternative strategy adapts the Word2Vec algorithm from Natural Language Processing (NLP) to the domain of materials science. In this framework, chemical formulas and material descriptions are treated as 'sentences', and the embeddings stem only from textual data. The basis for the use of this approach in our application is that chemical elements co-occur with other elements, properties, and phase descriptions – encodes in a systematic way, making it possible to infer some information about the.

One of the implementations of this approach into materials science, often referred to as Mat2Vec [56] trains a skip-gram neural network on millions of scientific abstracts (e.g., 3.3 million abstracts from 1,000 journals). Unlike supervised models that require labeled property data, this approach is unsupervised. The model iterates through the text with a sliding context window, learning to predict surrounding context words given a target element or chemical formula. The hidden layer weights of this network serve as the dense vector representations (embeddings) for the elements.

Word2Vec-derived representations autonomously capture sophisticated chemical intuition without explicit design. As the model is an unsupervised one. When projected into lower-dimensional spaces (e.g., via t-SNE or PCA), these embeddings reproduce the periodic table's structure. Elements with similar valence electron configurations, such as alkali metals (Li, Na, K) or halogens (F, Cl, Br), or elements of similar atomic weight/number form tight clusters in the vector space. This clustering can be seen on Figure 6.3.

CCIP Embedding

The primary limitation of the weighted sum approach for material representation is the loss of uniqueness. A weighted sum relies only on linear combinations of elemental features. It is mathematically possible for distinct material compositions to result in identical or nearly identical vector representations. A practical example of this ambiguity is illustrated in Figure 6.3, where the representation for a 1:1 Cu-Fe alloy lies very close to the independent representations of nickel and molybdenum. Such an ambiguity in the encoding space an

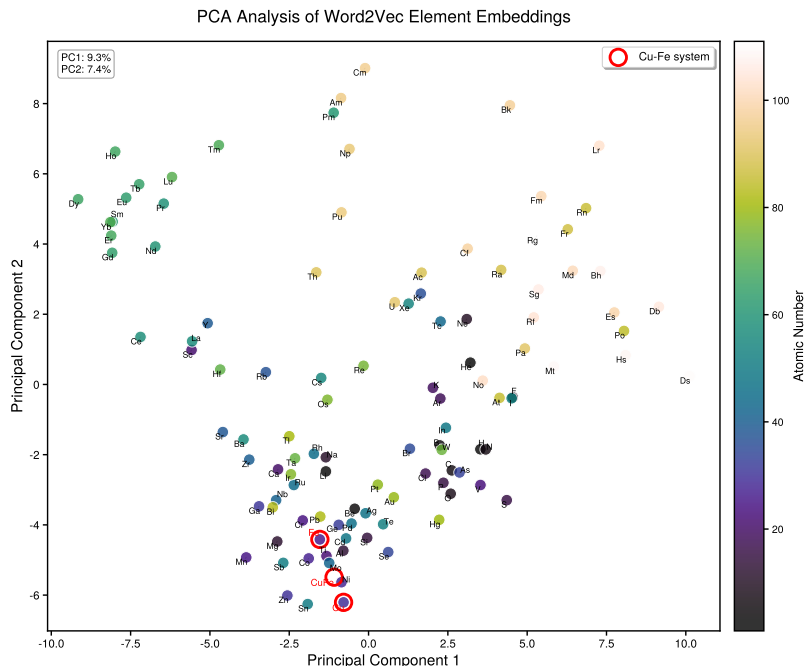


Figure 6.3: PCA Analysis of Word2Vec Element Embeddings. The projection of high-dimensional element vectors onto the first two principal components reveals distinct clustering based on chemical properties. The color scale represents atomic weight. Note the tight clustering of transition metals in the Cu-Fe system (red circles), indicating their high similarity.

leads ‘hallucinations’ in a model trained on these embeddings, effectively leading to unreliable predictions.

To mitigate this issue and construct a more robust representation space, we developed a novel conditioning framework inspired by the Contrastive Language-Image Pre-training (CLIP) model introduced by Radford et al. [41]. The original CLIP architecture is a multimodal neural network designed to learn a shared latent space for visual and textual data using a dual-encoder design:

- **Vision Encoder:** A Vision Transformer that processes raw images into feature vectors.
- **Text Encoder:** A Transformer that converts text prompts into aligned embedding vectors.

CLIP is trained via a contrastive loss objective that maximizes the cosine similarity between matched image-text pairs while minimizing the similarity for mismatched pairs within a batch. This process effectively aligns the two modalities in a joint latent space without the need for task-specific labels.

Adapting this paradigm to materials science, we introduce the Contrastive Composition-Image Pre-training (CCIP) model. In this framework, chemical compositions effectively replace text sentences as the semantic modality in the original model. A key modification in our architecture is the removal of positional encoding; unlike natural language, where word order dictates meaning, chemical formulas are permutation-invariant (e.g., FeCu is equivalent to CuFe).

The CCIP Architecture The CCIP model tokenizes both images and compositions using separate encoders. Subsequently their representations are aligned via a symmetric InfoNCE loss. The core innovation in the CCIP in comparison with CLIP is in the composition encoder, which employs a novel ‘Double Sum’ embedding strategy. This strategy aims to retain the stoichiometric ratios, and enable learnable embeddings:

1. **Identity Embedding:** We utilize learnable vectors for each elemental species, analogous to word embeddings in NLP. These vectors aim to capture fundamental chemical properties.
2. **Fraction Projection:** To retain precise stoichiometric information, we modify the discretization common in NLP tokenizers. The elemental embeddings are modified by addition of a stoichiometric factor to each element of the vector.
3. **Transformer Contextualization:** This model utilizes Multi-Head Self-Attention. This mechanism allows every element token to ‘attend’ to every other element in the composition. Through stacked Transformer layers, the model learns to dynamically weigh specific element-element interactions, thereby overcoming the linearity and locality limitations of simple weighted sums.

Additionally, in our model there is no positional encoding, as compositions are inherently position invariant.

The final material representation is computed via masked global pooling followed by a L2-normalization.

Architecture Specification and Training Methodology The dual-encoder system is composed of two specialized networks that project inputs into a shared d -dimensional embedding space ($d = 512$).

- **Composition Encoder ($\mathcal{E}_{comp}$):** This module processes chemical formulas parameterized as sequences of length $L = 20$. It utilizes an embedding dimension of 128 and consists of 6 Transformer layers with 8 attention heads each. The output is a normalized vector $v_{comp} \in \mathbb{R}^{512}$.
- **Image Encoder ($\mathcal{E}_{img}$):** Visual inputs are processed by a Convolutional Neural Network (CNN) comprising four sequential blocks. Each block follows a standard pattern: Conv2d $\rightarrow$ BatchNorm $\rightarrow$ ReLU. The channel depth is increased progressively ($3 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$) while spatial dimensions are reduced via max pooling or strided convolutions. In the end the feature map gets flattened by a final global average pooling layer before getting projected to the shared dimension, yielding $v_{img} \in \mathbb{R}^{512}$.

The visualization of the model pipeline can be found in Figure 6.4.

Loss Function Training is conducted using a symmetric contrastive loss. For a batch of B composition-image pairs $\{(c_i, I_i)\}_{i=1}^B$, the model computes an $B \times B$ similarity matrix where the entry (i, j) represents the cosine similarity scaled by a learnable temperature parameter τ (initialized at 0.07). The objective function L minimizes the cross-entropy loss along both axes of this matrix:

$$L = \frac{1}{2} (L_{c \rightarrow I} + L_{I \rightarrow c}) . \quad (6.13)$$

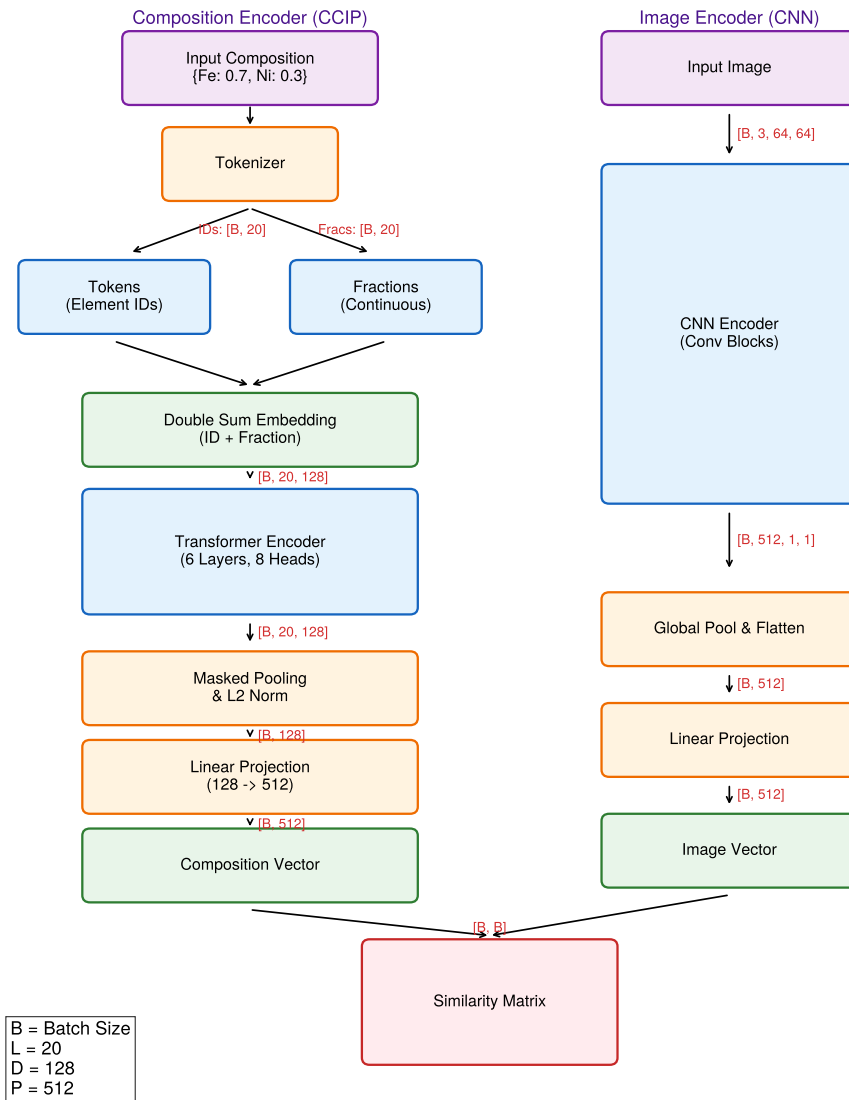


Figure 6.4: Schematic of the Contrastive Composition-Image Pre-training (CCIP) Architecture. The framework employs a dual-encoder design to align chemical and visual modalities in a shared latent space. **Left:** The Composition Encoder processes stoichiometric data using a specialized ‘Double Sum’ embedding strategy. Unlike standard tokenizers, this module explicitly combines learnable elemental identities with continuous atomic fractions before processing them through a 6-layer Transformer stack. **Right:** The Image Encoder processes material signatures (in this work, photographs) using a Convolutional Neural Network (CNN). Both branches project inputs into a normalized 512-dimensional vector space (v_{comp}, v_{img}), where a symmetric contrastive loss (InfoNCE) is used to maximize the cosine similarity between matched pairs. Tensor dimensions are annotated at each stage, where B denotes the batch size, $L = 20$ represents the fixed sequence length (padded), $D = 128$ is the internal embedding dimension, and $P = 512$ is the final projection dimension.

The two losses are the cross entropy losses of classifying the right image to the right composition and the right composition to the right image. This symmetric objective forces the diagonal elements (matched pairs) to have the highest similarity while pushing off-diagonal elements (mismatched pairs) apart.

Optimization and Distributed Training The model is trained using the Distributed Data Parallel (DDP) strategy to leverage multi-GPU infrastructure. Optimization is performed using AdamW (see algorithm 5) with a base learning rate of $10E-4$ and weight decay of 0.01. To ensure training stability and efficiency, we implement:

- **Cosine Annealing Scheduler:** The learning rate follows a cosine decay schedule over 100 epochs, allowing for smooth convergence.
- **Automatic Mixed Precision (AMP):** Training utilizes FP16 mixed precision to reduce memory overhead and accelerate throughput.
- **Gradient Clipping:** Gradients are clipped at a norm of 1.0 to prevent diverging updates.

6.2.6 Diffusion Architecture and Dynamics

Gaussian Noise Scheduler

The forward diffusion process is governed by a variance-preserving Markov chain that gradually corrupts the latent structure of the image data. We implement a linear noise scheduler defined by a fixed set of variance parameters $\{\beta_t\}_{t=1}^T$. For a total of $T = 1000$ diffusion timesteps, the schedule is parameterized linearly starting from $\beta_1 = 8.5 \times 10^{-4}$ to $\beta_T = 1.2 \times 10^{-2}$.

At any arbitrary timestep t , the noisy latent z_t can be sampled directly from the original latent z_0 via the closed-form property:

$$q(z_t|z_0) = \mathcal{N}(z_t; \sqrt{\bar{\alpha}_t}z_0, (1 - \bar{\alpha}_t)\mathbf{I}), \quad (6.14)$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. This schedule ensures that by $t = T$, the signal-to-noise ratio is sufficiently low that z_T approximates an isotropic Gaussian distribution $\mathcal{N}(0, \mathbf{I})$, providing the starting point for the generative reverse process. the derivation of this formula can be found in Equations (4.45) to (4.50).

Sinusoidal Time Embeddings

To allow the neural network to share weights across all 1000 noise levels, the scalar timestep t is projected into a high-dimensional vector space. We utilize sinusoidal position embeddings (see section 4.5.3), identical to those used in Transformer architectures, to encode temporal information.

The embedding process consists of two stages:

1. **Sinusoidal Encoding:** For a base embedding dimension $d_{emb} = 320$, the timestep t is mapped to a vector $t_{emb} \in \mathbb{R}^{320}$ using geometric frequencies $\omega_i = 1/10000^{2i/d_{emb}}$.

2. **MLP Projection:** This vector passes through a two-layer Multi-Layer Perceptron (MLP) with SiLU activation. The first linear layer expands the dimension to $4 \times d_{emb}$ (1280) and the second maintains this dimension. The final time embedding vector $t_{proj} \in \mathbb{R}^{1280}$ is injected into every residual block of the U-Net to modulate feature maps.

The Conditional U-Net Backbone

The core denoising engine is a customized U-Net architecture that integrates spatial latents, temporal signals, and chemical context. It was introduced in detail in Section 4.3.2.

Dimensionality and Downsampling The network processes latents of shape $(B, 4, H/8, W/8)$. It begins with an initial convolution expanding the 4 input channels to 320. The encoder path follows a channel multiplier sequence of $[1, 2, 4, 4]$, resulting in the following feature hierarchy:

- **Level 0:** 320 channels (Original resolution),
- **Level 1:** 640 channels (Downsample $\times 2$),
- **Level 2:** 1280 channels (Downsample $\times 4$),
- **Level 3:** 1280 channels (Downsample $\times 8$),

Downsampling is performed via strided convolutions ($k = 3, s = 2, p = 1$), where k is the kernel size, s – stride, p – padding.

Residual and Attention Blocks Each resolution level contains a sequence of `SwitchSequential` blocks combining:

- **Residual Blocks:** These blocks combine the spatial features with the time embedding t_{proj} . We use Group Normalization (32 groups) and SiLU activations throughout.
- **Attention Blocks:** To condition generation on compositions, we employ dual-attention mechanisms. Each block contains (1) Self-Attention to capture global spatial dependencies and (2) Cross-Attention to integrate the chemical context vectors c .

Attention is applied at resolutions corresponding to 320, 640, and 1280 channels, utilizing 8 attention heads across all levels. The bottleneck section consists of two residual blocks sandwiching a global attention block operating at 1280 channels.

U-Net Decoder and Output Decoder symmetrically up-samples features using nearest-neighbor interpolation followed by convolution to recover the spatial dimensions. Crucially, the generative process remains conditioned throughout the reconstruction phase: each decoder level mirrors the encoder’s architecture, employing Cross-Attention blocks that continue to query the chemical context c at every up-sampling step. Skip connections concatenate these chemically-conditioned encoder features with the decoder features, effectively doubling the input channels for subsequent residual blocks. The final output layer normalizes the feature map and projects it back to the original 4 latent channels, predicting the noise residual ϵ_θ .

6.2.7 Latent Diffusion Training Protocol

With the perceptual compression (VAE) and semantic encoder (CCIP) pre-trained and frozen, the final training stage focuses exclusively on the conditional U-Net backbone. This phase effectively learns the generative distribution $p(z|c)$, mapping chemical semantics to the structural latent space.

Optimization Objective The model is trained to minimize the reweighted variational lower bound, simplified to the mean squared error between the added noise ϵ and the predicted noise ϵ_θ :

$$L_{simple} = \mathbb{E}_{x,c,\epsilon \sim \mathcal{N}(0,1),t} [\|\epsilon - \epsilon_\theta(z_t, t, \tau_\theta(c))\|_2^2], \quad (6.15)$$

where z_t is the noisy latent at timestep t and $\tau_\theta(c)$ is the fixed chemical embedding.

Training Setup and Stability Training is conducted using a Distributed Data Parallel (DDP) framework. We employ the AdamW optimizer with a learning rate of $10E-4$ and weight decay of $10E-2$, coupled with a Cosine Annealing scheduler to smoothly decay the learning rate over 100 epochs. To ensure numerical stability within the diffusion process, we apply a **Latent Scale Factor** to the VAE outputs $z = \mathcal{E}(x)$. Since the pre-trained VAE latent space may have arbitrary variance, this scalar normalization ensures the input latents approximate a standard normal distribution, aligning with the diffusion prior’s assumptions.

Computational Efficiency To maximize throughput, we leverage Automatic Mixed Precision (AMP). The forward pass (U-Net computation) and gradient calculation are performed in FP16 precision, while master weights are maintained in FP32. This reduces memory overhead, allowing for larger effective batch sizes which are critical for accurate noise estimation in generative modeling as well as speeding up the training.

6.3 Spectral Prediction Network

For mapping the spectral prediction of the compositions we used the same encoding scheme as in the Stable Diffusion Model, specifically Word2Vec and feed it through a shallow MLP.

6.3.1 SP Encoding Scheme

Given atomic fractions $\mathbf{c} = [c_1, c_2, \dots, c_n]$ and pre-trained embeddings $\mathbf{V} \in \mathbb{R}^{n_c \times d_c}$ (where d_c is the embedding dimension), we compute the composition encoding as (similarly to the LDM approach):

$$\mathbf{v}_{comp} = \mathbf{c}^T \mathbf{V} = \sum_{i=1}^{n_c} c_i \mathbf{v}_i, \quad (6.16)$$

where $\mathbf{v}_i$ is the embedding vector for element i . This weighted sum produces a d_c dimensional vector that captures the chemical context of the composition. For detailed description see Sections 4.4.3 and 6.2.5.

6.3.2 SP Architecture

The predictor is implemented as a lightweight Feed-Forward Neural Network consisting of fully connected layers. The architecture expands the compositional embedding (v_{comp}) into the high-dimensional spectral output space:

1. **Composition Encoder:** Word2Vec-based encoding layer,
2. **Hidden Layers:** Sequence of fully-connected layers with activation functions,
3. **Regularization:** Batch normalization and dropout,
4. **Output Layer:** Linear projection to spectral dimension.

Non-linearity is introduced via **ReLU** activations after the first two linear transformations. To prevent overfitting on the dense compositional inputs, **Dropout** is applied after each activation function.

6.3.3 SP Training Protocol

The model is trained to minimize the Mean Squared Error (MSE) between the predicted spectrum $\hat{y}$ and the ground truth spectrum y :

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2. \quad (6.17)$$

Optimization is performed using **AdamW**. Learning rate is scheduled via reducing it on plateau:

$$\eta_{\text{new}} = \eta_{\text{old}} \times \gamma \quad \text{if no improvement for } k \text{ epochs.} \quad (6.18)$$

Training gets terminated if validation loss does not improve for p consecutive epochs, preventing overfitting.

Hyperparameter Optimization

To ensure the SpectralPredictor model generalizes well and avoids suboptimal convergence, a systematic hyperparameter search was conducted. **Optuna** [57] was employed, an automated hyperparameter optimization framework that utilizes the Tree-structured Parzen Estimator (TPE) algorithm to efficiently sample the search space. The defined search space can be seen in Table 6.3.

Table 6.3: Hyperparameter search space for Optuna optimization.

Hyperparameter	Type	Range/Options
Batch size	Categorical	{16, 32, 64, 128}
Learning rate	Log-uniform	$[10^{-5}, 10^{-2}]$
Weight decay	Log-uniform	$[10^{-7}, 10^{-3}]$
Dropout rate	Uniform	$[0.1, 0.5]$
Number of layers	Integer	$[2, 5]$
Hidden dimensions	Categorical	{128, 256, 512, 768, 1024}
Batch normalization	Categorical	{True, False}
Freeze embeddings	Categorical	{True, False}

Optimization Objective and Sampler

The optimization process treats the model training as a black-box function $f(\theta) \rightarrow \mathbb{R}$, where θ represents the set of hyperparameters. The objective was to minimize the validation loss on the hold-out set:

$$\theta^* = \underset{\theta \in \Theta}{\operatorname{argmin}} L_{\text{val}}(f(\theta)). \quad (6.19)$$

Optuna’s ‘define-by-run’ API was utilized to construct dynamic search spaces, particularly for the network’s depth and width. The used TPE sampler models the probability density of good and bad trials, focusing computational resources on promising regions of the hyperparameter space rather than performing a blind random search or an exhaustive grid search. This greatly improves the speed at which the optimization can be performed.

Results and Discussion

7.1 Variational Autoencoder (VAE) Evaluation

The efficacy of the Latent Diffusion Model is fundamentally limited by the quality of the perceptual compression stage. Therefore, we first evaluate the VAE’s performance in terms of reconstruction fidelity, latent space stability, and manifold continuity to ensure it provides a robust foundation for downstream generation.

7.1.1 Reconstruction Fidelity

The VAE was trained to compress input photos into a low-dimensional latent space ($z \in \mathbb{R}^{4 \times 8 \times 8}$) and reconstruct them via the decoder. In Figure 7.1 a side-by-side comparison of ground truth micrographs and their reconstructions are shown. The images are generated for both the seen and unseen composition spaces.

Most of the samples are reconstructed well . As shown in Figure 7.1, the reconstructions accurately capture the variation in precipitate shapes – from circular to irregular – and their associated transparency. Minor smoothing is observed with the essential semantic structures required for materials analysis preserved intact. Only minor details like, uneven drying or small spots are lost, making the output images smoother.

7.1.2 Latent Space Analysis

For the subsequent diffusion model to function correctly, the VAE’s latent space must approximate a standard normal distribution $\mathcal{N}(0, 1)$ to facilitate stable sampling (see section 4.2.1).

Distribution and Regularization

We analyzed the statistical properties of the encoded latent vectors. Figure 7.2 (Left) illustrates the aggregate distribution of latent values, which closely follows a Gaussian bell curve centered at zero.

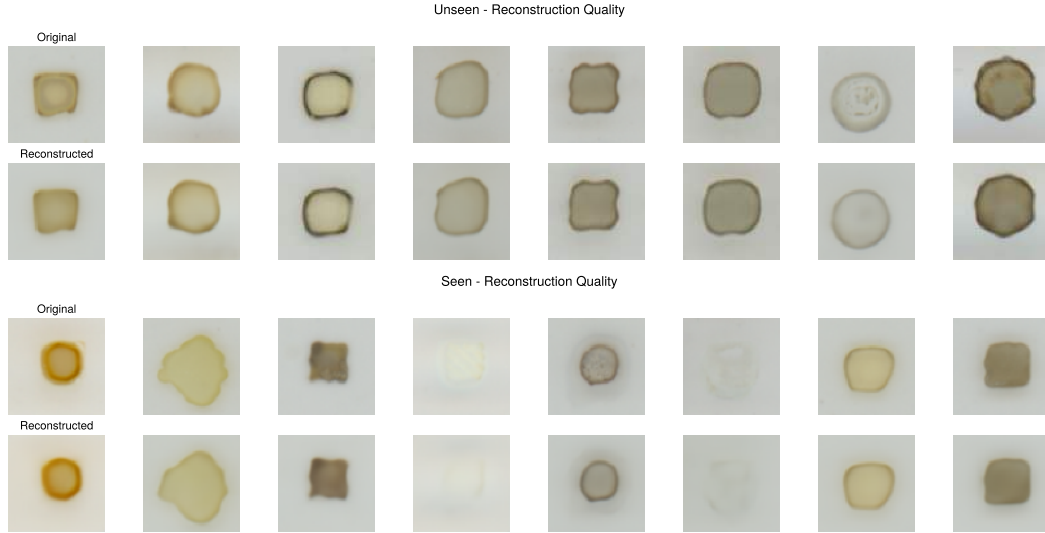


Figure 7.1: Qualitative assessment of VAE reconstruction on the unseen (**Top Panel**) and the seen (**Bottom Panel**) test set. **Top Row:** Original ground truth micrographs. **Bottom Row:** Reconstructed images. The model successfully preserves critical morphological features, such as the boundaries and distinct colors of the precipitates, despite the $8\times$ spatial compression; for both images of compositions previously seen as well as unseen by the model.

The channel-wise analysis (see fig. 7.2, Right) confirms that every channel is meaningful without any one collapsing or exploding, with mean values remaining within the range of $[-0.02, 0.12]$. This indicates that the Kullback-Leibler divergence term successfully constrained the model without posterior collapsing.

Generalization and Structure

To visualize the geometry of the latent space, we projected the high-dimensional latents into 2D space using Principal Component Analysis (PCA).

As shown in Figure 7.3, the Unseen Test set (green points) maps to the same region as the Seen set (blue points). The absence of distinct clusters or gaps suggests a continuous, meaningful latent space rather than a fragmented lookup table, which is crucial for generative interpolation. The explained variance for the first and second principal components is fairly low at 14%, this is a good indicator for a VAE model, as it points to a high information distribution across the channels in the latent space. If the explained variance was very high (e.g., $PC1 = 80\%$), it would suggest ‘Posterior Collapse,’ where the encoder ignores most latent dimensions and encodes all information into one or two.

7.1.3 Latent Continuity

To further verify the smoothness of the learned representation, we performed linear interpolation between two random samples in the latent space.

Figure 7.4 demonstrates the decoded outputs as we change the latent vector $z = (1 - \alpha)z_1 + \alpha z_2$ from the latent corresponding to one image, to the other. The smooth transition – morphing gradually from a darker, irregular in shape precipitate to a lighter, more rectangular

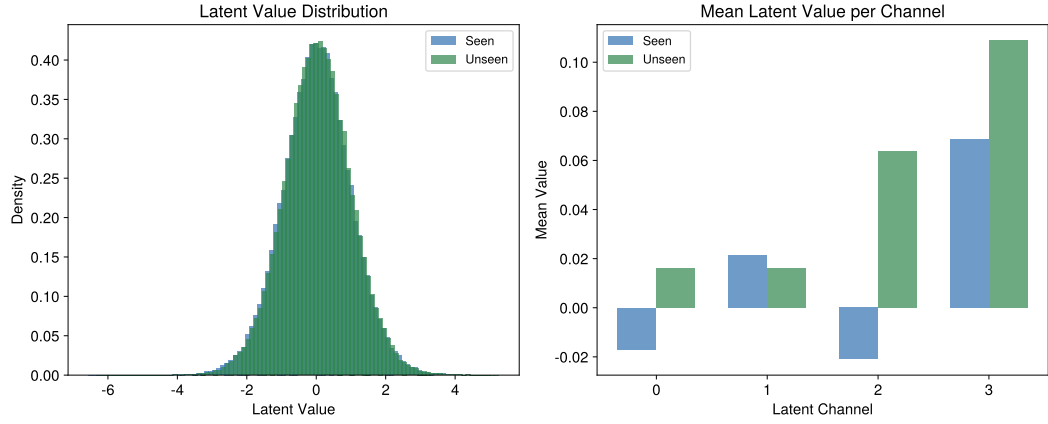


Figure 7.2: Statistical analysis of the learned latent space. **Left:** The aggregate density of latent values for Seen (blue) and Unseen (green). Test sets closely approximate standard normal distribution, pointing to a good regularization. **Right:** The mean latent value per channel, demonstrating that all channels are centered near zero (the mean values fall within $[-0.02; 0.12]$), confirming effective learning.

form without passing through unrealistic intermediate states – confirms that the VAE has learned a dense and well structured latent representation.

7.1.4 Quantitative Metrics and Generalization

VAE was further evaluated using standard metrics on two distinct subsets: the composition spaced seen in the training and the held-out during training, Unseen compositions, for both the number of samples on which the tests were performed was $N = 1000$. Table 7.1 summarizes the pixel-level and perceptual metrics.

Table 7.1: Comparison of VAE reconstruction metrics between test sets containing the seen and unseen material compositions. The equivalent performance across both sets points to a robust generalization.

Metric	Seen	Unseen
Reconstruction		
MSE ($\downarrow$)	$9.70 \times 10^{-5} \pm 1.08 \times 10^{-5}$	$1.03 \times 10^{-4} \pm 0.09 \times 10^{-4}$
MAE ($\downarrow$)	0.0064 ± 0.0002	0.0064 ± 0.0002
PSNR (dB, $\uparrow$)	40.99 ± 0.33	40.84 ± 0.22
SSIM ($\uparrow$)	0.9722 ± 0.0028	0.9738 ± 0.0021
Latent Statistics		
KL Divergence	226.89 ± 69.20	233.08 ± 57.33
Latent Mean	0.0145	0.0510
Latent Std	0.9956	0.9779

As illustrated in Table 7.1, the model exhibits very good stability in both sets. The Peak Signal-to-Noise Ratio (PSNR) for unseen compositions (40.84 dB) is nearly identical to that of seen compositions (40.99 dB). The similarity between the two sets indicates that the encoder has learned generalizable features of material images rather than memorizing just the training examples. Similarly, the latent space statistics for the unseen test set (Mean: 0.051,

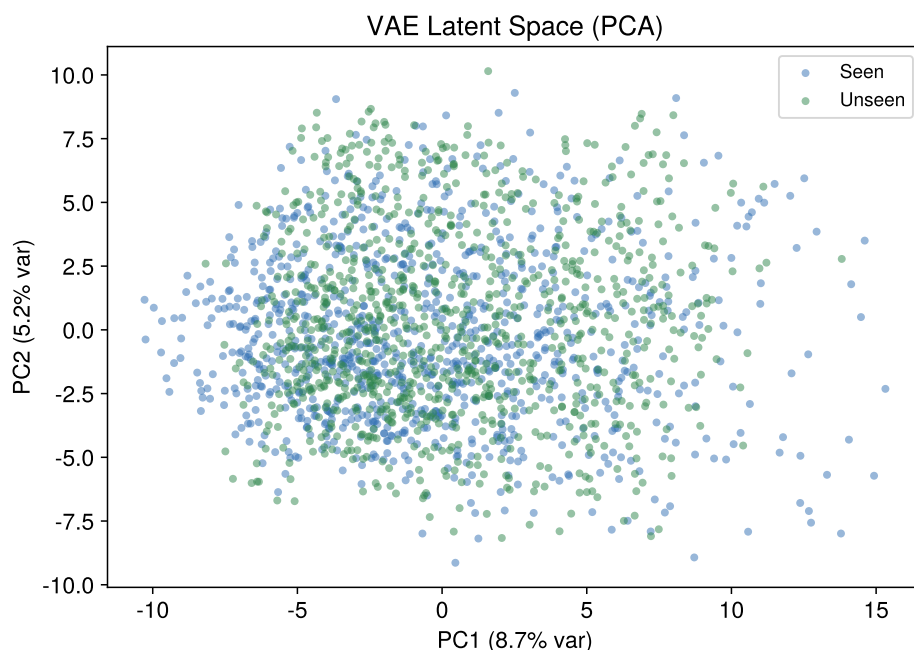


Figure 7.3: Principal Component Analysis (PCA) projection of the latent space. The seen samples (blue) and unseen samples (green) exhibit a high degree of overlap. This indicates that the model generalizes well to unseen data and which covers the same semantic space. Worth to mention that the variance explained by PC1 and PC2 is quite low at 14% indicating a high information distribution across channels.

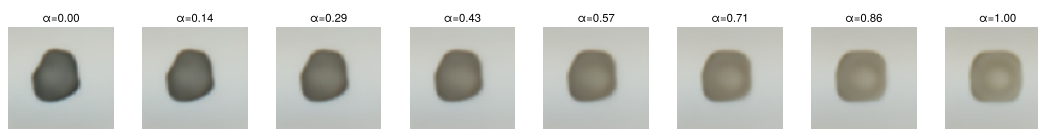


Figure 7.4: Latent space interpolation between two distinct material representations. As the interpolation factor α moves from 0.00 to 1.00, the decoded image transitions smoothly in shape and color, verifying that the latent space is continuous.

Std: 0.978) are very close to the prior Gaussian, indicating good regularization, and ensuring a well structured latent space for the subsequent diffusion model.

7.2 Evaluation of Compositional Embeddings (CCIP)

To validate the alignment between chemical and visual space, we evaluated the CCIP dual-encoder model on the held-out test split. The evaluation was performed on two distinct splits: the **Seen** set, which assesses the model's ability to work on correlations in previously seen composition spaces and the **Unseen** set, which tests the model's ability to generalize to novel composition systems.

7.2.1 Cross-Modal Retrieval Performance

We first quantified the model’s ability to retrieve the exact corresponding modality (image from composition and vice-versa) from a large database of candidates ($N = 10\,000$). Table 7.2 presents the standard retrieval metrics: Recall at Rank K ($R@1$, $R@5$, $R@10$) and Median Rank (MedR).

Table 7.2: Cross-modal retrieval performance for Seen vs. Unseen compositions. While exact formula retrieval is challenging for unseen data (low $R@1$), the Median Rank indicates the correct answer is consistently placed in the top percentile of candidates.

Task	Seen Compositions				Unseen Compositions			
	$R@1$	$R@5$	$R@10$	MedR	$R@1$	$R@5$	$R@10$	MedR
Comp $\rightarrow$ Img	3.88%	14.45%	24.17%	37	0.37%	1.82%	3.21%	317
Img $\rightarrow$ Comp	7.57%	28.14%	43.50%	14	0.87%	3.82%	7.48%	149

As expected, exact retrieval on unseen data is difficult ($R@1 \approx 0.9\%$). However, ‘failure’ in this metric often means the model retrieved a chemically similar material (e.g., $Fe_{70}Ni_{30}$) instead of the exact ground truth ($Fe_{72}Ni_{28}$). To quantify this behavior, we analyzed the elemental composition directly.

7.2.2 Chemical System Identification

Since the different samples can be quite close in the chemical space leading to a retrieval of samples which are close to the true label, however, not exactly, we introduced Element Recall Metrics. These metrics evaluate whether the retrieved composition contains the correct set of elements (e.g., identifying a sample as an “Al-Cu alloy”) regardless of the precise atomic fractions.

Table 7.3: Element-level identification metrics for Image $\rightarrow$ Composition retrieval. ‘Exact Element Match’ is the percentage of top-1 predictions containing exactly the correct set of elements. The high F1 scores on both the Seen and Unseen set confirm the model generalizes well and successfully identifies the chemical system (e.g., Fe-Ni vs. Al-Cu) even when it misses the exact stoichiometry.

Metric	Seen Set	Unseen Set
System Accuracy		
Exact Element Match @ 1	48.47%	60.39%
Exact Element Match @ 5	62.94%	63.30%
Element-Wise Scores		
Element Recall @ 1	86.96%	90.83%
Element Precision @ 1	84.16%	89.06%
Element F1 Score	84.25%	89.59%

Table 7.3 reveals a crucial finding: while the model struggles to predict the exact formula (low $R@1$ in Table 7.2), it is highly effective at identifying the chemical system of the composition. On the Unseen test set, the model correctly identifies the exact set of elements in the top-1 result 60.39% of the time, which is still not perfect, but much better compared to

the exact recall values. This confirms the hypothesis that the model correctly identifies the composition 'space' but not the specific composition. Furthermore, the Element F1 Score of 89.6% indicates that nearly all retrieved elements correct.

Precision is the fraction of the correct elements identified to all the elements on the composition, and Recall is the fraction of the correctly identified to all identified elements. F1 score is then calculated as a

$$F1 = 2 \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}. \quad (7.1)$$

These results suggest that the model map visual textures to elemental properties fairly well. The model can reliably distinguish between major compositional classes based on image. However, it fails at the exact classification.

7.3 Generative Modeling via Latent Diffusion

This section assesses the capability of the Conditioned Latent Diffusion Model to generate high-fidelity, conditioned solely on compositions, material images. Five embedding schemes were compared: MEGNet, Elem, Word2Vec, and CCIP; these were introduced in Section 6.2.5.

7.3.1 Qualitative Analysis

Four generative models were trained using different encoding strategies mentioned before. For MEGNet, Elemnet, and Word2Vec; before being fed into the U-Net, the conditioning vectors were run through one linear layer without an activation function, to reduce (or expand in case of MEGNet) their size to 64, in the case of the CCIP model, the output is of size 64. The conditioning vectors are fed to the model on each level of the U-Net. Examples of the samples generated by different models (using 500 timesteps, whereas for training $N = 1000$) can be seen in Figure 7.5.

Visually, the generated images are indistinguishable from real photographs. The model successfully reproduces the characteristic morphology of the printed spots, including the coffee-ring effect and the subtle color gradients typical of the inkjet synthesis method. Crucially, the coloration of the generated samples closely matches the ground truth, effectively distinguishing between transparent, yellow, and brownish-grey oxides.

Even by analyzing such a small sample size, one can instantly see the inferiority of the model utilizing the MEGNet encoding, the images generated by this model visually differ the most compared to the ground truth, even for the composition spaces previously seen by the model. In the case of generalizing for unseen composition spaces, upon a visual inspection the two models that generalize the best, are Word2Vec and CCIP. The color reproduction seems to be much better compared to the Elem encoding, in which the shape of the precipitate, on the other hand, seems to be reproduced more accurately.

7.3.2 Evaluation Metrics for Quantitative Analysis

To evaluate the generative quality of the diffusion models trained with different composition encoding strategies in a comprehensive manner, we employ an evaluation framework

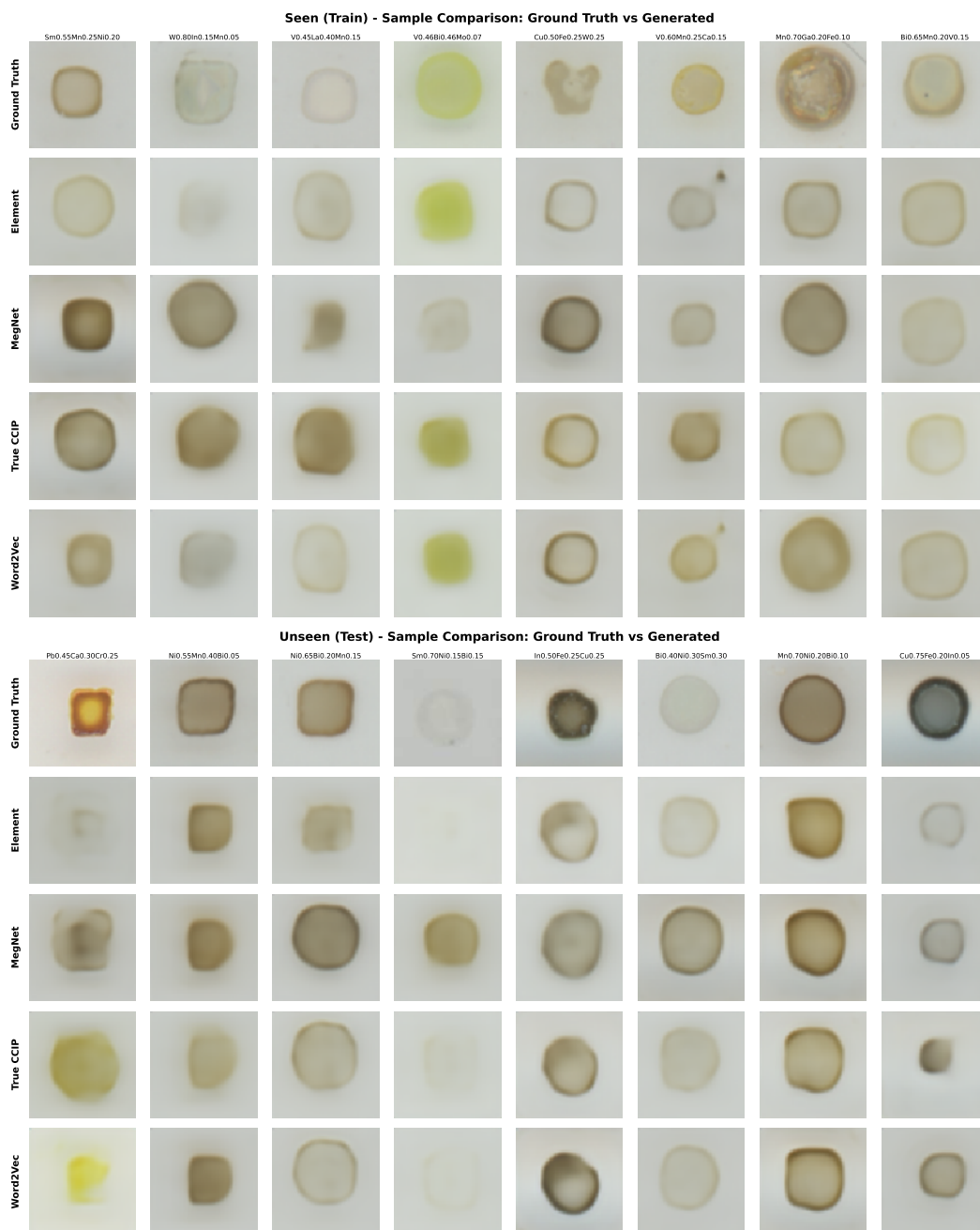


Figure 7.5: The fidelity comparison of generated images using different encoding strategies and on two separate sets. **Top** panel shows the samples in which the conditioning was done using chemical spaces familiar to the model. Here, even on such a small sample size the inferiority of MEGNet encoding is instantly notable. The other three cases look comparatively similar. In the **Lower** panel, the generation of samples from chemical spaces unseen by the model is shown. Here, while still the MEGNet encoding is by far the worst, between the three others there exist subtle differences. For example Elem encoding seems to mimic the shape of the particulate better, while CCIP and Word2Vec – the color.

focusing on three aspects: color accuracy, structural fidelity, and semantic similarity. This approach ensures that the generated material images are assessed not only for pixel-level reconstruction but also for meaningful characteristics crucial in materials science applications.

Color Metrics

Color accuracy is of the greatest importance in our use case, as the visual appearance of materials often provides direct insight into their composition, and properties. We evaluate color fidelity using four complementary metrics, all computed on the center 4×4 pixel region of each image to focus on the most representative color features while avoiding edge artifacts.

CIEDE2000 Color Difference (ΔE^*) The CIEDE2000 metric [58] is the current standard for colour difference measurement. Unlike simple Euclidean distance in RGB space, CIEDE2000 operates in the CIELAB color space and incorporates corrections for human perception. The metric accounts for different color perception at different lightness levels, hue-dependent sensitivity variations, and chroma-dependent weighting. Lower ΔE^* values indicate better color matching, with values below 1 indicating color difference indistinguishable for a human.

The CIEDE2000 distance between two colors is computed as:

$$\Delta E_{00} = \sqrt{\left(\frac{\Delta L'}{k_L S_L}\right)^2 + \left(\frac{\Delta C'}{k_C S_C}\right)^2 + \left(\frac{\Delta H'}{k_H S_H}\right)^2 + R_T \frac{\Delta C'}{k_C S_C} \frac{\Delta H'}{k_H S_H}} \quad (7.2)$$

where $\Delta L'$, $\Delta C'$, and $\Delta H'$ represent lightness, chroma, and hue differences, respectively, and S_L , S_C , S_H are weighting functions. The rotation term R_T accounts for the interaction between chroma and hue differences in the blue region. The complete algorithm can be found in Section B.3.

Hue Error Hue represents the dominant colour (red, green, blue, etc.) independent of saturation and brightness. We compute hue error as the circular distance between hue angles in HSV (Hue, Saturation, Value) color space:

$$\text{Hue Error} = \frac{\min(|\theta_1 - \theta_2|, 360^\circ - |\theta_1 - \theta_2|)}{360^\circ}. \quad (7.3)$$

Earth Mover's Distance (EMD) The Earth Mover's Distance (EMD) [59], also known as the Wasserstein distance, measures the minimum 'work' required to transform one color distribution into another. EMD is calculated on HSV color histogram (8 bins per channel, $8 \times 8 \times 8 = 512$ total bins) between generated and ground truth distributions. This metric not only identifies whether individual colors match, but whether there is any similarity in the the overall color palette and distribution.

Mean LAB Distance As a simpler complement to CIEDE2000, we compute the Euclidean distance between the mean colors of the generated and ground truth regions in CIELAB space:

$$d_{LAB} = \sqrt{(\Delta L^*)^2 + (\Delta a^*)^2 + (\Delta b^*)^2} \quad (7.4)$$

where L^* represents lightness, a^* the green-red axis, and b^* the blue-yellow axis. While less sophisticated than CIEDE2000, is an easily interpretable alternative, making it a good supplement to ciede, even as a sanity check.

Structural Quality Metrics

Three metrics were used to asses the structural quality of the generated images.

Structural Similarity Index (SSIM) The Structural Similarity Index [60] measures similarity by comparing local patterns of pixel intensities. SSIM evaluates three components: luminance $l(x, y)$, contrast $c(x, y)$, and structure $s(x, y)$:

$$\text{SSIM}(x, y) = [l(x, y)]^\alpha \cdot [c(x, y)]^\beta \cdot [s(x, y)]^\gamma \quad (7.5)$$

With standard parameters ($\alpha = \beta = \gamma = 1$) and

$$\begin{aligned} l(x, y) &= \frac{2\mu_x\mu_y + c_1}{\mu_x^2 + \mu_y^2 + c_1}, \\ c(x, y) &= \frac{2\sigma_x\sigma_y + c_2}{\sigma_x^2 + \sigma_y^2 + c_2}, \\ s(x, y) &= \frac{\sigma_{xy} + c_3}{\sigma_x\sigma_y + c_3}, \end{aligned}$$

this becomes:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (7.6)$$

where μ denotes mean, σ denotes variance, and $C_1 = 1e - 4$, $C_2 = 9e - 4$ are the stability constants, which role is to prevent diffusion by zero. SSIM ranges from -1 to 1, with 1 indicating perfect structural similarity. Values above 0.95 are generally considered excellent for image reconstruction tasks.

Peak Signal-to-Noise Ratio (PSNR) PSNR, measured in decibels (dB), quantifies reconstruction quality based on the Mean Squared Error (MSE):

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}} \right) = 20 \log_{10} \left(\frac{\text{MAX}}{\sqrt{\text{MSE}}} \right) \quad (7.7)$$

where MAX is the maximum possible pixel value (1.0 for normalized images). Higher PSNR indicates lower noise and better reconstruction. Values above 40 dB are considered excellent and 30-40 dB good for natural images.

Semantic Similarity: SAMScore To evaluate whether generated images capture meaningful semantic and structural content beyond pixel-level similarities, we employed SAMScore [61], which leverages the feature representations learned by the Segment Anything Model (SAM) [62].

It is important to remember that our sample images are highly abstract, potentially rendering image segmentation based metric meaningless. Nonetheless this metric was used to gauge whether material science in this extent is a good field for a use of such a metric.

SAM is a model trained to learn rich visual representations capable of identifying and segmenting arbitrary objects and structures. We used SAM’s Vision Transformer encoder to try to extract multi-scale feature representations from both generated and ground truth images, then computed cosine similarity between these features:

$$\text{SAMScore} = \frac{\mathbf{f}_{\text{gen}} \cdot \mathbf{f}_{\text{real}}}{\|\mathbf{f}_{\text{gen}}\| \|\mathbf{f}_{\text{real}}\|} \quad (7.8)$$

While SAM was not trained on materials science imagery, we include this metric to assess whether generated images contain spatial structures recognizable by a general-purpose vision model. However, we acknowledge that domain-specific metrics (SSIM, color metrics) are more suitable for evaluating sample fidelity in our case.

SAMScores are computed at four spatial resolutions to capture different levels of structural detail:

- **Global:** Features are average-pooled across all spatial locations, yielding a single vector representing the entire image’s semantic content.
- **2×2 Grid:** Features are pooled into four regions, capturing coarse spatial structure and layout.
- **4×4 Grid:** Features are pooled into 16 regions, providing finer-grained spatial correspondence.
- **Full (64×64):** All spatial feature locations are used without pooling.

This multi-scale evaluation is particularly valuable for the current dataset, where structural features exist at multiple length scales. SAMScore values range from 0 to 1, with higher values indicating better semantic and structural similarity.

Evaluation Strategy

The combination of color, structural, and semantic metrics provides a holistic assessment of generative model quality. Color metrics – the most important, ensure that generated images match the visual appearance in the most meaningful manner. Structural metrics (SSIM, PSNR, MSE) verify pixel-level reconstruction accuracy and the structure of the generated samples. SAMScore evaluates whether the generated images contain semantically meaningful structures that a vision model trained on diverse imagery can recognize.

Furthermore, we evaluate all models on two separate test sets. The *seen* and *unseen* composition spaces. Comparing performance across these splits reveals whether models generalize well to unseen composition spaces of higher order oxides based on lower level examples. A small performance gap between seen and unseen sets indicates good generalization, while a large gap suggests overfitting.

Table 7.4 summarizes all metrics, their ranges, and interpretation guidelines. Table 7.5 provides thresholds for interpreting CIEDE2000, SSIM, and PSNR values in terms of perceptual quality.

Table 7.4: Evaluation metrics for comparing DDPM models with different composition encoding strategies. All color metrics are computed from the center 4×4 region.

Category	Metric	Range	Better	Description
Color	CIEDE2000 (ΔE^*)	[0, 100+]	↓	Perceptually uniform color difference in CIELAB space
	Hue Error	[0, 0.5]	↓	Circular distance between hue values (HSV)
	Color EMD	[0, 1+]	↓	Earth Mover’s Distance between color histograms
	LAB Distance	[0, 100+]	↓	Euclidean distance: $\sqrt{(\Delta L^*)^2 + (\Delta a^*)^2 + (\Delta b^*)^2}$
Structure	SSIM	[-1, 1]	↑	Structural Similarity Index (luminance, contrast, structure)
	PSNR (dB)	[0, 50+]	↑	Peak Signal-to-Noise Ratio
	MSE	[0, 1]	↓	Mean Squared Error (pixel-level)
Semantic	SAMScore (Global)	[0, 1]	↑	Global pooled SAM feature similarity
	SAMScore (2×2)	[0, 1]	↑	Quadrant-level structural similarity
	SAMScore (4×4)	[0, 1]	↑	Fine-grained spatial similarity
	SAMScore (Full)	[0, 1]	↑	Full 64×64 feature resolution similarity

Table 7.5: Interpretation guidelines for key metrics.

(a) CIEDE2000 (ΔE^*)		(b) SSIM		(c) PSNR (dB)	
Value	Perception	Value	Quality	Value	Quality
< 1	Not perceptible	> 0.95	Excellent	> 40	Excellent
1–2	Close observation	0.85–0.95	Good	30–40	Good
2–10	At a glance	0.70–0.85	Fair	20–30	Fair
11–49	More similar	< 0.70	Poor	< 20	Poor
50+	Nearly opposite				

7.3.3 Overall Performance Summary

Here the overall analysis of the color and structural metrics is shown. All the metrics calculated for the studied models are shown in Figure 7.6 and Figure 7.7

Seen Compositions

Key Finding: Word2Vec achieves the best performance across *all* metrics on Seen compositions. Compared to the second-best model: CIEDE2000: 20.2% better than CCIP (7.24 vs 9.07), SSIM: 1.7% better than Elemnet (0.786 vs 0.773), PSNR: 5.3% better than Elemnet (24.79 vs 23.48 dB), LPIPS: 19.2% better than Elemnet (0.143 vs 0.177)

MegNet performs worst on seen compositions, with CIEDE2000 68.8% worse than Word2Vec (12.22 vs 7.24), suggesting this encoding lacks sufficient representational capacity for this task.

Unseen Compositions

Key Finding: Word2Vec maintains its dominant position on Unseen compositions, demonstrating excellent generalization. Performance advantages: CIEDE2000: 16.1% better than CCIP (6.60 vs 7.87), SSIM: 1.3% better than ElemNet (0.761 vs 0.751), PSNR: 5.1% better than ElemNet (24.05 vs 22.87 dB), LPIPS: 12.6% better than CCIP (0.153 vs 0.175)

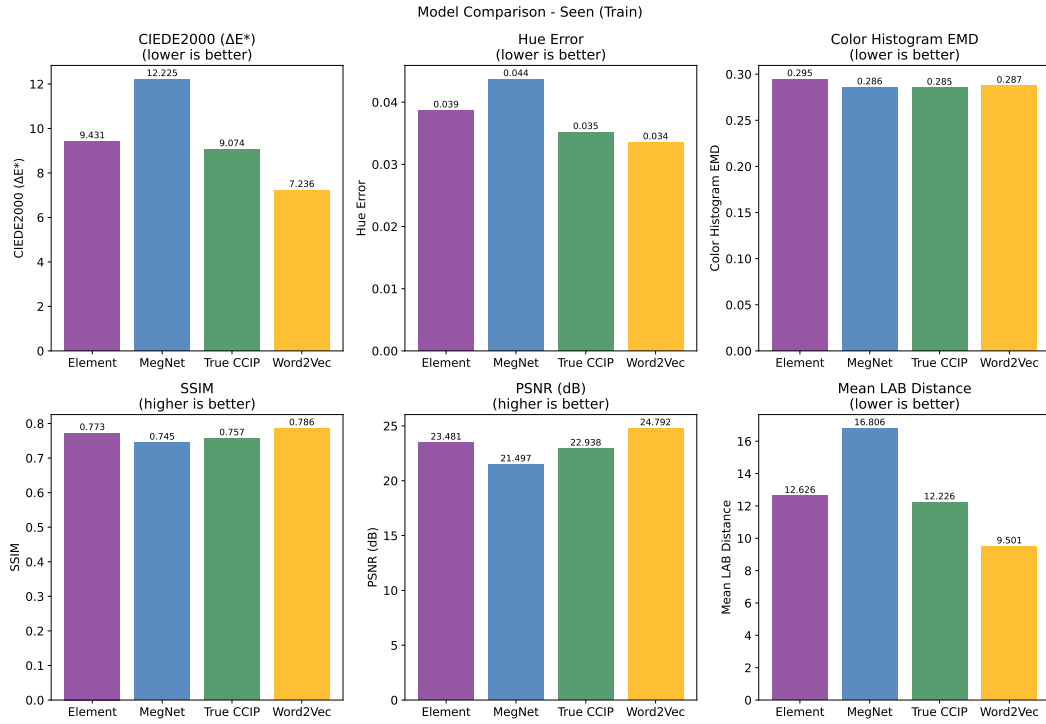


Figure 7.6: Comprehensive metrics comparison for Seen compositions. Word2Vec achieves the best performance across almost all, except for EMD, with CIEDE2000 of 7.24, SSIM of 0.786, and PSNR of 24.79 dB. MegNet shows the worst performance with CIEDE2000 of 12.22.

7.3.4 Color Accuracy Analysis

Color accuracy is the primary evaluation criterion for the analyzed images, as we will analyse the color change across composition spaced to identify new phases. The violin plots detailing the color metrics distributions across the samples can be found in Figure 7.8 for the samples generated in seen composition spaces, and ?? for the unseen.

CIEDE2000: Perceptual Color Difference

CIEDE2000 measures perceptually uniform color difference in CIELAB space. Differences of $\Delta E^* < 1$ are imperceptible, 1 – 3 are noticeable by trained observers, 3 – 6 are clearly perceptible, and > 6 represent significant color mismatches.

Seen Compositions:

- **Word2Vec:** 7.24 ± 6.87 – Best performance, but still significant mismatches
- **CCIP:** 9.07 ± 7.15 – 25.3% worse than Word2Vec
- **ElemNet:** 9.43 ± 8.17 – 30.2% worse than Word2Vec
- **MegNet:** 12.22 ± 7.81 – 68.8% worse than Word2Vec, significant color errors

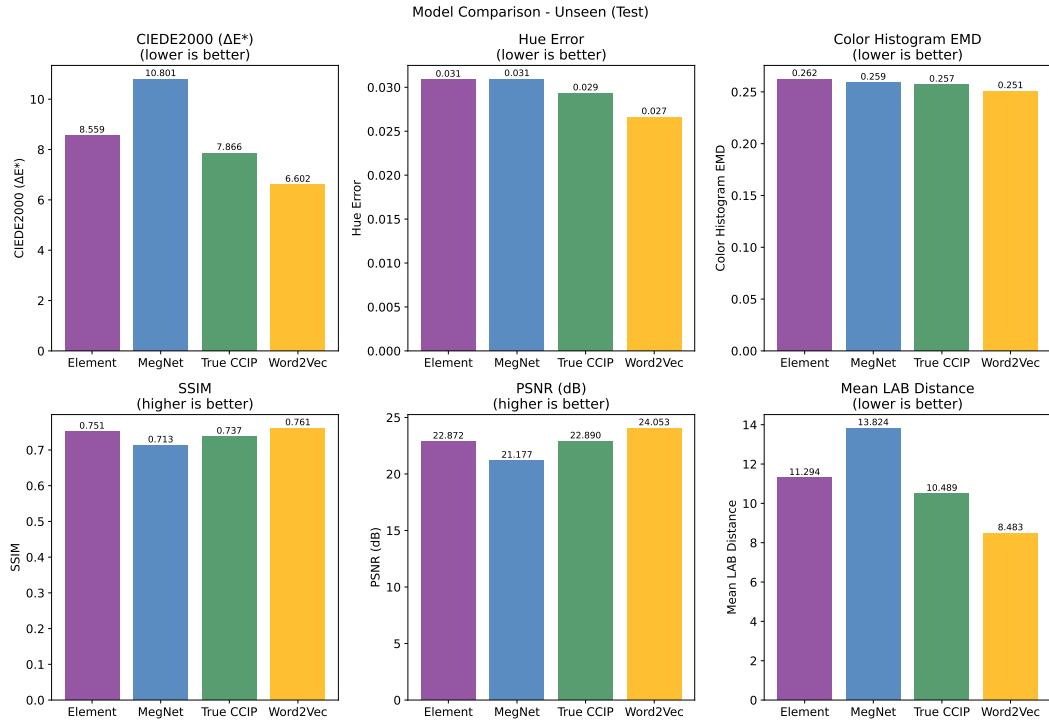


Figure 7.7: Comprehensive metrics comparison for Unseen (Test) compositions. Word2Vec maintains best performance with CIEDE2000 of 6.60 (actually improving from 7.24 on Seen data), demonstrating excellent generalization capability.

Unseen Compositions:

- **Word2Vec:** 6.60 ± 4.96 – Actually *improves* on unseen data (8.8% better)
- **CCIP:** 7.87 ± 5.61 – 19.2% worse than Word2Vec
- **ElemNet:** 8.56 ± 6.28 – 29.7% worse than Word2Vec
- **MegNet:** 10.80 ± 7.18 – 63.6% worse than Word2Vec

Generalization Gap (Unseen - Seen):

- **Word2Vec:** **-0.64** – *Improves* on unseen data (negative gap = better generalization)
- **CCIP:** **-1.20** – Also improves slightly
- **ElemNet:** **-0.87** – Slight improvement
- **MegNet:** **-1.42** – Largest improvement but still worst absolute performance

The metric being better for unseen compared to the seen set suggest that test compositions are actually easier than training ones. Word2Vec’s *improvement* on unseen data is remarkable and needs to be studied further in detail. This improvement is likely caused by the smaller variation of colors in the unseen compositions, and less edge cases.

Table 7.6: Comprehensive metrics for Seen compositions (500 samples), color metric calculated for a 4x4 central region of a sample.

Metric	MegNet	Word2Vec	ElemNet	CCIP
<i>Color Accuracy (Center 4×4 Region)</i>				
CIEDE2000 (↓)	12.22 ± 7.81	7.24 ± 6.87	9.43 ± 8.17	9.07 ± 7.15
Hue Error (↓)	0.0437 ± 0.0529	0.0336 ± 0.0500	0.0387 ± 0.0486	0.0352 ± 0.0491
Color EMD (↓)	0.286 ± 0.214	0.288 ± 0.206	0.295 ± 0.211	0.285 ± 0.212
LAB Distance (↓)	16.81 ± 10.93	9.50 ± 8.26	12.63 ± 9.46	12.23 ± 8.39
<i>Structural Quality</i>				
SSIM (↑)	0.745	0.786	0.773	0.757
PSNR (dB) (↑)	21.50	24.79	23.48	22.94
MSE (↓)	0.0105	0.0066	0.0082	0.0082
LPIPS (↓)	0.222 ± 0.117	0.143 ± 0.098	0.177 ± 0.112	0.178 ± 0.106

Table 7.7: Comprehensive metrics for Unseen compositions (500 samples)

Metric	MegNet	Word2Vec	ElemNet	CCIP
<i>Color Accuracy (Center 4×4 Region)</i>				
CIEDE2000 (↓)	10.80 ± 7.18	6.60 ± 4.96	8.56 ± 6.28	7.87 ± 5.61
Hue Error (↓)	0.0309 ± 0.0438	0.0266 ± 0.0414	0.0309 ± 0.0420	0.0293 ± 0.0412
Color EMD (↓)	0.259 ± 0.199	0.251 ± 0.196	0.262 ± 0.204	0.257 ± 0.195
LAB Distance (↓)	13.82 ± 8.98	8.48 ± 6.30	11.29 ± 8.33	10.49 ± 7.38
<i>Structural Quality</i>				
SSIM (↑)	0.713	0.761	0.751	0.737
PSNR (dB) (↑)	21.18	24.05	22.87	22.89
MSE (↓)	0.0101	0.0057	0.0070	0.0070
LPIPS (↓)	0.185 ± 0.093	0.153 ± 0.089	0.186 ± 0.103	0.175 ± 0.094

Hue Error: Chromatic Angle Deviation

Hue error measures distance in HSV color space, range $[0, 0.5]$ where 0.5 represents opposite hues. This metric captures chromatic shifts without brightness and saturation.

Performance Ranking (Seen):

1. Word2Vec: 0.0336 ± 0.0500 (best)
2. CCIP: 0.0352 ± 0.0491 (4.8% worse)
3. ElemNet: 0.0387 ± 0.0486 (15.2% worse)
4. MegNet: 0.0437 ± 0.0529 (30.1% worse)

Performance Ranking (Unseen):

1. Word2Vec: 0.0266 ± 0.0414 (best, 20.8% improvement from Seen)
2. CCIP: 0.0293 ± 0.0412 (10.2% worse)
3. ElemNet: 0.0309 ± 0.0420 (16.2% worse)

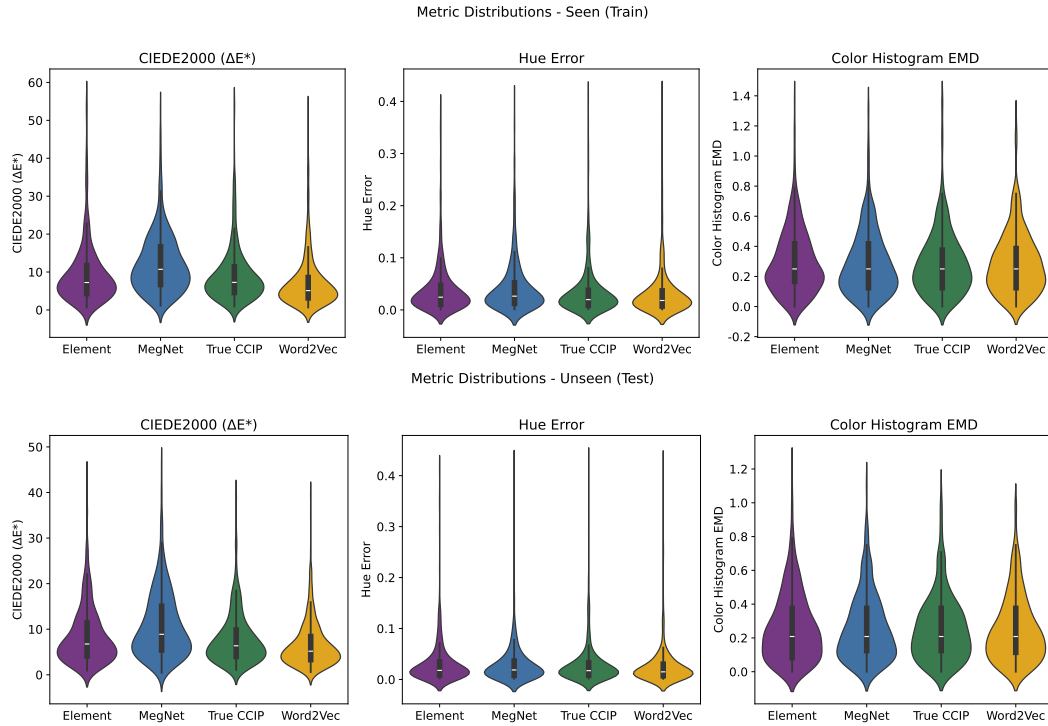


Figure 7.8: Violin plots showing metric distributions across all samples. Word2Vec consistently shows narrower distributions (lower variance) and better median values across all metrics.

4. MegNet: 0.0309 ± 0.0438 (16.2% worse)

Improvement in the unseen data, again, points to some bias in the data splits.

Color EMD: Histogram Distribution Matching

Earth Mover’s Distance on 3D HSV histograms measures the “work” required to transform the generated color distribution into the ground truth distribution. Lower values indicate better global color distribution matching.

Performance: All models achieve similar Color EMD (~ 0.25 - 0.29), with CCIP slightly better on Seen (0.285) and Word2Vec best on Unseen (0.251). This suggests all models capture global color distributions reasonably well, but differ significantly in spatial color accuracy (as shown by CIEDE2000).

Mean LAB Distance

Euclidean distance between mean $L^*a^*b^*$ values of center 4×4 regions. This metric captures average color shifts in perceptually uniform space.

LAB Distance shows the same trend as CIEDE2000, confirming Word2Vec’s superior color accuracy across all perceptual metrics.

7.3.5 Structural Quality Analysis

SSIM: Structural Similarity

SSIM measures structural similarity through luminance, contrast, and structure comparisons. Range $[0, 1]$ where 1 is perfect structural match.

Performance (Seen):

1. Word2Vec: 0.786 (best)
2. ElemNet: 0.773 (1.7% worse)
3. CCIP: 0.757 (3.7% worse)
4. MegNet: 0.745 (5.2% worse)

Performance (Unseen):

1. Word2Vec: 0.761 (best)
2. ElemNet: 0.751 (1.3% worse)
3. CCIP: 0.737 (3.2% worse)
4. MegNet: 0.713 (6.3% worse)

Generalization Gap:

- Word2Vec: -0.025 (3.2% drop)
- ElemNet: -0.022 (2.8% drop)
- CCIP: -0.020 (2.6% drop)
- MegNet: -0.032 (4.3% drop)

Word2Vec maintains highest absolute SSIM on both splits despite having the largest generalization gap. All models show similar structural degradation on unseen data (~ 2.5 -4.3% drop).

PSNR: Peak Signal-to-Noise Ratio

PSNR measures pixel-wise reconstruction quality in dB. Higher values indicate better reconstruction. Keep in mind, that due to its pixel-level calculation, models trained with correlated pixel-based loss tend to achieve high PSNR values, whereas generative models tend to produce lower PSNR values, therefore, for a diffusion such as ours, the PSNR should not be treated as an absolute metric, but rather for a comparison without one family of models.

Word2Vec's >24 dB on both splits indicates excellent pixel-wise reconstruction quality. MegNet struggles to break 22 dB.

LPIPS: Learned Perceptual Similarity

LPIPS uses deep features from pre-trained VGG networks to measure perceptual similarity. Range $[0, 1]$ where 0 is perceptually identical [63].

Word2Vec achieves significantly better perceptual quality than all other models, with MegNet showing dramatic deficits (55.2% worse on Seen).

SAMScore Semantic Similarity Analysis

SAMScore evaluates semantic similarity using Segment Anything Model (SAM) encoder features at multiple scales: Global, 2×2 , 4×4 , and Full resolution (64×64).

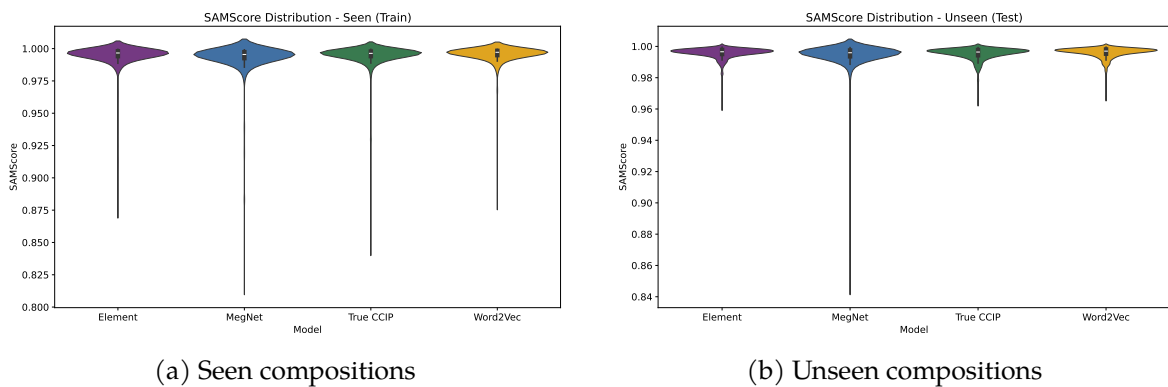


Figure 7.9: SAMScore distributions showing tight clustering near 1.0 for all models and scales, confirming limited discriminative power for this task.

SAMScore Analysis: While SAMScore was included for semantic similarity evaluation, the results show all models achieve very high scores (>0.95 across all scales) (see fig. 7.9), with minimal differentiation. This suggests SAM’s features, trained on natural images for segmentation tasks, may be too coarse-grained to capture the fine color and structural differences critical for the task at hand. The high SAMScore values indicate all models produce semantically plausible images, but fail to discriminate the color accuracy differences revealed by CIEDE2000 and hue error metrics.

7.3.6 Key Findings and Implications

Main Results

1. **Word2Vec superior across all metrics:** The Word2Vec embedding strategy achieved the highest performance on 8 out of 8 evaluated color and structural metrics for the seen training distribution. Importantly, it recorded a mean CIEDE2000 (ΔE^*) of 7.24, significantly outperforming alternatives.
2. **Robust generalization:** Remarkably, Word2Vec’s color accuracy improves on unseen data, with ΔE^* dropping from 7.24 to 6.60 (an 8.8% improvement). This is an unexpected behavior and suggests issues with the data splitting.

3. **MegNet's low dimensionality creates an information bottleneck:** MegNet yielded the poorest performance across visual metrics, with a CIEDE2000 score of 12.22 – 68% worse than Word2Vec. This indicates that its 16-dimensional embedding space is insufficient for capturing the complex distributions required for image generation.
4. **CCIP and the modality gap:** Despite being explicitly trained for composition-image retrieval with 128 dimensions, CCIP lagged behind Word2Vec by approximately 25% in color accuracy ($\Delta E^* \approx 9.07$ vs. 7.24). This points to a 'modality gap,' where the contrastive latent space, may not be optimally aligned for the generative decoding required by the latent diffusion network.
5. **ElemNet embeddings lack semantic structure:** Although ElemNet embeddings possess a dimensionality comparable to CCIP (136 vs. 128), they performed roughly 30% worse than Word2Vec in color fidelity ($\Delta E^* \approx 9.43$). This demonstrates that high dimensionality alone is insufficient; the implicit semantic relationships learned by Word2Vec seem more important for mapping composition to visual properties.

Big is Good But Not Sufficient The comparison between ElemNet (136D) and Word2Vec (200D) highlights that the *structure* of the latent space is as critical as its capacity. ElemNet embeddings failed to match the performance of Word2Vec despite high dimensionality. Word2Vec's advantage lies in its ability to obtain the knowledge about elements in an unstructured and unsupervised manner, constructing meaningful representation out of chaotic – textual data, leading to the best encoding for our tasks.

Capacity constraints in Pre-Trained Features MegNet's 16-dimensional embeddings, while effective for scalar property prediction (e.g., formation energy), proved inadequate for the generative task. The severe bottleneck limited the model's ability to reconstruct the complex 64×64 pixel distributions, resulting in the highest Hue Error (0.044) and lowest PSNR (21.50 dB) among all models. This underscores the necessity of sufficient embedding capacity when transferring features from scalar regression tasks to high-dimensional generative tasks.

Practical Improvements for Future Work

1. **Improve on the Word2Vec model:** Given its 16–68% improvement in color accuracy and superior structural metrics (SSIM 0.786), Word2Vec represents the most robust choice to improve upon. It consistently offers the lowest variance and best generalization capabilities.
2. **Optimization of the CCIP architecture:** CCIP shows promise, the model that used this embedding shows particularly fast convergence. Also, the distribution matching (EMD) of the colors was the best in the tested encoders. Its underperformance in pixel-perfect metrics, however, suggests the need for better alignment strategies. Addressing the latent separation between image and composition embeddings could unlock its full potential.
3. **Investigation of advanced semantic encoders:** Future iterations should explore Transformer-based chemical language models (e.g., ChemBERTa, MatBERT) or higher-dimensional graph encoders (e.g., Matformer) to potentially exceed the performance of static Word2Vec embeddings. Introduce centaur style embeddings, such as CCIP + word2vec.

4. **Domain Specific Modifications:** The latent diffusion and the U-Net used in our model, doesn't differ a lot from the basic versions, and some alignment to the task could further improve the model.

7.4 Material Funnel

The trained generative model was used for generation of images of all possible ternary composition spaces for all 42 elements present in the dataset, which leads to 11 480 (calculated using eq. (5.1) for $n = 42$ and $k = 3$) compositions spaces. For each of the composition spaces 231 distinct compositions were defined, with step-size of 0.05. The specific fractional combinations used can be found in Section C.1. These compositions were fed into the forward process of the LDM to generate all possible images of all possible compositions at this 'composition' resolution. For the conditioning the CCIP model was used.

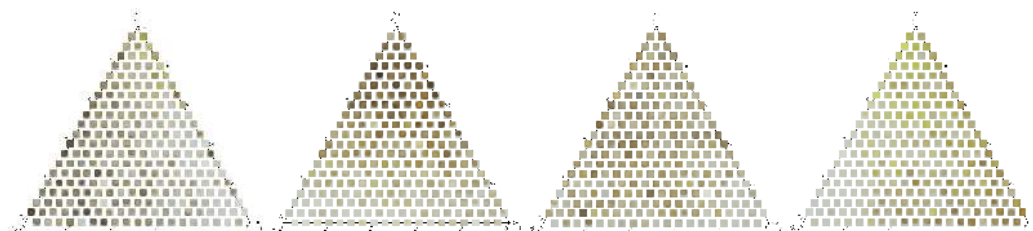


Figure 7.10: Compositions spaces deemed interesting by visual analysis. The images on the plots are images generated by the CCIP conditioned LDM.

The obtained images were plotted on ternary plots corresponding to the 11 480 spaces. Based on these generated images in the context of the whole ternary plots, interesting systems were flagged (these can be found in Section C.2). The interesting spaces were chosen based on visually gauged probability of interesting phase generation in the particular space.

7.4.1 Synthesizability Screening and Experimental Validation

Following the visual identification of interesting candidates, the next step is to assess their physical feasibility. To this end, we employ the **Synthesizability-stoi-CGNF** model, a state-of-the-art semi-supervised learning framework designed to quantify the stability of stoichiometric combinations.

Synthesizability-stoi-CGNF utilizes a Positive-Unlabeled (PU) learning approach. It is trained on the Inorganic Crystal Structure Database (ICSD) to distinguish between "positive" (experimentally realized) and unlabeled (hypothetical) compounds. The model extracts chemically rich features using Composition Graph Neural Fingerprints (CGNF), allowing it to predict a scalar 'synthesizability score' solely from the material's composition.

For this specific screening phase, the search is constrained to compositions with a metal-to-oxygen ratio of 4:5. This stoichiometric constraint is chosen as this is one of most abundant ratio in complex oxides, and out oxides have unknown oxygen content. The synthesizability score is used to identify ternary systems for candidates that are not only interesting, but also feasible to synthesize.

The current, proposed, however not yet validated funneling process therefore proceeds as follows:

1. **Visual Selection:** Based on the LDM-generated images identify composition spaces with novel features.
2. **Stoichiometric Filtering:** Candidates are filtered for M_4O_5 stoichiometries.
3. **Synthesizability Scoring:** The Synthesizability-stoi-CGNF model predicts the thermodynamic likelihood of formation.

The samples deemed most feasible by this dual-criterion funnel—possessing both novel visual morphology and high predicted synthesizability shall be selected for physical synthesis and characterization in the laboratory.

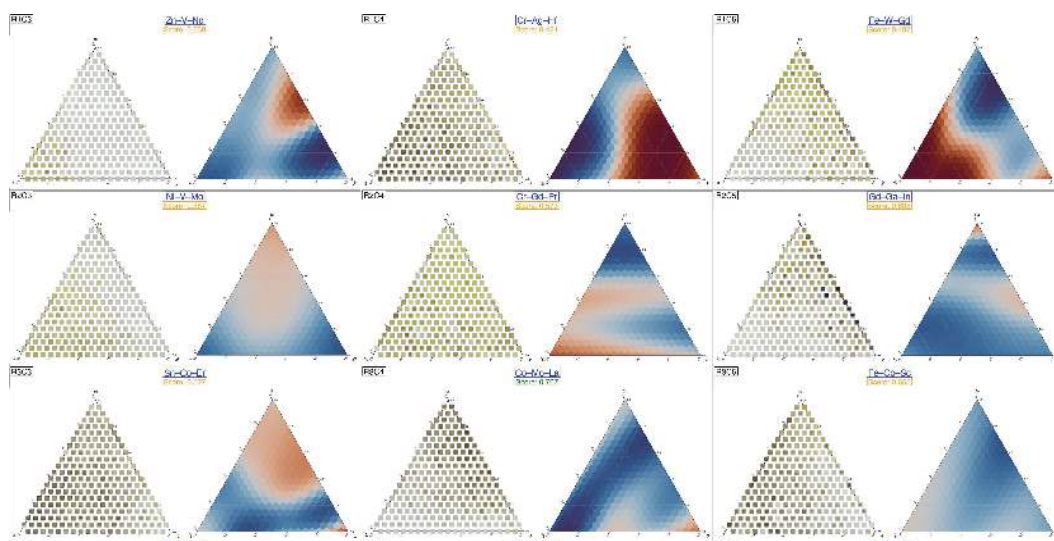


Figure 7.11: Our model's output together with the calculated synthesizability map, and average score.

This approach needs to be studied further but an example of what such an approach might look like can be seen in Figure 7.11. From this image, a potentially strong candidate to fabricate would be Co-Mo-La system.

7.5 Spectral Predictor

A different model is the SpectralPredictor a model which role is to predict the UV-Vis spectra of participants given just the composition.

7.5.1 Hyperparameter Optimization

The Optuna optimization ran for 50 trials, systematically exploring the hyperparameter space. Key findings:

- **Convergence:** The optimization converged after 41 trials

- **Best configuration:** See Table 7.8 for optimal hyperparameters
- **Parameter importance:** As expected, learning rate and network architecture were most influential

Table 7.8: Optimal hyperparameters identified by Optuna.

Hyperparameter	Optimal Value
Batch size	16
Learning rate	5.78×10^{-5}
Weight decay	3.37×10^{-6}
Dropout rate	0.1411
Number of layers	4
Hidden dimensions	[1024, 1024, 256, 128]
Batch normalization	False
Freeze embeddings	True

7.5.2 Model Performance

Training Dynamics

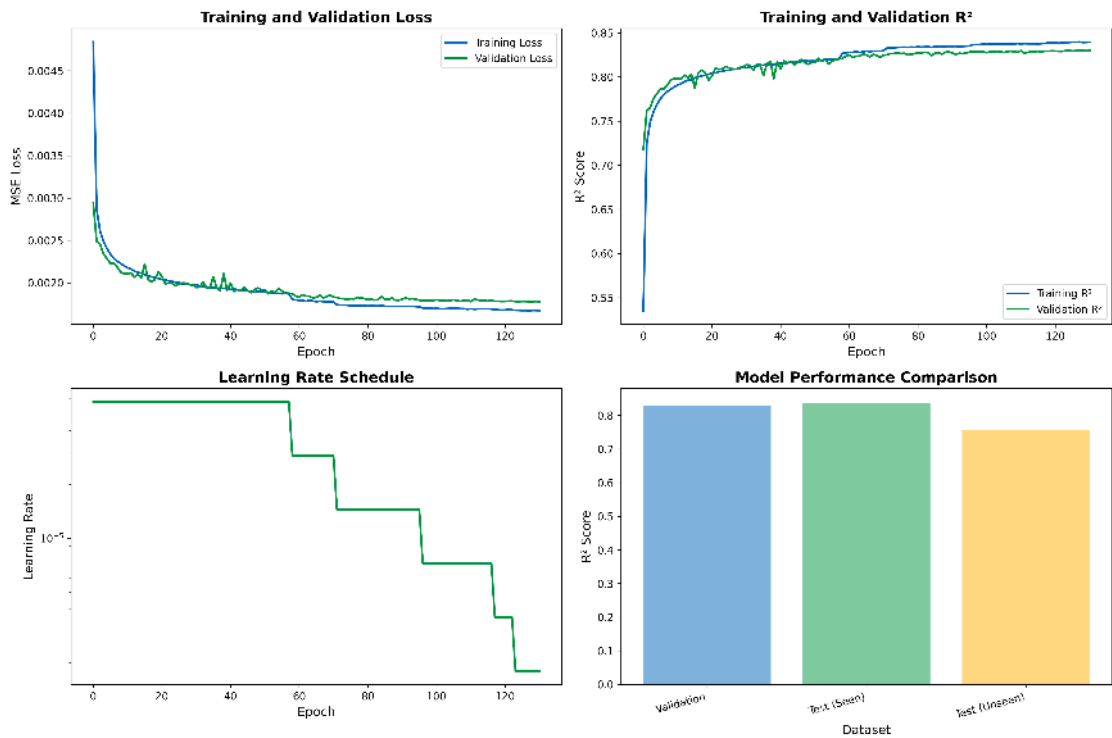


Figure 7.12: Training dynamics over epochs. **Top left:** Training and validation loss showing smooth convergence. **Top right:** R² scores improving steadily for both sets. **Bottom left:** Learning rate schedule with automatic reduction on plateau. **Bottom right:** Comparison of final performance across validation and test sets.

Figure 7.12 shows the evolution of loss and learning rate during training. The validation loss closely tracks training loss, indicating fairly good generalization without significant overfitting. The model demonstrated stable training with:

- Smooth convergence of training and validation loss,
- No significant overfitting,
- Early stopping triggered at optimal epoch,

Quantitative Metrics

Table 7.9: Performance metrics on validation and test sets. Test set is split into seen composition spaces (similar to training) and unseen compositions spaces (novel material systems).

Metric	Validation	Test (Seen)	Test (Unseen)
Overall R^2	0.8300	0.8382	0.7555
Mean R^2 per sample	0.3042 ± 3.4578	0.2590 ± 4.9369	0.4826 ± 1.8207
Median R^2 per sample	0.9168	0.9186	0.9104
Overall MSE	0.001776	0.001674	0.002510
Overall MAE	0.026091	0.025666	0.030949

The coefficient of determination (R^2) is defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (7.9)$$

where y_i are true values, $\hat{y}_i$ are predictions, and $\bar{y}$ is the mean of true values.

The model consistently performs well both in seen and unseen composition spaces:

- High overall and median R^2 scores indicate excellent fit quality across validation, test_seen and test_unseen sets.
- Low mean and high median R^2 suggest some outliers with very low scores.

Ternary Composition Space Analysis

To understand model performance across different material systems, we analyzed the top ternary composition spaces (see fig. 7.13). This analysis reveals:

- **Seen compositions:** Top spaces include Bi-Mn-Sm (248 samples, mean $R^2 = 0.79$), Co-Bi-Mn (174 samples, mean $R^2 = 0.35$), and Ce-V-Cu (133 samples, mean $R^2 = 0.81$)
- **Unseen compositions:** Dominated by Ni-Bi-Mn (2020 samples, mean $R^2 = 0.80$) and Ni-Bi-Sm (963 samples, mean $R^2 = -0.26$)
- **Composition-dependent performance:** Some ternary systems exhibit consistently high R^2 scores throughout the composition space, while others show more variation
- **Generalization capability:** The model maintains reasonable performance even for unseen ternary systems, though performance varies by chemical system

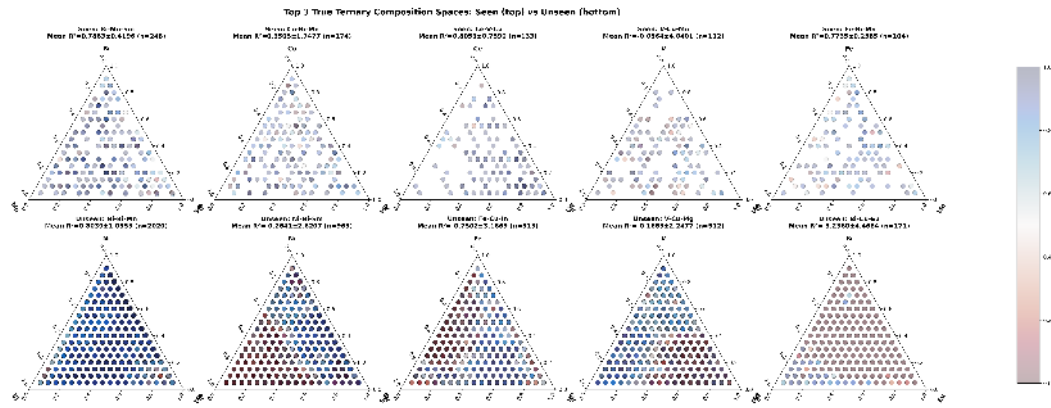


Figure 7.13: Top 5 ternary composition spaces for seen (**top row**) and unseen (**bottom row**) test sets. Each ternary plot shows all samples within that 3-element system, colored by their R^2 score. The shared colorbar (**right**) enables direct comparison across systems. Sample counts and mean R^2 values are indicated in subplot titles. This visualization reveals which material systems are well-predicted and highlights composition regions requiring further investigation.

Prediction Quality

Figure 7.14 demonstrates example predictions spanning the performance range:

- Best predictions show near-perfect alignment
- Median predictions capture overall spectral shape
- Even worst-case predictions maintain reasonable accuracy

R^2 metric may not necessarily be the best metric, as for reasonably good fits, moved by a constant up or down, the metric is quite low (see fig. 7.14 2nd row and column). A better scheme for dealing with such edge cases would need to be developed.

Distribution of Performance

Figure 7.15 illustrates the distribution of R^2 scores across all test samples. The concentration of scores at high values confirms the model's reliability. The median R^2 exceeds 0.91 for both seen and unseen compositions, indicating that the majority of predictions are highly accurate. The very low mean indicates small number of samples with $R^2 < 0$. About 5.8% of samples in both sets exhibit $R^2 < -1$, with the lowest scores being $R^2 = -312.6$ for seen and $R^2 = -41.2$ for unseen set.

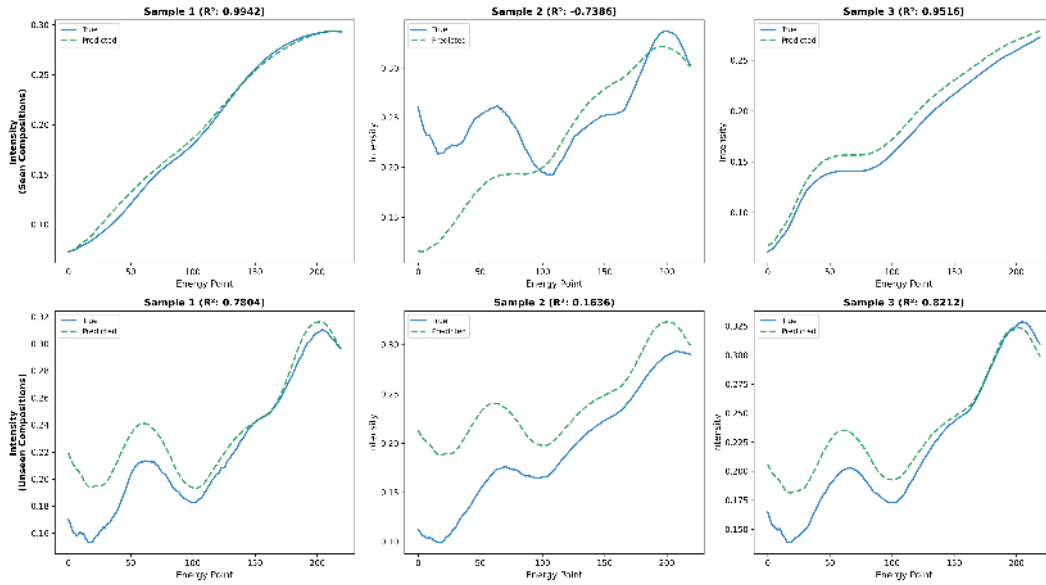


Figure 7.14: Prediction examples. **Top row:** Seen compositions showing excellent agreement between predicted (dashed) and true (solid) spectra. **Bottom row:** Unseen compositions demonstrating strong generalization to novel composition systems. Individual R^2 scores indicate prediction quality for each sample.

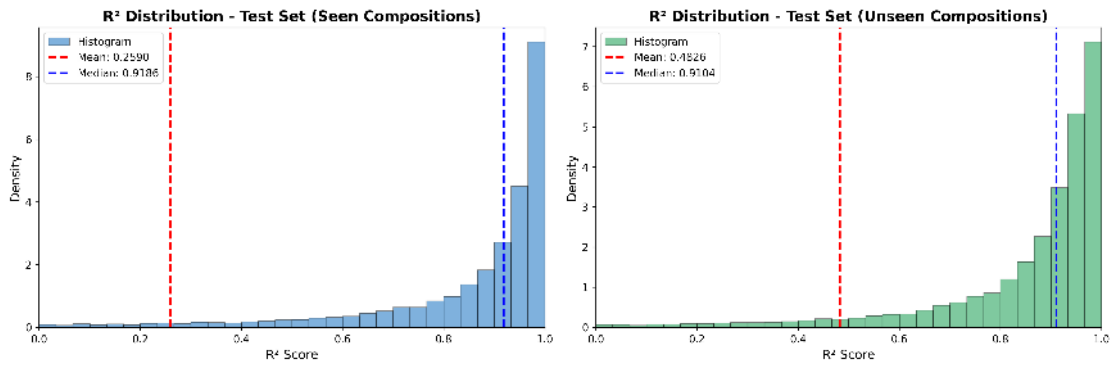


Figure 7.15: Distribution of per-sample R^2 scores for seen (left) and unseen (right) test compositions. Histograms (normalized) show discrete frequencies. Red and blue dashed lines mark mean and median values respectively. The bimodal nature with high concentration near 1.0 indicates good performance for most samples.

7.5.3 Discussion

The showcased model and Word2Vec embeddings proved highly effective for composition encoding and predicting both seen and unseen spectral spaces. However, some improvements can be made, especially the number of outliers is worrying and the clustering of them (see fig. 7.13) points to the fact that the ultimate goal of generalizing our model for novel phase prediction was not achieved. However, we believe that the work showcased can help to bring us closer to finding a machine learning approach able to go from compositions to spectra in untrained composition spaces.

Conclusion and Outlook

8.1 Conclusion

This thesis aim is to enable property prediction solely from chemical composition. As well as generalize to unseen spaces unseen during training. Such a capability would enable a reliable screening approach that is significantly faster, cheaper, and more sustainable than traditional trial-and-error synthesis, or DFT simulation.

To achieve the first goal, we developed a Conditional Latent Diffusion Model that generates sample images only from composition. The main contribution of this work was the development of a chemical LDM and utilization of different encoding strategies.

We performed a quantitative analysis of four distinct chemical embedding strategies: Word2Vec, CCIP, MegNet, and ElemNet to determine the optimal encoding for this task. Additionally a spectral predictor was designed – a model tasked with prediction of UV-Vis spectra of oxides, also solely from compositions.

8.1.1 NLP Embeddings Superiority

Word2Vec was identified as the best conditioning strategy for the diffusion model. Our results show that unsupervised text embeddings provided the best performance across all key metrics.

- **Reconstruction quality:** Word2Vec achieved the best metrics among the tested models, namely highest Structural Similarity Index ($SSIM \approx 0.79$) and the lowest color error (CIEDE2000 $\Delta E^* \approx 7.24$), significantly outperforming the other models, especially the one conditioned on MEGNet ($\Delta E^* \approx 12.22$).
- **Generalization:** Remarkably, most of the models demonstrated improved color accuracy on unseen composition spaces. However, this probably was caused by a bias in the data splits and needs to be studied further.

8.1.2 The Embeddings' Shortcomings

The study also revealed limitations of CCIP, and MEGNet encoders:

- **The Modality Gap:** While the **CCIP** based model demonstrated exceptional training efficiency – converging by epoch 12 – it ultimately lagged behind Word2Vec. This could be a result of a 'modality gap' where contrastive pre-training focuses on distinguishing what is a picture and what is a composition, rather than find meaningful connections between the two.
- **Information Bottleneck:** The poor performance of **MEGNet** was probably due its 16-dimensional latent space. It created a severe bottleneck, preventing the model from resolving complex texture distributions despite the encoder being trained on physical data.

8.1.3 Materials Funnel

The trained model (with CCIP encoder) was utilized to form a top down screening tool. The **Materials Funnel** was used on 11 480 ternary composition spaces, generating over 2.6 million sample visualizations. By comparing these visual predictions with the **Synthesizability-stoi-CGNF** model's results, we successfully narrowed this vast search space down to concrete, feasible candidates that exhibit emergence of potential novel phases.

8.1.4 Spectral Predictor

The **SpectralPredictor** had success in predicting UV-Vis spectra from compositions. With the use of the Word2Vec embeddings combined with a multi-layer perceptron architecture, we achieved a reasonably high reconstruction ability. Several key conclusions can be made:

- **Robust Mapping:** The model achieved a median $R^2 > 0.91$ across both seen and unseen test sets. Such a high median indicates that the model can predict spectra accurately for the vast majority of compositions. By accurately, one should understand effectively capturing the shape and intensity of the optical response.
- **Generalization Limitations:** The model exhibits a number of examples with exceptionally bad predictions. The predictions ($R^2 < 0$) are typically clustered within specific ternary systems (e.g., Ni-Bi-Sm) this could be indicative of the model struggling to generalize physical interactions in some chemical spaces.

8.2 Outlook

This work establishes proof of concept for the use of generative models in materials discovery, however, several bottlenecks remain to move from computational prediction to experimental reality.

Bridging the Modality Gap

The efficiency of CCIP suggests it captures valid chemical interactions. It, however, still lags behind the static Word2Vec embeddings in the generative models. Future work should investigate the improvement of this novel embedding approach.

Experimental Validation

The immediate next step is the physical validation of the flagged candidates from the Materials Funnel. The specific ternary oxide systems identified both as highly synthesizable by the Synthesizability-stoi-CGNF model and interesting from the point of view of our model, should be synthesized via ink-jet printing. Comparing the images of these physical samples with their LDM-generated counterparts is going to provide the ultimate validation of the model's predictive power on truly unseen samples.

Refining Loss Functions and Metrics

In the SpectralPredictor the current reliance on MSE and R^2 makes the model sensitive to intensity shifts (vertical offsets) even when the spectral features (peaks and valleys) are correctly predicted. Future work should implement loss function that better mitigate this variation in the data.

Active Learning for Outlier Mitigation

The analysis of the ternary plots revealed that outliers are not random but frequently cluster in specific chemical regions, potentially differing in a meaningful way from the learned correlations. This suggests a 'knowledge gap' in the model's latent space. An active learning loop could be implemented where:

1. The model estimates its uncertainty, using Monte Carlo Dropout or Ensemble methods for example.
2. The system suggests compositions in high-uncertainty regions for synthesis and characterization.
3. These new data points are fed back into the training set to specifically target the "blind spots" in the material space.

This approach would increase the weight put on the 'more difficult' cases.

Bibliography

- [1] Google Gemini. Gemini. URL: <https://gemini.google.com> (visited on 01/12/2026).
- [2] Grammarly: Free AI Writing Assistance. URL: <https://www.grammarly.com/> (visited on 01/12/2026).
- [3] Google Translate. URL: <https://translate.google.pl/?sl=de&tl=en&op=translate> (visited on 01/12/2026).
- [4] Mary Meeker et al. *Trends – Artificial Intelligence*. May 30, 2025. URL: <https://www.bondcap.com/report/tai/> (visited on 10/20/2025).
- [5] Lusann Yang et al. “Discovery of complex oxides via automated experiments and data science”. In: *Proceedings of the National Academy of Sciences* 118.37 (Sept. 14, 2021). Publisher: Proceedings of the National Academy of Sciences, e2106042118. DOI: 10.1073/pnas.2106042118. URL: <https://www.pnas.org/doi/10.1073/pnas.2106042118> (visited on 10/20/2025).
- [6] Tian Xie. “Crystal Graph Convolutional Neural Networks for an Accurate and Interpretable Prediction of Material Properties”. In: *Physical Review Letters* 120.14 (2018). DOI: 10.1103/PhysRevLett.120.145301.
- [7] Jörg Behler. “Generalized Neural-Network Representation of High-Dimensional Potential-Energy Surfaces”. In: *Physical Review Letters* 98.14 (2007). DOI: 10.1103/PhysRevLett.98.146401.
- [8] Juan Carrasquilla and Roger G. Melko. “Machine learning phases of matter”. In: *Nature Physics* 13.5 (May 2017), pp. 431–434. ISSN: 1745-2473, 1745-2481. DOI: 10.1038/nphys4035. arXiv: 1605.01735[cond-mat]. URL: <http://arxiv.org/abs/1605.01735> (visited on 10/20/2025).
- [9] P. Baldi, P. Sadowski, and D. Whiteson. “Searching for exotic particles in high-energy physics with deep learning”. In: *Nature Communications* 5.1 (July 2, 2014). Publisher: Nature Publishing Group, p. 4308. ISSN: 2041-1723. DOI: 10.1038/ncomms5308. URL: <https://www.nature.com/articles/ncomms5308> (visited on 10/20/2025).
- [10] Javier Duarte and Jean-Roch Vlimant. “Graph Neural Networks for Particle Tracking and Reconstruction”. In: Mar. 2022, pp. 387–436. DOI: 10.1142/9789811234033_0012. arXiv: 2012.01249[hep-ph]. URL: <http://arxiv.org/abs/2012.01249> (visited on 10/20/2025).
- [11] Samuel Bein et al. “Refining fast simulation using machine learning”. In: *EPJ Web of Conferences* 295 (2024), p. 09032. ISSN: 2100-014X. DOI: 10.1051/epjconf/202429509032. arXiv: 2309.12919[physics]. URL: <http://arxiv.org/abs/2309.12919> (visited on 10/20/2025).

- [12] Jia-Ming Dai and Jizhou Tong. “Galaxy Morphology Classification with Deep Convolutional Neural Networks”. In: *Astrophysics and Space Science* 364.4 (Apr. 2019), p. 55. ISSN: 0004-640X, 1572-946X. DOI: 10.1007/s10509-019-3540-1. arXiv: 1807.10406 [astro-ph]. URL: <http://arxiv.org/abs/1807.10406> (visited on 10/20/2025).
- [13] Daniel George and E. A. Huerta. “Deep Learning for Real-time Gravitational Wave Detection and Parameter Estimation: Results with Advanced LIGO Data”. In: *Physics Letters B* 778 (Mar. 2018), pp. 64–70. ISSN: 03702693. DOI: 10.1016/j.physletb.2017.12.053. arXiv: 1711.03121 [gr-qc]. URL: <http://arxiv.org/abs/1711.03121> (visited on 10/20/2025).
- [14] Zhiwei Min et al. “Deep learning for cosmological parameter inference from a dark matter halo density field”. In: *Physical Review D* 110.6 (Sept. 16, 2024), p. 063531. ISSN: 2470-0010, 2470-0029. DOI: 10.1103/PhysRevD.110.063531. arXiv: 2404.09483 [astro-ph]. URL: <http://arxiv.org/abs/2404.09483> (visited on 10/20/2025).
- [15] John Jumper et al. “Highly accurate protein structure prediction with AlphaFold”. In: *Nature* 596.7873 (Aug. 2021). Publisher: Nature Publishing Group, pp. 583–589. ISSN: 1476-4687. DOI: 10.1038/s41586-021-03819-2. URL: <https://www.nature.com/articles/s41586-021-03819-2> (visited on 12/11/2024).
- [16] Josh Abramson et al. “Accurate structure prediction of biomolecular interactions with AlphaFold 3”. In: *Nature* 630.8016 (June 2024). Publisher: Nature Publishing Group, pp. 493–500. ISSN: 1476-4687. DOI: 10.1038/s41586-024-07487-w. URL: <https://www.nature.com/articles/s41586-024-07487-w> (visited on 10/20/2025).
- [17] Fábio J. N. Ferreira and Agnaldo S. Carneiro. “AI-Driven Drug Discovery: A Comprehensive Review”. In: *ACS Omega* 10.23 (), pp. 23889–23903. ISSN: 2470-1343. DOI: 10.1021/acsomega.5c00549. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12177741/> (visited on 10/20/2025).
- [18] J. G. Bednorz and K. A. Müller. “Possible highT_c superconductivity in the Ba–La–Cu–O system”. In: *Zeitschrift für Physik B Condensed Matter* 64.2 (June 1, 1986), pp. 189–193. ISSN: 1431-584X. DOI: 10.1007/BF01303701. URL: <https://doi.org/10.1007/BF01303701> (visited on 10/21/2025).
- [19] P. Hohenberg and W. Kohn. “Inhomogeneous Electron Gas”. In: *Physical Review* 136.3 (Nov. 9, 1964). Publisher: American Physical Society, B864–B871. DOI: 10.1103/PhysRev.136.B864. URL: <https://link.aps.org/doi/10.1103/PhysRev.136.B864> (visited on 01/10/2026).
- [20] W. Kohn and L. J. Sham. “Self-Consistent Equations Including Exchange and Correlation Effects”. In: *Physical Review* 140.4 (Nov. 15, 1965). Publisher: American Physical Society, A1133–A1138. DOI: 10.1103/PhysRev.140.A1133. URL: <https://link.aps.org/doi/10.1103/PhysRev.140.A1133> (visited on 01/10/2026).
- [21] G. Kresse and J. Furthmüller. “Efficient iterative schemes for *ab initio* total-energy calculations using a plane-wave basis set”. In: *Physical Review B* 54.16 (Oct. 15, 1996), pp. 11169–11186. ISSN: 0163-1829, 1095-3795. DOI: 10.1103/PhysRevB.54.11169. URL: <https://link.aps.org/doi/10.1103/PhysRevB.54.11169> (visited on 01/10/2026).
- [22] Paolo Giannozzi et al. “QUANTUM ESPRESSO: a modular and open-source software project for quantum simulations of materials”. In: *Journal of Physics: Condensed Matter* 21.39 (Sept. 2009), p. 395502. ISSN: 0953-8984. DOI: 10.1088/0953-8984/21/39/395502. URL: <https://doi.org/10.1088/0953-8984/21/39/395502> (visited on 01/10/2026).

- [23] Anubhav Jain et al. "Commentary: The Materials Project: A materials genome approach to accelerating materials innovation". In: *APL Materials* 1.1 (July 1, 2013), p. 011002. ISSN: 2166-532X. DOI: 10.1063/1.4812323. URL: <https://pubs.aip.org/apm/article/1/1/011002/119685/Commentary-The-Materials-Project-A-materials> (visited on 01/10/2026).
- [24] David Silver et al. "Mastering the game of Go with deep neural networks and tree search". In: *Nature* 529.7587 (Jan. 2016). Publisher: Nature Publishing Group, pp. 484–489. ISSN: 1476-4687. DOI: 10.1038/nature16961. URL: <https://www.nature.com/articles/nature16961> (visited on 10/23/2025).
- [25] admin. *AN EMPIRICAL SCIENCE RESEARCH ON BIOINFORMATICS IN MACHINE LEARNING – JOURNAL OF MECHANICS OF CONTINUA AND MATHEMATICAL SCIENCES*. URL: https://www.journalimcms.org/special_issue/an-empirical-science-research-on-bioinformatics-in-machine-learning/ (visited on 10/22/2025).
- [26] Ian Goodfellow, Aaron Courville, and Yoshua Bengio. *Deep learning*. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, 2016. 1 p. ISBN: 978-0-262-33737-3.
- [27] Tom M. Mitchell. *Machine learning*. Nachdr. McGraw-Hill series in Computer Science. New York: McGraw-Hill, 2013. 414 pp. ISBN: 978-0-07-042807-2 978-0-07-115467-3.
- [28] R. A. Fisher. "The Use of Multiple Measurements in Taxonomic Problems". In: *Annals of Eugenics* 7.2 (1936). _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1469-1809.1936.tb02137.x>, pp. 179–188. ISSN: 2050-1439. DOI: 10.1111/j.1469-1809.1936.tb02137.x. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1469-1809.1936.tb02137.x> (visited on 10/26/2025).
- [29] Li Deng. "The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web]". In: *IEEE Signal Processing Magazine* 29.6 (Nov. 2012), pp. 141–142. ISSN: 1558-0792. DOI: 10.1109/MSP.2012.2211477. URL: <https://ieeexplore.ieee.org/document/6296535> (visited on 10/26/2025).
- [30] Tony Jebara. *Machine Learning*. Boston, MA: Springer US, 2004. ISBN: 978-1-4613-4756-9 978-1-4419-9011-2. DOI: 10.1007/978-1-4419-9011-2. URL: <http://link.springer.com/10.1007/978-1-4419-9011-2> (visited on 10/27/2025).
- [31] Calvin Luo. *Understanding Diffusion Models: A Unified Perspective*. Aug. 25, 2022. DOI: 10.48550/arXiv.2208.11970. arXiv: 2208.11970[cs]. URL: <http://arxiv.org/abs/2208.11970> (visited on 10/27/2025).
- [32] Jascha Sohl-Dickstein et al. "Deep Unsupervised Learning using Nonequilibrium Thermodynamics". In: *Proceedings of the 32nd International Conference on Machine Learning*. International Conference on Machine Learning. ISSN: 1938-7228. PMLR, June 1, 2015, pp. 2256–2265. URL: <https://proceedings.mlr.press/v37/sohl-dickstein15.html> (visited on 11/03/2025).
- [33] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. Dec. 16, 2020. DOI: 10.48550/arXiv.2006.11239. arXiv: 2006.11239[cs]. URL: <http://arxiv.org/abs/2006.11239> (visited on 11/04/2025).
- [34] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. "U-Net: Convolutional Networks for Biomedical Image Segmentation". In: *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*. Ed. by Nassir Navab et al. Cham: Springer International Publishing, 2015, pp. 234–241. ISBN: 978-3-319-24574-4. DOI: 10.1007/978-3-319-24574-4_28.

- [35] Ashish Vaswani et al. *Attention Is All You Need*. Issue: arXiv:1706.03762. Aug. 2, 2023. doi: 10.48550/arXiv.1706.03762. arXiv: 1706.03762[cs]. URL: <http://arxiv.org/abs/1706.03762> (visited on 12/04/2024).
- [36] Kaiming He et al. *Deep Residual Learning for Image Recognition*. Dec. 10, 2015. doi: 10.48550/arXiv.1512.03385. arXiv: 1512.03385[cs]. URL: <http://arxiv.org/abs/1512.03385> (visited on 11/18/2025).
- [37] Alex Nichol and Prafulla Dhariwal. *Improved Denoising Diffusion Probabilistic Models*. Feb. 18, 2021. doi: 10.48550/arXiv.2102.09672. arXiv: 2102.09672[cs]. URL: <http://arxiv.org/abs/2102.09672> (visited on 11/27/2025).
- [38] Allan Jabri, David Fleet, and Ting Chen. *Scalable Adaptive Computation for Iterative Generation*. June 14, 2023. doi: 10.48550/arXiv.2212.11972. arXiv: 2212.11972[cs]. URL: <http://arxiv.org/abs/2212.11972> (visited on 11/27/2025).
- [39] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. Apr. 13, 2022. doi: 10.48550/arXiv.2112.10752. arXiv: 2112.10752[cs]. URL: <http://arxiv.org/abs/2112.10752> (visited on 11/19/2025).
- [40] Tomas Mikolov et al. *Efficient Estimation of Word Representations in Vector Space*. Sept. 7, 2013. doi: 10.48550/arXiv.1301.3781. arXiv: 1301.3781[cs]. URL: <http://arxiv.org/abs/1301.3781> (visited on 11/25/2025).
- [41] Alec Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. Feb. 26, 2021. doi: 10.48550/arXiv.2103.00020. arXiv: 2103.00020[cs]. URL: <http://arxiv.org/abs/2103.00020> (visited on 07/03/2025).
- [42] Taku Kudo and John Richardson. *SentencePiece: A simple and language independent sub-word tokenizer and detokenizer for Neural Text Processing*. Aug. 19, 2018. doi: 10.48550/arXiv.1808.06226. arXiv: 1808.06226[cs]. URL: <http://arxiv.org/abs/1808.06226> (visited on 11/25/2025).
- [43] Rico Sennrich, Barry Haddow, and Alexandra Birch. *Neural Machine Translation of Rare Words with Subword Units*. June 10, 2016. doi: 10.48550/arXiv.1508.07909. arXiv: 1508.07909[cs]. URL: <http://arxiv.org/abs/1508.07909> (visited on 11/25/2025).
- [44] Xinying Song et al. *Fast WordPiece Tokenization*. Oct. 5, 2021. doi: 10.48550/arXiv.2012.15524. arXiv: 2012.15524[cs]. URL: <http://arxiv.org/abs/2012.15524> (visited on 11/25/2025).
- [45] Matthew E. Peters et al. *Deep contextualized word representations*. Mar. 22, 2018. doi: 10.48550/arXiv.1802.05365. arXiv: 1802.05365[cs]. URL: <http://arxiv.org/abs/1802.05365> (visited on 11/25/2025).
- [46] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. May 24, 2019. doi: 10.48550/arXiv.1810.04805. arXiv: 1810.04805[cs]. URL: <http://arxiv.org/abs/1810.04805> (visited on 11/25/2025).
- [47] Jonas Gehring et al. *Convolutional Sequence to Sequence Learning*. July 25, 2017. doi: 10.48550/arXiv.1705.03122. arXiv: 1705.03122[cs]. URL: <http://arxiv.org/abs/1705.03122> (visited on 11/28/2025).
- [48] Helge S. Stein et al. "Machine learning of optical properties of materials – predicting spectra from images and images from spectra". In: *Chemical Science* 10.1 (Dec. 19, 2018). Number: 1 Publisher: The Royal Society of Chemistry, pp. 47–55. ISSN: 2041-6539. doi: 10.1039/C8SC03077D. URL: <https://pubs.rsc.org/en/content/articlelanding/2019/sc/c8sc03077d> (visited on 11/09/2024).
- [49] John Gregoire et al. *HDF5 file containing JCAP images and absorption spectra for 179072 metal oxides*. Oct. 29, 2018. URL: <https://data.caltech.edu/records/km8xg-1yd31> (visited on 11/29/2025).

- [50] John Gregoire and Edwin Soedarmadji. *Summary tables for 2019 release of JCAP Materials Experiment and Analysis Database*. July 10, 2019. URL: <https://data.caltech.edu/records/yrn11-jb916> (visited on 12/02/2025).
- [51] J. Tauc, R. Grigorovici, and A. Vancu. "Optical Properties and Electronic Structure of Amorphous Germanium". In: *Physica Status Solidi B Basic Research* 15 (Jan. 1, 1966). ADS Bibcode: 1966PSSBR..15..627T, pp. 627–637. ISSN: 0370-1972. DOI: 10.1002/pssb.19660150224. URL: <https://ui.adsabs.harvard.edu/abs/1966PSSBR..15..627T> (visited on 12/05/2025).
- [52] Santosh K. Suram, Paul F. Newhouse, and John M. Gregoire. "High Throughput Light Absorber Discovery, Part 1: An Algorithm for Automated Tauc Analysis". In: *ACS Combinatorial Science* 18.11 (Nov. 14, 2016). Publisher: American Chemical Society, pp. 673–681. ISSN: 2156-8952. DOI: 10.1021/acscombsci.6b00053. URL: <https://doi.org/10.1021/acscombsci.6b00053> (visited on 12/05/2025).
- [53] Abraham. Savitzky and M. J. E. Golay. "Smoothing and Differentiation of Data by Simplified Least Squares Procedures." In: *Analytical Chemistry* 36.8 (July 1, 1964). Publisher: American Chemical Society, pp. 1627–1639. ISSN: 0003-2700. DOI: 10.1021/ac60214a047. URL: <https://doi.org/10.1021/ac60214a047> (visited on 12/08/2025).
- [54] Chi Chen et al. "Graph Networks as a Universal Machine Learning Framework for Molecules and Crystals". In: *Chemistry of Materials* 31.9 (May 14, 2019). Publisher: American Chemical Society, pp. 3564–3572. ISSN: 0897-4756. DOI: 10.1021/acs.chemmater.9b01294. URL: <https://doi.org/10.1021/acs.chemmater.9b01294> (visited on 12/28/2025).
- [55] Dipendra Jha et al. "ElemNet: Deep Learning the Chemistry of Materials From Only Elemental Composition". In: *Scientific Reports* 8.1 (Dec. 4, 2018). Publisher: Nature Publishing Group, p. 17593. ISSN: 2045-2322. DOI: 10.1038/s41598-018-35934-y. URL: <https://www.nature.com/articles/s41598-018-35934-y> (visited on 12/30/2025).
- [56] Vahe Tshitoyan et al. "Unsupervised word embeddings capture latent knowledge from materials science literature". In: *Nature* 571.7763 (July 2019). Publisher: Nature Publishing Group, pp. 95–98. ISSN: 1476-4687. DOI: 10.1038/s41586-019-1335-8. URL: <https://www.nature.com/articles/s41586-019-1335-8> (visited on 12/30/2025).
- [57] Takuya Akiba et al. *Optuna: A Next-generation Hyperparameter Optimization Framework*. July 25, 2019. DOI: 10.48550/arXiv.1907.10902. arXiv: 1907.10902[cs]. URL: <http://arxiv.org/abs/1907.10902> (visited on 01/09/2026).
- [58] *Improvement to industrial colour-difference evaluation*. OCLC: 223775480. Vienna, Austria: CIE Central Bureau, 2001. ISBN: 978-3-901906-08-4.
- [59] Yossi Rubner, Carlo Tomasi, and Leonidas J. Guibas. "The Earth Mover's Distance as a Metric for Image Retrieval". In: *International Journal of Computer Vision* 40.2 (Nov. 1, 2000), pp. 99–121. ISSN: 1573-1405. DOI: 10.1023/A:1026543900054. URL: <https://doi.org/10.1023/A:1026543900054> (visited on 01/07/2026).
- [60] Zhou Wang et al. "Image quality assessment: from error visibility to structural similarity". In: *IEEE Transactions on Image Processing* 13.4 (Apr. 2004), pp. 600–612. ISSN: 1941-0042. DOI: 10.1109/TIP.2003.819861. URL: <https://ieeexplore.ieee.org/document/1284395> (visited on 01/07/2026).
- [61] Yunxiang Li et al. *SAMScore: A Content Structural Similarity Metric for Image Translation Evaluation*. version: 2. Jan. 24, 2025. DOI: 10.48550/arXiv.2305.15367. arXiv: 2305.15367[cs]. URL: <http://arxiv.org/abs/2305.15367> (visited on 07/28/2025).

- [62] Alexander Kirillov et al. *Segment Anything*. Apr. 5, 2023. doi: 10.48550/arXiv.2304.02643. arXiv: 2304.02643[cs]. URL: <http://arxiv.org/abs/2304.02643> (visited on 01/07/2026).
- [63] *The Unreasonable Effectiveness of Deep Features as a Perceptual Metric*. In *CVPR, 2018*. URL: <https://richzhang.github.io/PerceptualSimilarity/> (visited on 01/07/2026).
- [64] Ilya Loshchilov and Frank Hutter. *Decoupled Weight Decay Regularization*. Jan. 4, 2019. doi: 10.48550/arXiv.1711.05101. arXiv: 1711.05101[cs]. URL: <http://arxiv.org/abs/1711.05101> (visited on 12/27/2025).

Additional Materials

A.1 Jensen's Inequality

Jensen's inequality is a fundamental result in convex analysis and probability theory.

Definition A.1.1 *Let X be a random variable and let $\phi : \mathbb{R} \rightarrow \mathbb{R}$ be a convex function. Then Jensen's inequality states that*

$$\phi(\mathbb{E}[X]) \leq \mathbb{E}[\phi(X)]. \quad (\text{A.1})$$

If ϕ is concave, the inequality is reversed:

$$\phi(\mathbb{E}[X]) \geq \mathbb{E}[\phi(X)]. \quad (\text{A.2})$$

Intuition. For a convex function, the function value at the mean of X is always less than or equal to the mean of the function values. Geometrically, this expresses the fact that a convex function lies below any of its chords.

Example. Consider $\phi(x) = x^2$, which is convex. For a random variable X , Jensen's inequality gives

$$(\mathbb{E}[X])^2 \leq \mathbb{E}[X^2]. \quad (\text{A.3})$$

This is consistent with the non-negativity of the variance:

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \geq 0. \quad (\text{A.4})$$

Applications. Jensen's inequality appears frequently in statistics, information theory, and machine learning. For example:

- In the derivation of the Evidence Lower Bound (ELBO) in variational inference.
- In proving properties of entropy and the Kullback–Leibler divergence.
- In bounding expectations and moments of random variables.

A.2 Distribution of a Sum of Independent Gaussian Variables

A significant property of the normal (or Gaussian) distribution is its **stability under addition**. This means that when two **independent** random variables, X and Y , both follow normal distributions, their sum, $Z = X + Y$, is also normally distributed.

Formally, if we have:

$$X \sim \mathcal{N}(\mu_X, \sigma_X^2) \quad \text{and} \quad Y \sim \mathcal{N}(\mu_Y, \sigma_Y^2)$$

Then their sum $Z = X + Y$ is distributed as:

$$Z \sim \mathcal{N}(\mu_X + \mu_Y, \sigma_X^2 + \sigma_Y^2)$$

As the formula indicates, the mean of the resulting distribution is the sum of the individual means. Similarly, the variance of the sum is the sum of the individual variances.

The **independence** of X and Y is a critical assumption for this specific result. It is worth noting that the linearity of expectation ($E[Z] = E[X] + E[Y]$) *always* holds, so the mean of the sum is $\mu_X + \mu_Y$ regardless of independence. However, the rule for adding variances ($\text{Var}(Z) = \text{Var}(X) + \text{Var}(Y)$) requires X and Y to be **uncorrelated**, a condition which is guaranteed by independence.

Derivation using Characteristic Functions

We can prove this result efficiently using **characteristic functions (CFs)**. A fundamental property of CFs states that for two independent random variables, the characteristic function of their sum is the product of their respective characteristic functions:

$$\varphi_{X+Y}(t) = \mathbb{E}(e^{it(X+Y)}) = \mathbb{E}(e^{itX}) \mathbb{E}(e^{itY}) = \varphi_X(t) \varphi_Y(t)$$

We first recall the standard form of the characteristic function for a normal distribution $\mathcal{N}(\mu, \sigma^2)$:

$$\varphi(t) = \exp\left(it\mu - \frac{\sigma^2 t^2}{2}\right)$$

Now, we can apply this to find the CF of $Z = X + Y$:

$$\begin{aligned} \varphi_{X+Y}(t) &= \varphi_X(t) \varphi_Y(t) \\ &= \exp\left(it\mu_X - \frac{\sigma_X^2 t^2}{2}\right) \exp\left(it\mu_Y - \frac{\sigma_Y^2 t^2}{2}\right) \\ &= \exp\left(\left(it\mu_X - \frac{\sigma_X^2 t^2}{2}\right) + \left(it\mu_Y - \frac{\sigma_Y^2 t^2}{2}\right)\right) \\ &= \exp\left(it(\mu_X + \mu_Y) - \frac{(\sigma_X^2 + \sigma_Y^2)t^2}{2}\right) \end{aligned}$$

By inspection, this final expression is itself the characteristic function of a normal distribution.

Due to the **uniqueness property** of characteristic functions – which states that a probability distribution is uniquely determined by its CF – we can conclude that the distribution of $X + Y$ must be a normal distribution with mean $\mu_X + \mu_Y$ and variance $\sigma_X^2 + \sigma_Y^2$.

Algorithms

B.1 Bandgap Extraction Using Tauc Plot

The Tauc plot is performed by first transforming the UV-Vis absorption spectrum according to the Tauc relation: $(E)^{(1/n)}$ versus photon energy E , where α is the absorption coefficient and n depends on the transition type ($n = 2$ for direct transitions, $n = 1/2$ for indirect transitions). To estimate the bandgap, an improved derivative-based method is employed rather than traditional manual linear extrapolation. First, the Tauc-transformed data is smoothed using a Savitzky-Golay filter (15-point window, 3rd order polynomial) to reduce noise. The first derivative of the smoothed Tauc plot is then calculated to identify the absorption edge – the energy at which the slope is maximum, which physically corresponds to the onset of fundamental absorption. A linear fit is performed only within a narrow window (± 0.2 eV) around this maximum slope point, ensuring that the fit captures the truly linear region of the absorption edge while excluding non-linear regions at lower and higher energies. Finally, the bandgap is determined by extrapolating this localized linear fit to the x-axis intercept (where $y = 0$). This automated, derivative-based approach is more robust and reproducible than traditional Tauc plot analysis because it objectively identifies the absorption onset and is less sensitive to baseline effects and the subjective choice of fitting range. The pseudocode can be found in Algorithm 4.

Algorithm 4 Tauc Plot Bandgap Estimation using Derivative Method

Require: Energy array E , Absorption spectrum α , transition type n (default: $n = 0.5$ for indirect)

Ensure: Estimated bandgap E_g , absorption edge position E_{edge}

```

1: Step 1: Calculate Tauc transformation
2: for  $i = 1$  to  $\text{length}(E)$  do
3:   if  $\alpha[i] > 0$  and  $E[i] > 0$  then
4:      $\text{Tauc}[i] = (\alpha[i] \cdot E[i])^{1/n}$ 
5:   else
6:      $\text{Tauc}[i] = 0$ 
7:   end if
8: end for
9: Step 2: Smooth the Tauc data
10:  $\text{Tauc}_{\text{smooth}} = \text{Savitzky-Golay filter}(\text{Tauc}, \text{window} = 15, \text{order} = 3)$ 
11: Step 3: Calculate first derivative
12:  $\frac{d\text{Tauc}}{dE} = \nabla(\text{Tauc}_{\text{smooth}}, E)$ 
13: Step 4: Find absorption edge (maximum slope position)
14:  $\text{valid\_region} = \{i : E[i] > 1.5 \text{ eV}\}$ 
15:  $\left. \frac{d\text{Tauc}}{dE} \right|_{\text{valid}} = \frac{d\text{Tauc}}{dE}[\text{valid\_region}]$ 
16:  $E_{\text{valid}} = E[\text{valid\_region}]$ 
17:  $i_{\text{max}} = \arg \max \left( \left. \frac{d\text{Tauc}}{dE} \right|_{\text{valid}} \right)$ 
18:  $E_{\text{edge}} = E_{\text{valid}}[i_{\text{max}}]$ 
19: Step 5: Define fitting window around steepest point
20:  $w = 0.2$  ▷ Window size in eV
21:  $\text{fit\_mask} = \{i : |E[i] - E_{\text{edge}}| \leq w\}$ 
22: Step 6: Perform linear regression in window
23: if  $\text{count}(\text{fit\_mask}) \geq 3$  then
24:    $m, b = \text{LinearRegression}(E[\text{fit\_mask}], \text{Tauc}_{\text{smooth}}[\text{fit\_mask}])$ 
25: ▷ Fit:  $y = mx + b$ 
26: else
27:   return NaN,  $E_{\text{edge}}$ 

```

B.2 AdamW optimizer

AdamW is an optimizer in which there is no accumulation of the weight decay in this work the implementation from pytorch was used, which follows [64]. The algorithm can be seen in Algorithm 5. The difference between this optimizer and the standard Adam optimizer is that in this case the weight decay step is performed separately and directly on the weights before the main update, compared to gradient update in the standard case. This fixes the issue of worse regularization in the standard Adam compared to SGD. Additionally in this algorithm the maximum second moment is used for the calculation of the adaptive learning rate – in the standard Adam, the current second moment is used rather than the maximum one.

```

28: end if
29: Step 7: Calculate bandgap (x-intercept)
30: if  $|m| > 10^{-10}$  then
31:    $E_g = -b/m$  ▷ When  $y = 0$ :  $E_g = -b/m$ 
32:   if  $E_g > 0$  then
33:     return  $E_g, E_{\text{edge}}$ 
34:   else
35:     return NaN,  $E_{\text{edge}}$ 
36:   end if
37: else
38:   return NaN,  $E_{\text{edge}}$ 
39: end if

```

Algorithm 5 Adam with Weight Decay and AMSGrad

```

1: input :  $\gamma(\text{lr}), \beta_1, \beta_2(\text{betas}), \theta_0(\text{params}), f(\theta)(\text{objective}), \epsilon(\text{epsilon})$ 
2:    $\lambda(\text{weight decay}), \text{amsgrad}, \text{maximize}$ 
3: initialize :  $m_0 \leftarrow 0$  (first moment),  $v_0 \leftarrow 0$  (second moment),  $v_0^{\text{max}} \leftarrow 0$ 

4: for  $t = 1$  to  $\dots$  do
5:   if maximize then
6:      $g_t \leftarrow -\nabla_{\theta} f_t(\theta_{t-1})$ 
7:   else
8:      $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ 
9:   end if
10:   $\theta_t \leftarrow \theta_{t-1} - \gamma \lambda \theta_{t-1}$ 
11:   $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
12:   $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
13:   $\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ 
14:  if amsgrad then
15:     $v_t^{\text{max}} \leftarrow \max(v_{t-1}^{\text{max}}, v_t)$ 
16:     $\widehat{v}_t \leftarrow v_t^{\text{max}} / (1 - \beta_2^t)$ 
17:  else
18:     $\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
19:  end if
20:   $\theta_t \leftarrow \theta_t - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ 
21: end for
22: return  $\theta_t$ 

```

B.3 CIEDE2000 Color Difference Formula

The CIEDE2000 color difference formula (ΔE_{00}) represents the current state-of-the-art in color difference measurement, addressing limitations in earlier formulas such as CIE76 (ΔE_{ab}^*) and CIE94 (ΔE_{94}^*). This appendix provides a complete description of the CIEDE2000 algorithm.

Algorithm 6 CIEDE2000 Color Difference Algorithm

Require: Two colors in CIELAB space: (L_1^*, a_1^*, b_1^*) and (L_2^*, a_2^*, b_2^*)

Ensure: Color difference ΔE_{00}

```

1: // Step 1: Compute mean lightness and chroma
2:  $\bar{L}' \leftarrow \frac{L_1^* + L_2^*}{2}$ 
3:  $C_1^* \leftarrow \sqrt{a_1^{*2} + b_1^{*2}}, \quad C_2^* \leftarrow \sqrt{a_2^{*2} + b_2^{*2}}$ 
4:  $\bar{C}^* \leftarrow \frac{C_1^* + C_2^*}{2}$ 
5: // Step 2: Apply  $a^*$  correction for blue region
6:  $G \leftarrow 0.5 \left( 1 - \sqrt{\frac{\bar{C}^{*7}}{\bar{C}^{*7} + 25^7}} \right)$ 
7:  $a_1' \leftarrow (1 + G) \cdot a_1^*, \quad a_2' \leftarrow (1 + G) \cdot a_2^*$ 
8: // Step 3: Compute new chroma and hue
9:  $C_1' \leftarrow \sqrt{a_1'^2 + b_1^{*2}}, \quad C_2' \leftarrow \sqrt{a_2'^2 + b_2^{*2}}$ 
10:  $h_1' \leftarrow \text{atan2}(b_1^*, a_1'), \quad h_2' \leftarrow \text{atan2}(b_2^*, a_2')$  ▷ Convert to  $[0^\circ, 360^\circ]$ 
11:  $\bar{C}' \leftarrow \frac{C_1' + C_2'}{2}$ 
12:  $\bar{h}' \leftarrow \text{CircularMean}(h_1', h_2')$  ▷ Account for  $0^\circ/360^\circ$  wrap
13: // Step 4: Compute differences
14:  $\Delta L' \leftarrow L_2^* - L_1^*$ 
15:  $\Delta C' \leftarrow C_2' - C_1'$ 
16:  $\Delta h' \leftarrow \text{CircularDiff}(h_2', h_1')$  ▷ Shortest arc distance
17:  $\Delta H' \leftarrow 2\sqrt{C_1' C_2'} \sin(\Delta h' / 2)$ 
18: // Step 5: Compute weighting functions
19:  $S_L \leftarrow 1 + \frac{0.015(\bar{L}' - 50)^2}{\sqrt{20 + (\bar{L}' - 50)^2}}$ 
20:  $S_C \leftarrow 1 + 0.045\bar{C}'$ 
21:  $T \leftarrow 1 - 0.17 \cos(\bar{h}' - 30^\circ) + 0.24 \cos(2\bar{h}')$ 
22:  $\quad + 0.32 \cos(3\bar{h}' + 6^\circ) - 0.20 \cos(4\bar{h}' - 63^\circ)$ 
23:  $S_H \leftarrow 1 + 0.015\bar{C}' T$ 
24: // Step 6: Rotation term for blue region
25:  $\Delta\theta \leftarrow 30^\circ \exp\left(-\left(\frac{\bar{h}' - 275^\circ}{25^\circ}\right)^2\right)$ 
26:  $R_C \leftarrow 2\sqrt{\frac{\bar{C}'^7}{\bar{C}'^7 + 25^7}}$ 
27:  $R_T \leftarrow -\sin(2\Delta\theta) \cdot R_C$ 
28: // Step 7: Compute final color difference
29:  $k_L \leftarrow 1, \quad k_C \leftarrow 1, \quad k_H \leftarrow 1$  ▷ Reference conditions
30:  $\Delta E_{00} \leftarrow \sqrt{\left(\frac{\Delta L'}{k_L S_L}\right)^2 + \left(\frac{\Delta C'}{k_C S_C}\right)^2 + \left(\frac{\Delta H'}{k_H S_H}\right)^2} + R_T \frac{\Delta C'}{k_C S_C} \frac{\Delta H'}{k_H S_H}$ 
31: return  $\Delta E_{00}$ 

```

Algorithm 7 Helper Functions for CIEDE2000

```

1: function CIRCULARMEAN( $h_1, h_2$ )
2:   if  $|h_1 - h_2| \leq 180^\circ$  then
3:     return  $(h_1 + h_2)/2$ 
4:   else if  $h_1 + h_2 < 360^\circ$  then
5:     return  $(h_1 + h_2 + 360^\circ)/2$ 
6:   else
7:     return  $(h_1 + h_2 - 360^\circ)/2$ 
8:   end if
9: end function
10: function CIRCULARDIFF( $h_2, h_1$ )
11:   if  $C'_1 \cdot C'_2 = 0$  then ▷ Both achromatic
12:     return  $0^\circ$ 
13:   else if  $|h_2 - h_1| \leq 180^\circ$  then
14:     return  $h_2 - h_1$ 
15:   else if  $h_2 \leq h_1$  then
16:     return  $h_2 - h_1 + 360^\circ$ 
17:   else
18:     return  $h_2 - h_1 - 360^\circ$ 
19:   end if
20: end function

```

B.3.1 Interpretation and Thresholds

The thresholds commonly used for interpretation of the results can be found in Table B.1.

Table B.1: CIEDE2000 perceptual thresholds

ΔE_{00}	Interpretation
< 1.0	Just noticeable difference (JND); not perceptible by most observers
$1.0 - 2.0$	Perceptible through close observation; acceptable for most applications
$2.0 - 3.5$	Perceptible at a glance; noticeable difference
$3.5 - 5.0$	Clear difference; may be unacceptable depending on application
$5.0 - 10.0$	Significant difference; colors clearly distinguishable
> 10.0	Very large difference; colors have little perceptual similarity

Additional Results

C.1 The Generative Funnel Fractions

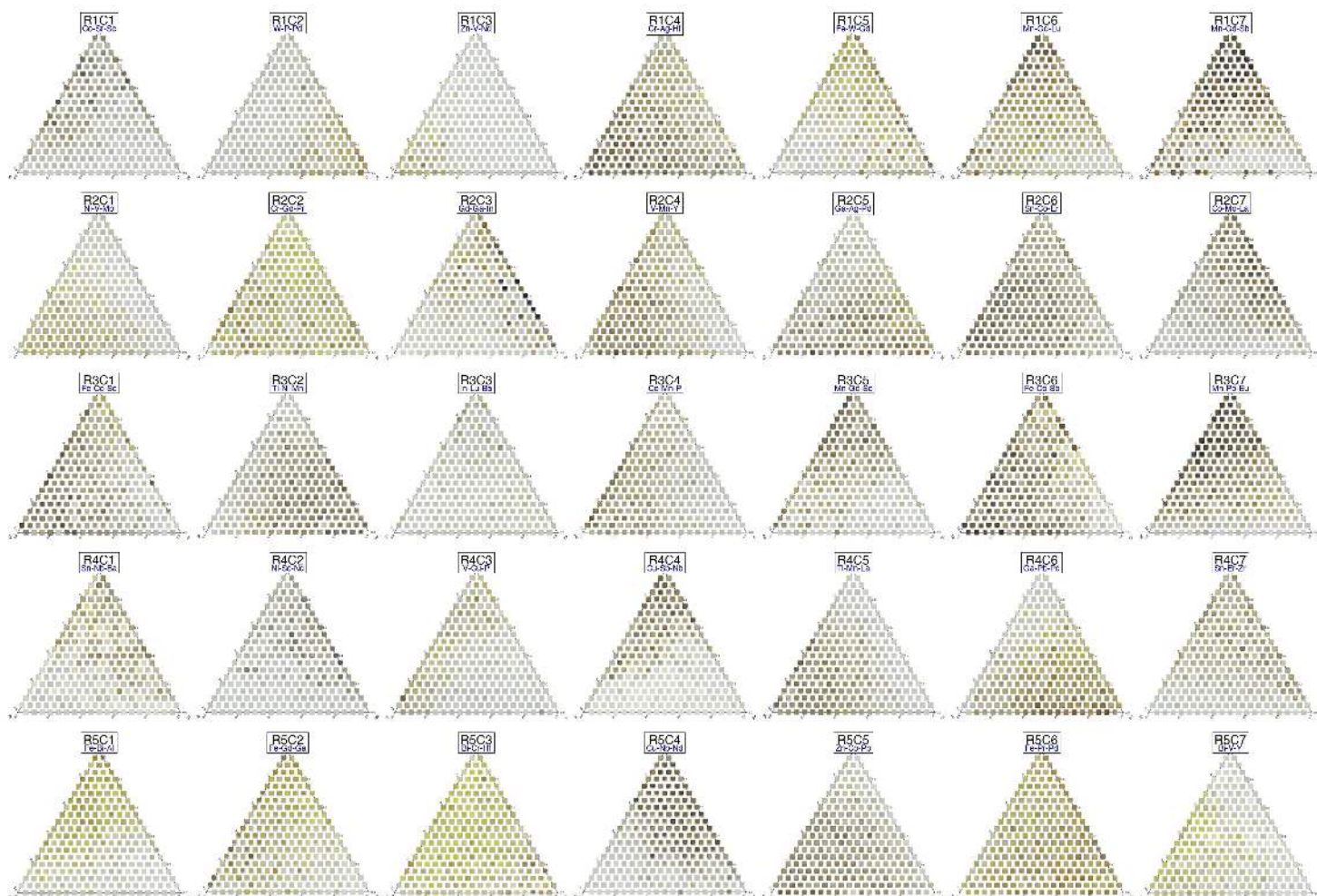
In Table C.1 the fractions that were used for generation of the final images used for phase prediction are shown.

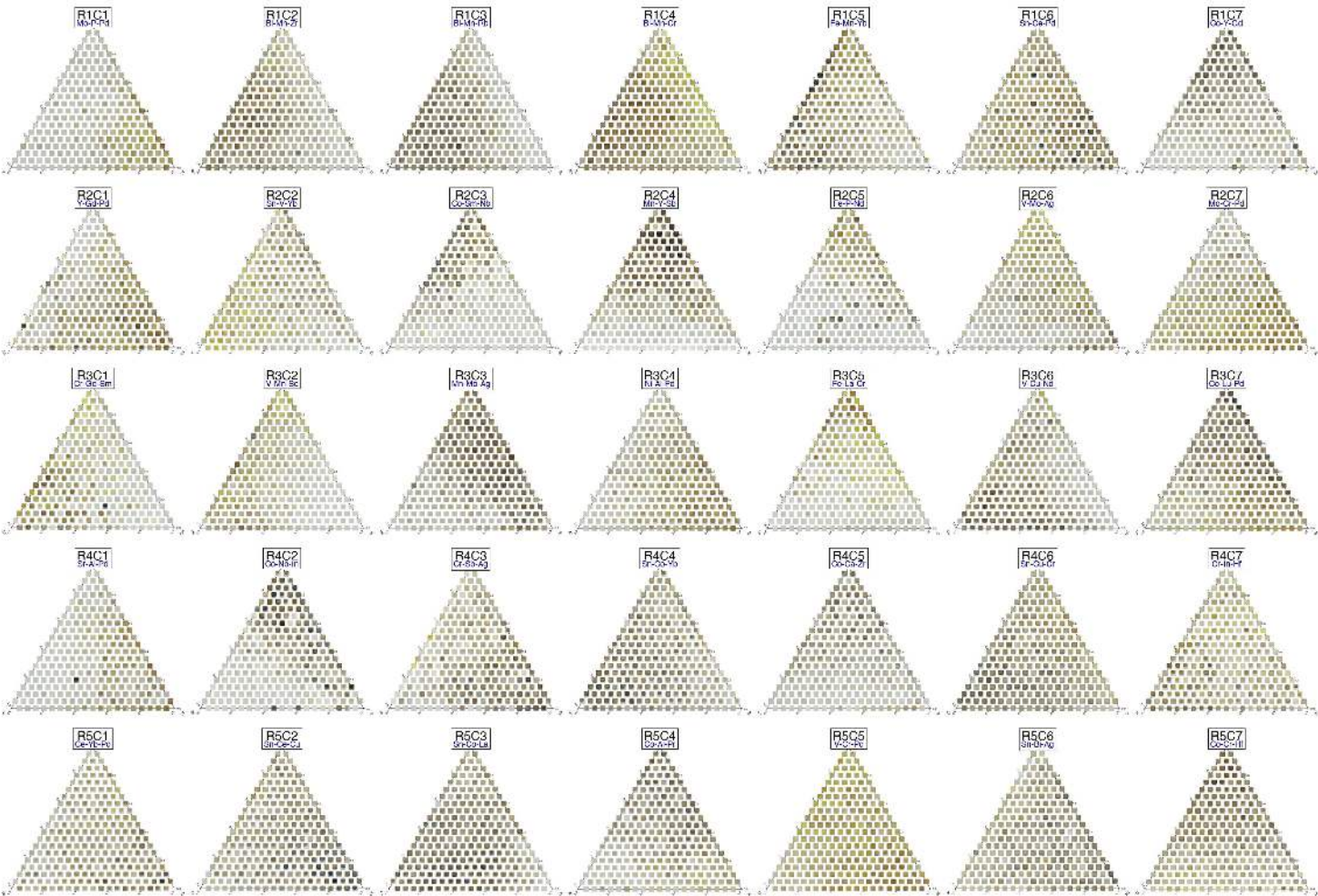
Table C.1: Ternary Composition Fractions

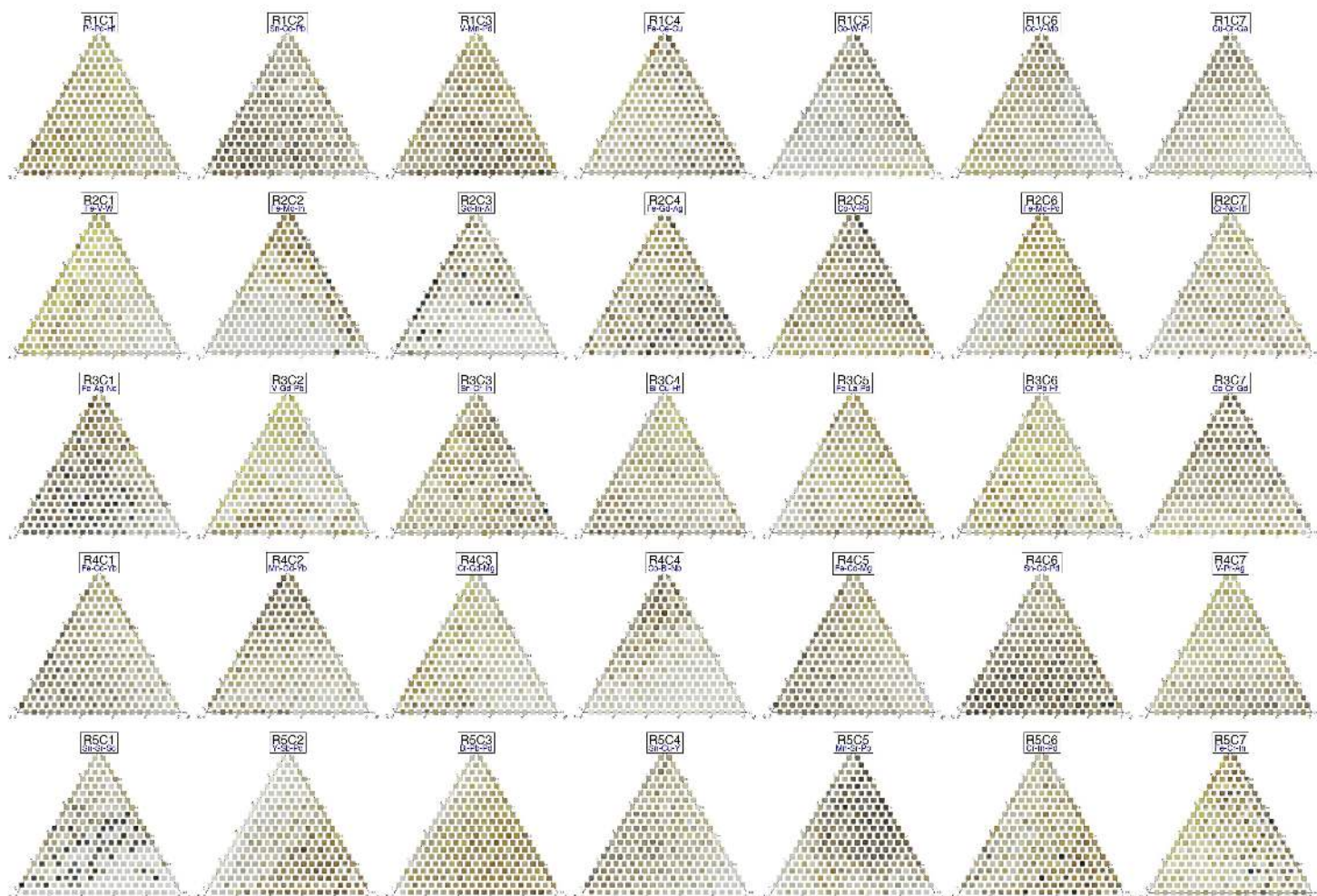
Idx	x_1	x_2	x_3	Idx	x_1	x_2	x_3	Idx	x_1	x_2	x_3	Idx	x_1	x_2	x_3
1	0	0	1.00	59	0.10	0.85	0.05	117	0.30	0.25	0.45	175	0.50	0.45	0.05
2	0	0.05	0.95	60	0.10	0.90	0	118	0.30	0.30	0.40	176	0.50	0.50	0
3	0	0.10	0.90	61	0.15	0	0.85	119	0.30	0.35	0.35	177	0.55	0	0.45
4	0	0.15	0.85	62	0.15	0.05	0.80	120	0.30	0.40	0.30	178	0.55	0.05	0.40
5	0	0.20	0.80	63	0.15	0.10	0.75	121	0.30	0.45	0.25	179	0.55	0.10	0.35
6	0	0.25	0.75	64	0.15	0.15	0.70	122	0.30	0.50	0.20	180	0.55	0.15	0.30
7	0	0.30	0.70	65	0.15	0.20	0.65	123	0.30	0.55	0.15	181	0.55	0.20	0.25
8	0	0.35	0.65	66	0.15	0.25	0.60	124	0.30	0.60	0.10	182	0.55	0.25	0.20
9	0	0.40	0.60	67	0.15	0.30	0.55	125	0.30	0.65	0.05	183	0.55	0.30	0.15
10	0	0.45	0.55	68	0.15	0.35	0.50	126	0.30	0.70	0	184	0.55	0.35	0.10
11	0	0.50	0.50	69	0.15	0.40	0.45	127	0.35	0	0.65	185	0.55	0.40	0.05
12	0	0.55	0.45	70	0.15	0.45	0.40	128	0.35	0.05	0.60	186	0.55	0.45	0
13	0	0.60	0.40	71	0.15	0.50	0.35	129	0.35	0.10	0.55	187	0.60	0	0.40
14	0	0.65	0.35	72	0.15	0.55	0.30	130	0.35	0.15	0.50	188	0.60	0.05	0.35
15	0	0.70	0.30	73	0.15	0.60	0.25	131	0.35	0.20	0.45	189	0.60	0.10	0.30
16	0	0.75	0.25	74	0.15	0.65	0.20	132	0.35	0.25	0.40	190	0.60	0.15	0.25
17	0	0.80	0.20	75	0.15	0.70	0.15	133	0.35	0.30	0.35	191	0.60	0.20	0.20
18	0	0.85	0.15	76	0.15	0.75	0.10	134	0.35	0.35	0.30	192	0.60	0.25	0.15
19	0	0.90	0.10	77	0.15	0.80	0.05	135	0.35	0.40	0.25	193	0.60	0.30	0.10
20	0	0.95	0.05	78	0.15	0.85	0	136	0.35	0.45	0.20	194	0.60	0.35	0.05
21	0	1.00	0	79	0.20	0	0.80	137	0.35	0.50	0.15	195	0.60	0.40	0
22	0.05	0	0.95	80	0.20	0.05	0.75	138	0.35	0.55	0.10	196	0.65	0	0.35
23	0.05	0.05	0.90	81	0.20	0.10	0.70	139	0.35	0.60	0.05	197	0.65	0.05	0.30
24	0.05	0.10	0.85	82	0.20	0.15	0.65	140	0.35	0.65	0	198	0.65	0.10	0.25
25	0.05	0.15	0.80	83	0.20	0.20	0.60	141	0.40	0	0.60	199	0.65	0.15	0.20
26	0.05	0.20	0.75	84	0.20	0.25	0.55	142	0.40	0.05	0.55	200	0.65	0.20	0.15
27	0.05	0.25	0.70	85	0.20	0.30	0.50	143	0.40	0.10	0.50	201	0.65	0.25	0.10
28	0.05	0.30	0.65	86	0.20	0.35	0.45	144	0.40	0.15	0.45	202	0.65	0.30	0.05
29	0.05	0.35	0.60	87	0.20	0.40	0.40	145	0.40	0.20	0.40	203	0.65	0.35	0
30	0.05	0.40	0.55	88	0.20	0.45	0.35	146	0.40	0.25	0.35	204	0.70	0	0.30
31	0.05	0.45	0.50	89	0.20	0.50	0.30	147	0.40	0.30	0.30	205	0.70	0.05	0.25
32	0.05	0.50	0.45	90	0.20	0.55	0.25	148	0.40	0.35	0.25	206	0.70	0.10	0.20
33	0.05	0.55	0.40	91	0.20	0.60	0.20	149	0.40	0.40	0.20	207	0.70	0.15	0.15
34	0.05	0.60	0.35	92	0.20	0.65	0.15	150	0.40	0.45	0.15	208	0.70	0.20	0.10
35	0.05	0.65	0.30	93	0.20	0.70	0.10	151	0.40	0.50	0.10	209	0.70	0.25	0.05
36	0.05	0.70	0.25	94	0.20	0.75	0.05	152	0.40	0.55	0.05	210	0.70	0.30	0
37	0.05	0.75	0.20	95	0.20	0.80	0	153	0.40	0.60	0	211	0.75	0	0.25
38	0.05	0.80	0.15	96	0.25	0	0.75	154	0.45	0	0.55	212	0.75	0.05	0.20
39	0.05	0.85	0.10	97	0.25	0.05	0.70	155	0.45	0.05	0.50	213	0.75	0.10	0.15
40	0.05	0.90	0.05	98	0.25	0.10	0.65	156	0.45	0.10	0.45	214	0.75	0.15	0.10
41	0.05	0.95	0	99	0.25	0.15	0.60	157	0.45	0.15	0.40	215	0.75	0.20	0.05
42	0.10	0	0.90	100	0.25	0.20	0.55	158	0.45	0.20	0.35	216	0.75	0.25	0
43	0.10	0.05	0.85	101	0.25	0.25	0.50	159	0.45	0.25	0.30	217	0.80	0	0.20
44	0.10	0.10	0.80	102	0.25	0.30	0.45	160	0.45	0.30	0.25	218	0.80	0.05	0.15
45	0.10	0.15	0.75	103	0.25	0.35	0.40	161	0.45	0.35	0.20	219	0.80	0.10	0.10
46	0.10	0.20	0.70	104	0.25	0.40	0.35	162	0.45	0.40	0.15	220	0.80	0.15	0.05
47	0.10	0.25	0.65	105	0.25	0.45	0.30	163	0.45	0.45	0.10	221	0.80	0.20	0
48	0.10	0.30	0.60	106	0.25	0.50	0.25	164	0.45	0.50	0.05	222	0.85	0	0.15
49	0.10	0.35	0.55	107	0.25	0.55	0.20	165	0.45	0.55	0	223	0.85	0.05	0.10
50	0.10	0.40	0.50	108	0.25	0.60	0.15	166	0.50	0	0.50	224	0.85	0.10	0.05
51	0.10	0.45	0.45	109	0.25	0.65	0.10	167	0.50	0.05	0.45	225	0.85	0.15	0
52	0.10	0.50	0.40	110	0.25	0.70	0.05	168	0.50	0.10	0.40	226	0.90	0	0.10
53	0.10	0.55	0.35	111	0.25	0.75	0	169	0.50	0.15	0.35	227	0.90	0.05	0.05
54	0.10	0.60	0.30	112	0.30	0	0.70	170	0.50	0.20	0.30	228	0.90	0.10	0
55	0.10	0.65	0.25	113	0.30	0.05	0.65	171	0.50	0.25	0.25	229	0.95	0	0.05
56	0.10	0.70	0.20	114	0.30	0.10	0.60	172	0.50	0.30	0.20	230	0.95	0.05	0
57	0.10	0.75	0.15	115	0.30	0.15	0.55	173	0.50	0.35	0.15	231	1.00	0	0
58	0.10	0.80	0.10	116	0.30	0.20	0.50	174	0.50	0.40	0.10				

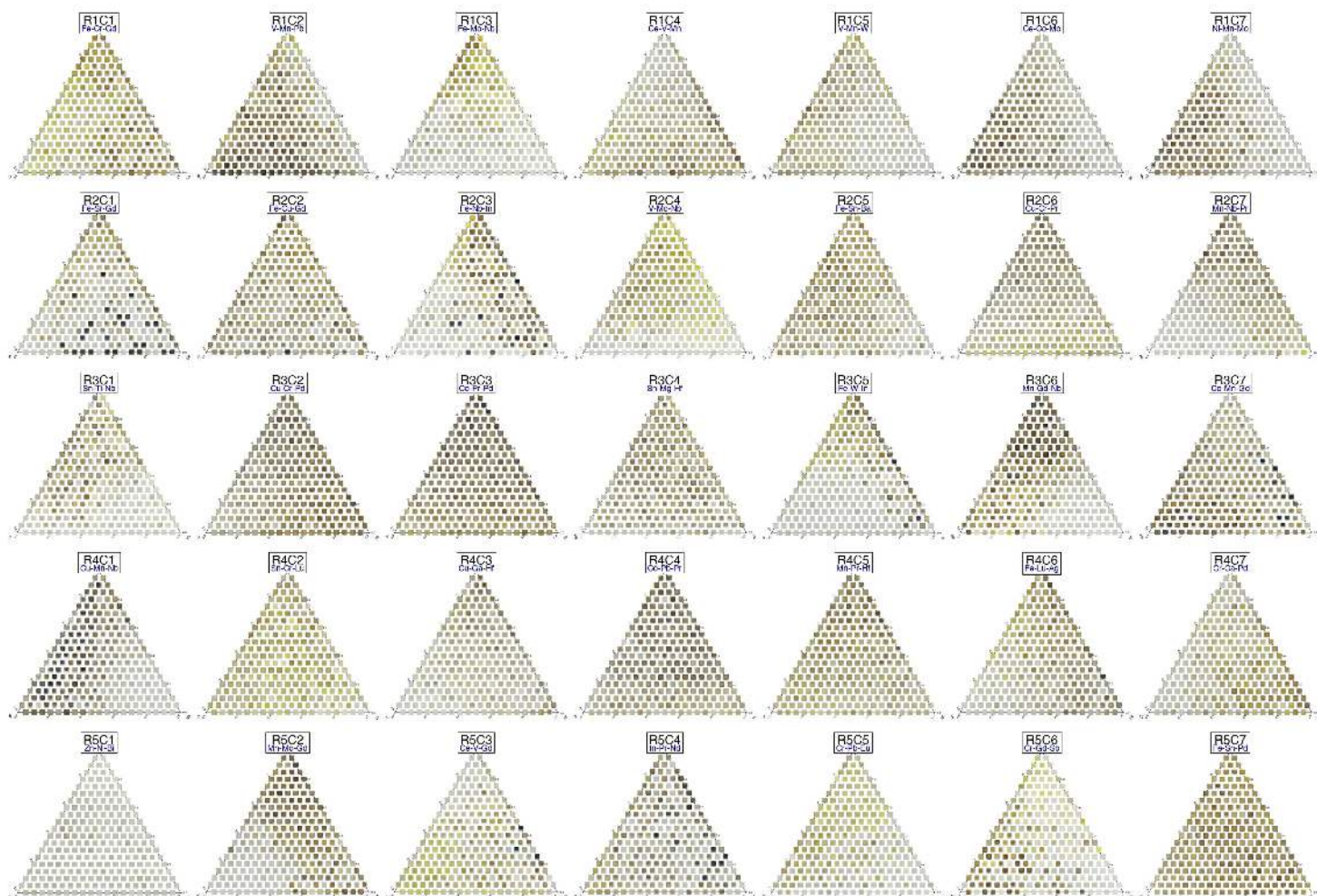
C.2 Interesting Composition Spaces

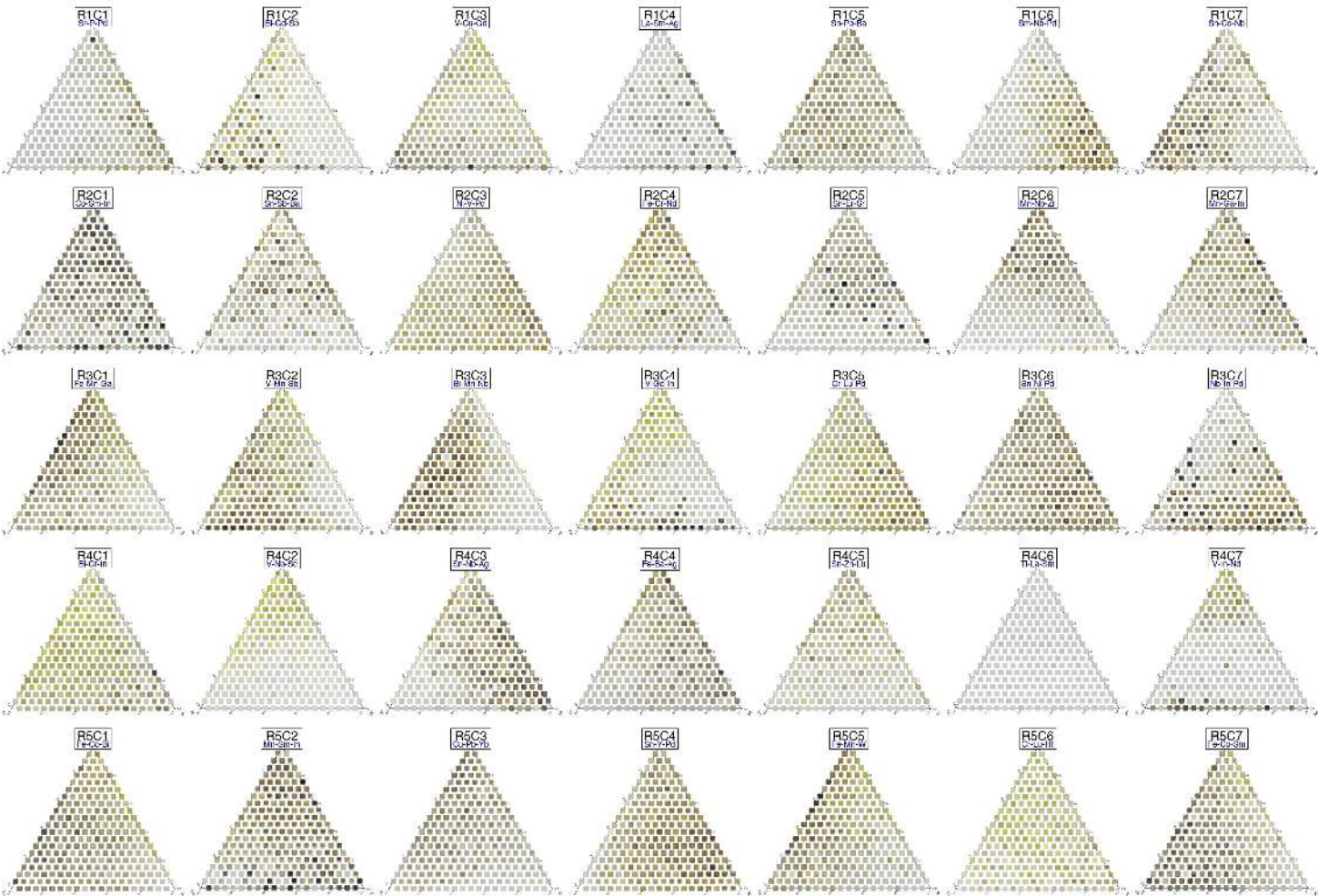
On the following pages, the composition spaces deemed interesting by a human expert are shown. Above each plot there is a nameplate containing the row and column number, to provide a coordinate system for look-up in a form of [Page, Row, Column]; and the three composition spaces

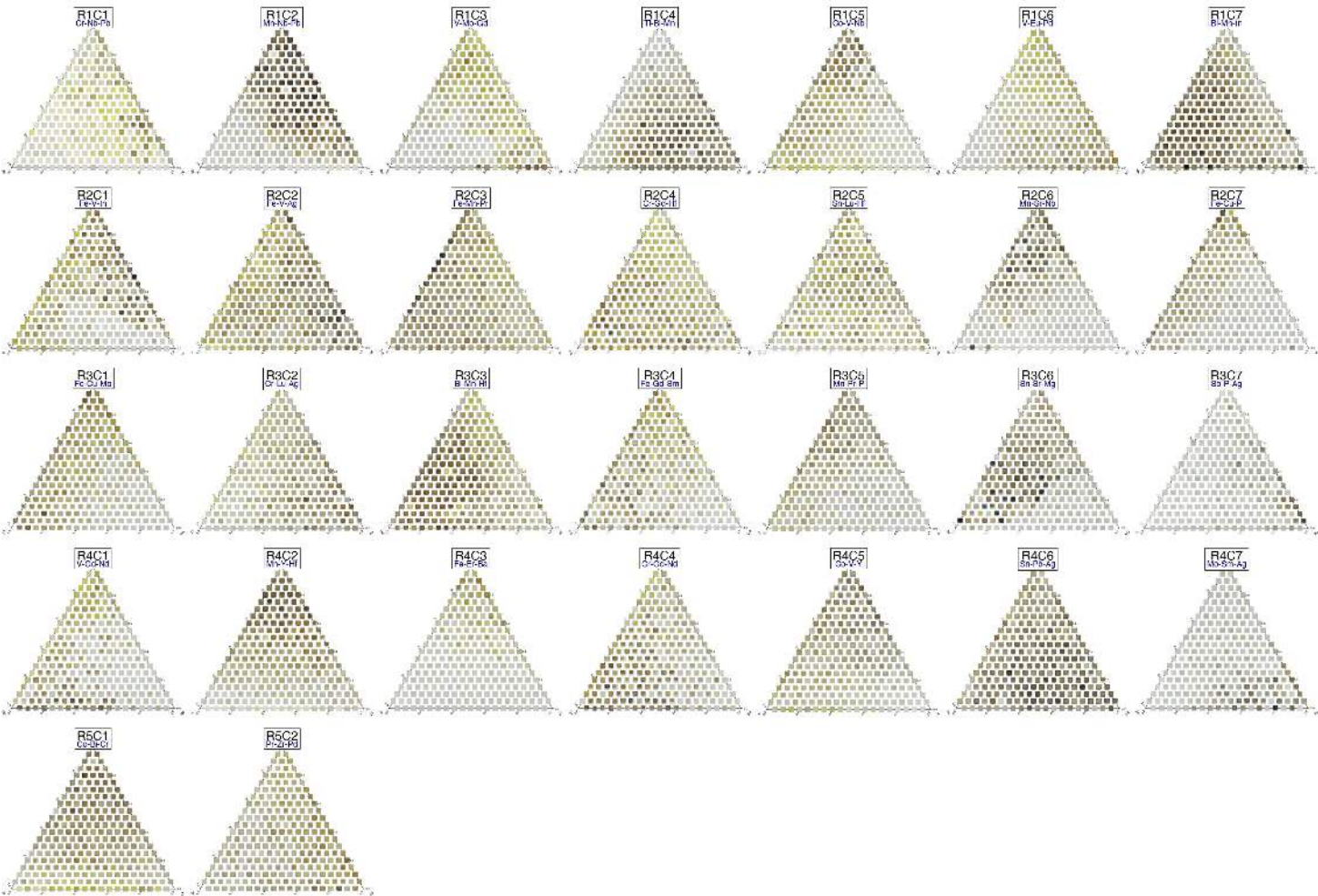












Declaration of Authorship

I hereby declare that this thesis is my own work, and that I have not used any sources and aids other than those stated in the thesis.

Signed: _____ Date: _____

Joachim Czapiński